

Programación Orientada a Objetos

[GUÍA RÁPIDA DE ESTUDIO]

📷 @garydevelop

GM

Índice

Parte 1 · Introducción a la programación orientada a objetos 6

1	Tu primer programa en Java	6
2	Tipos primitivos, variables y operadores	7
3	Condicionales y ciclos	8
4	Cadenas: String y StringBuilder	9
5	Arreglos	10
6	Métodos	11
7	Errores comunes del principiante	12
8	Clases y objetos	13
9	Constructores y la palabra this	14
10	Encapsulamiento: private, getters y setters	16
11	Métodos de instancia	18
12	Miembros static y constantes	19
13	Los métodos toString, equals y hashCode	21
14	Composición: objetos dentro de objetos	22
15	enum: constantes con nombre	24
16	record: clases de datos (Java 16+)	25
17	Paquetes, modificadores de acceso y la clase Object	26

Parte 2 · Introducción al manejo de colecciones 28

18	Genéricos, diamante y clases envoltorio	28
19	ArrayList	29
20	Recorrer colecciones	30
21	LinkedList, ArrayDeque, pilas y colas	31
22	HashSet, LinkedHashSet y TreeSet	32
23	HashMap, LinkedHashMap y TreeMap	33
24	Ordenar: Comparable y Comparator	34
25	Las clases Collections y Arrays	35
26	Lambdas y Streams básicos	36
27	PriorityQueue y elección de la colección	38

Parte 3 · Diseño orientado a objetos y UML 40

28	Del problema al diseño: clases, atributos y responsabilidades	40
29	Diagrama de clases: la caja y su traducción a Java	41
30	Relaciones: asociación, multiplicidad, agregación y composición	43
31	Herencia, realización y dependencia en UML	44
32	De UML a Java: la biblioteca completa	46
33	Casos de uso y diagramas de secuencia	47
34	Principios de buen diseño: cohesión, acoplamiento y SOLID	49

35	Patrones de diseño I: Singleton y Factory Method	51
36	Patrones de diseño II: Strategy y Observer	53

Parte 4 · Herencia, polimorfismo e interfaces 55

37	Herencia: extends, superclase y subclase	55
38	super y constructores en herencia	57
39	Sobrescritura de métodos y @Override	58
40	Control de acceso en herencia: protected y final	59
41	Polimorfismo: upcasting y enlace dinámico	60
42	instanceof, casting y pattern matching	61
43	Clases abstractas	63
44	Interfaces: declarar, implementar y lambdas	64
45	Abstracta vs interfaz y la clase Object	66
46	Interfaces selladas y diseño de jerarquías	67

Parte 5 · Manipulación de archivos y manejo de excepciones 69

47	Excepciones: try, catch y finally	69
48	Jerarquía de excepciones	70
49	Varios catch, multi-catch y finally	71
50	throw y throws	72
51	Excepciones propias	73
52	try-with-resources y AutoCloseable	74
53	Archivos y rutas	75
54	Leer y escribir texto	75
55	Procesar datos: CSV de estudiantes	77
56	Serialización y flujos binarios	78

Parte 6 · Programación con interfaces gráficas y manejo de eventos 81

57	Primera ventana Swing	81
58	Componentes básicos de Swing	82
59	Contenedores y paneles	84
60	Administradores de diseño (layouts)	86
61	Eventos: el modelo de delegación	88
62	Más eventos: ratón, teclado, ítems, texto y ventana	90
63	Diálogos y menús	92
64	Tablas y listas con modelo	93
65	Dibujo con Graphics	95
66	Arquitectura de una aplicación GUI: MVC	97

Parte 7 · Introducción a la programación concurrente 99

67	Concurrencia: procesos e hilos	99
68	Crear hilos	100

69	Ciclo de vida y control de hilos	101	73	ExecutorService y pools de hilos	108
70	Condiciones de carrera	103	74	CompletableFuture e hilos con GUI	109
71	Sincronización: synchronized, volatile y locks	105	75	Problemas clásicos: deadlock, starvation y wait/notify	111
72	Atomic y colecciones concurrentes	106			

Cómo usar esta guía

Una guía rápida de **Programación Orientada a Objetos con Java**: conceptos cortos, trucos para recordarlos y tablas con ejemplos de código cuyos resultados salen de **compilar y ejecutar el programa de verdad**.

1 · Lee el concepto

Dos líneas que explican de qué trata el tema, sin vueltas.

2 · Memoriza el truco

La caja **RECUERDA** trae una comparación fácil para no olvidar la idea.

3 · Prueba la tabla

Cada fila trae **elemento, descripción, ejemplo y resultado**. Cópialo en tu IDE y córralo tú mismo.

Cómo leer una tabla

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
ArrayList	Qué es y para qué sirve, en una frase.	<pre>List<String> l = new ArrayList<>(); l.add("Ana"); System.out.println(l.size());</pre>	1

Sobre el curso

- Programación Orientada a Objetos es una materia obligatoria de formación profesional. Requiere Fundamentos de Programación y se refuerza con laboratorios semanales y tareas de programación.
- El objetivo es aplicar los conceptos de la POO, hacer programas con interfaz gráfica y manejo de eventos, y diseñar jerarquías de clases escalables. La guía sigue el orden del sílabo; el texto base es *Java: Cómo programar* (10.ª ed.), de Deitel y Deitel.

Mapa de la guía

PARTE 1 Introducción a la programación orientada a objetos **pág. 6**

Primero la base de Java: programas, datos, decisiones, cadenas, arreglos y métodos. (17 temas)

PARTE 2 Introducción al manejo de colecciones **pág. 28**

Guardar, buscar y ordenar datos con listas, conjuntos, mapas y colas. (10 temas)

PARTE 3 Diseño orientado a objetos y UML **pág. 40**

Cómo pensar un programa antes de escribirlo: clases, diagramas UML y buenas prácticas. (9 temas)

PARTE 4 Herencia, polimorfismo e interfaces **pág. 55**

Reutilizar código con jerarquías de clases y tratar objetos distintos de forma uniforme. (10 temas)

PARTE 5 Manipulación de archivos y manejo de excepciones **pág. 69**

Qué hacer cuando algo falla, y cómo leer, escribir y guardar datos en disco. (10 temas)

PARTE 6 Programación con interfaces gráficas y manejo de eventos **pág. 81**

Ventanas, componentes y eventos con Swing: de la primera ventana al patrón MVC. (10 temas)

PARTE 7 Introducción a la programación concurrente **pág. 99**

Hacer varias cosas a la vez: hilos, sincronización y pools. (9 temas)

Antes de empezar

- Los resultados salieron de **ejecutar el código de verdad** con Java 21. En cada ejemplo ves solo las líneas importantes; si lo copias a tu IDE, ponlo dentro de `main` o de la clase que se indica.
- Las **clases de apoyo** que aparecen arriba de una tabla ya existen en todos los ejemplos del tema.
- Los nombres de clases van en `MayúsculaInicial`, los de métodos y variables en `minúscula`; así lo pide el estilo de Java.
- Compila y prueba tú mismo cada ejemplo: en POO se aprende escribiendo código, no solo leyéndolo.

Material de estudio no oficial, hecho por estudiantes para estudiar. No reemplaza las clases, el sílabo ni el material oficial de la materia.

Introducción a la programación orientada a objetos

Primero la base de Java: programas, datos, decisiones, cadenas, arreglos y métodos.

TEMA 1

Tu primer programa en Java

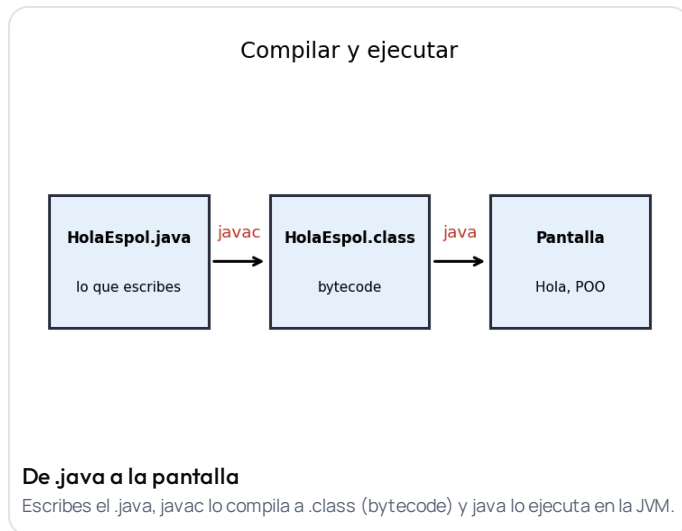
Todo programa Java vive dentro de una clase y arranca en el método main. Escribes el código en un archivo .java, lo compilas con javac (genera un .class) y lo ejecutas con java.

RECUERDA Piensa en una receta: la clase es el libro, main es el primer paso que el cocinero lee y println es servir el plato.

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Clase y main	La clase agrupa el código; la ejecución empieza en main.	<pre>public class HolaEspol { public static void main(String[] args) { System.out.println("Hola, P00"); } }</pre>	Hola, P00
print y println	println salta de línea al final; print no.	<pre>System.out.print("Hola "); System.out.print("Java"); System.out.println("!"); System.out.println("Fin");</pre>	Hola Java! Fin
printf	Imprime con formato: %s texto, %d entero, %.1f decimal, %n salto.	<pre>String n = "Ana"; int e = 20; System.out.printf("%s: %d%n", n, e); System.out.printf("Nota: %.1f%n", 8.46);</pre>	Ana: 20 Nota: 8.5
Secuencias de escape	\n salta línea, \t es tabulador, \" pone comillas.	<pre>System.out.println("Linea1\nLinea2"); System.out.println("A\tB"); System.out.println("Dijo \"holo\"");</pre>	Linea1 Linea2 A B Dijo "holo"
Comentarios	Java ignora lo que va después de // o entre /* y */.	<pre>// una línea System.out.println("Uno"); /* varias líneas */ System.out.println("Dos");</pre>	Uno Dos
Leer con Scanner	Scanner lee lo que el usuario escribe por teclado.	<pre>Scanner sc = new Scanner(System.in); System.out.println("Nombre:"); String n = sc.nextLine(); System.out.println("Edad:"); int e = sc.nextInt(); System.out.println(n + " tiene " + e);</pre>	Nombre: Edad: Ana tiene 20
Compilar y ejecutar	javac crea el .class; java lo corre (sin escribir .class).	<pre>javac HolaEspol.java java HolaEspol</pre>	(compila sin mensajes) Hola, P00
Nombre del archivo	El .java debe llamarse igual que su clase public.	<pre>// Archivo: Hola.java public class HolaEspol { }</pre>	error: class HolaEspol is public, should be declared in a file named HolaEspol.java
Unir con +	El + junta texto con números y otros textos.	<pre>int anio = 2026; System.out.println("ESPOL " + anio); System.out.println("Suma: " + 2 + 3); System.out.println("Suma: " + (2 + 3));</pre>	ESPOL 2026 Suma: 23 Suma: 5

Para abreviar, en el resto de la guía las filas muestran solo el interior de main, salvo cuando se declara una clase o un método.

Diagramas y ventanas de ejemplo



TEMA 2

Tipos primitivos, variables y operadores

Una variable es una caja con nombre y tipo. Los tipos primitivos (int, double, char, boolean...) guardan valores simples. Con final creas constantes y con casting conviertes entre tipos.

RECUERDA Cada tipo es una caja de tamaño fijo: un int no cabe en un byte, pero un byte sí cabe en un int.

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Tipos primitivos	int enteros, double decimales, char letra, boolean true o false.	<pre>int edad = 20; double nota = 8.5; char letra = 'A'; boolean pasa = true; System.out.println(edad + " " + nota); System.out.println(letra + " " + pasa);</pre>	20 8.5 A true
Rango de los tipos	Cada tipo tiene un máximo; las constantes MAX_VALUE lo dicen.	<pre>System.out.println(Byte.MAX_VALUE); System.out.println(Integer.MAX_VALUE); System.out.println(Long.MAX_VALUE);</pre>	127 2147483647 9223372036854775807
Constante con final	final impide cambiar el valor; se escribe EN_MAYUSCULAS.	<pre>final double IVA = 0.15; double precio = 200; System.out.println(precio * IVA);</pre>	30.0
División entera y resto	Con enteros, / descarta decimales y % da el resto.	<pre>System.out.println(17 / 5); System.out.println(17 % 5); System.out.println(17 / 5.0);</pre>	3 2 3.4
Incremento y +=	n++ suma 1; += suma y guarda en la misma variable.	<pre>int n = 5; n++; n += 10; System.out.println(n); System.out.println(n++ + " " + n);</pre>	16 16 17
Conversión automática	Un tipo pequeño pasa solo a uno más grande.	<pre>int cantidad = 7; double d = cantidad; long grande = cantidad; System.out.println(d); System.out.println(grande);</pre>	7.0 7
Casting explícito	(int) convierte a entero y corta los decimales.	<pre>double d = 9.99; int i = (int) d; System.out.println(i); System.out.println((double) 7 / 2); System.out.println((char) 67);</pre>	9 3.5 C

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Desbordamiento	Pasarse del máximo da la vuelta, sin avisar.	<pre>int max = Integer.MAX_VALUE; System.out.println(max + 1); byte b = (byte) 130; System.out.println(b);</pre>	-2147483648 -126
Comparar y combinar	Comparar da boolean; && es Y, es O, ! niega.	<pre>int e = 20; System.out.println(e >= 18 && e < 65); System.out.println(e < 18 e == 20); System.out.println(!(e == 20));</pre>	true true false

TEMA 3

Condicionales y ciclos

Las condicionales (if, switch) eligen qué código se ejecuta. Los ciclos (for, while, do-while, for-each) repiten código. Con break sales del ciclo y con continue saltas a la siguiente vuelta.

RECUERDA if es un semáforo que decide el camino; un ciclo es dar vueltas a una pista hasta que la condición diga basta.

ESTRUCTURA	DESCRIPCIÓN	EJEMPLO	RESULTADO
if / else if / else	Elige un camino según condiciones, de arriba hacia abajo.	<pre>double nota = 7.5; if (nota >= 9) { System.out.println("Excelente"); } else if (nota >= 7) { System.out.println("Aprobado"); } else { System.out.println("Reprobado"); }</pre>	Aprobado
switch clásico	Compara un valor con cada case; break evita seguir de largo.	<pre>int dia = 2; String r; switch (dia) { case 1: r = "Lun"; break; case 2: r = "Mar"; break; default: r = "Otro"; } System.out.println(r);</pre>	Mar
switch sin break	Sin break, la ejecución cae en los case siguientes.	<pre>int n = 1; switch (n) { case 1: System.out.print("A"); case 2: System.out.print("B"); break; case 3: System.out.print("C"); }</pre>	AB
switch con flecha	La forma moderna: →, sin break y devuelve un valor.	<pre>int dia = 6; String tipo = switch (dia) { case 6, 7 -> "Fin de semana"; default -> "Entre semana"; }; System.out.println(tipo);</pre>	Fin de semana
for	Para repetir un número conocido de veces.	<pre>for (int i = 1; i <= 3; i++) { System.out.println("Vuelta " + i); }</pre>	Vuelta 1 Vuelta 2 Vuelta 3
while	Repite mientras la condición sea verdadera.	<pre>int n = 3; while (n > 0) { System.out.println("Quedan " + n); n--; }</pre>	Quedan 3 Quedan 2 Quedan 1
do-while	Revisa la condición al final: corre al menos una vez.	<pre>int x = 10; do { System.out.println("Corre una vez"); } while (x < 5);</pre>	Corre una vez

ESTRUCTURA	DESCRIPCIÓN	EJEMPLO	RESULTADO
for-each	Recorre cada elemento de un arreglo o lista.	<pre>String[] m = {"P00", "Redes", "Física"}; for (String x : m) { System.out.println(x); }</pre>	P00 Redes Física
break y continue	continue salta esta vuelta; break termina el ciclo.	<pre>for (int i = 1; i <= 6; i++) { if (i == 2) continue; if (i == 5) break; System.out.println(i); }</pre>	1 3 4

TEMA 4

Cadenas: String y StringBuilder

Un String es un texto que no se puede modificar (inmutable): sus métodos devuelven un texto nuevo. Para comparar contenido usa equals, no ==. StringBuilder sirve para armar textos con muchos cambios.

RECUERDA Un String es como una foto impresa: para cambiarla haces otra. Un StringBuilder es una pizarra: borras y escribes encima.

MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
String es immutable	Los métodos no cambian el texto: devuelven uno nuevo.	<pre>String s = "ESPOL"; s.toLowerCase(); System.out.println(s); s = s.toLowerCase(); System.out.println(s);</pre>	ESPOL espol
length, charAt, indexOf	Largo, letra en una posición y dónde aparece un texto.	<pre>String s = "Guayaquil"; System.out.println(s.length()); System.out.println(s.charAt(0)); System.out.println(s.indexOf("q"));</pre>	9 G 5
trim, upper, substring	Quitar espacios, pasar a mayúsculas y cortar.	<pre>String s = " Hola Java "; String t = s.trim(); System.out.println(t.toUpperCase()); System.out.println(t.substring(5)); System.out.println(t.substring(0, 4));</pre>	HOLA JAVA Java Hol
equals vs ==	== compara referencias; equals compara el contenido.	<pre>String a = new String("java"); String b = new String("java"); System.out.println(a == b); System.out.println(a.equals(b)); boolean r = a.equalsIgnoreCase("JAVA"); System.out.println(r);</pre>	false true true
contains, endsWith, replace	Buscar y reemplazar dentro del texto.	<pre>String f = "notas.txt"; System.out.println(f.endsWith(".txt")); System.out.println(f.contains("nota")); String g = f.replace("notas", "lista"); System.out.println(g);</pre>	true true lista.txt
split y join	Partir un texto en trozos y volver a unirlos.	<pre>String[] p = "Ana,Luis,Eva".split(","); System.out.println(p.length); System.out.println(p[1]); System.out.println(String.join("-", p));</pre>	3 Luis Ana-Luis-Eva
StringBuilder en ciclos	append agrega sin crear un texto nuevo cada vez.	<pre>StringBuilder sb = new StringBuilder(); for (int i = 1; i <= 5; i++) { sb.append(i).append(","); } sb.setLength(sb.length() - 1); System.out.println(sb);</pre>	1,2,3,4,5

MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
insert, reverse, delete	StringBuilder sí modifica su contenido.	<pre>StringBuilder sb = new StringBuilder(); sb.append("Java"); sb.insert(0, "Hola "); System.out.println(sb); sb.reverse(); System.out.println(sb); System.out.println(sb.deleteCharAt(0));</pre>	<pre>Hola Java avaJ aloH vaJ aloH</pre>
String.format	Arma un texto con el mismo formato de printf.	<pre>String t = String.format("%-6s %5d %7.2f", "Ana", 7, 3.14); System.out.println(t); String n = String.format("%05d", 42); System.out.println(n);</pre>	<pre>Ana 7 3.14 00042</pre>

TEMA 5

Arreglos

Un arreglo guarda varios valores del mismo tipo en posiciones numeradas desde 0. Su tamaño es fijo y se consulta con `length` (sin paréntesis). La clase `Arrays` trae herramientas para imprimir, ordenar y llenar.

RECUERDA Un arreglo es una fila de casilleros numerados desde 0: el primero es el 0 y el último es `length - 1`.

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Crear con new	Reserva el espacio; los números empiezan en 0.	<pre>int[] notas = new int[3]; notas[0] = 8; notas[1] = 9; System.out.println(notas[1]); System.out.println(notas[2]);</pre>	<pre>9 0</pre>
Con llaves y length	Crea el arreglo con valores; <code>length</code> da el tamaño.	<pre>String[] dias = {"Lun", "Mar", "Mie"}; System.out.println(dias.length); int u = dias.length - 1; System.out.println(dias[u]);</pre>	<pre>3 Mie</pre>
Recorrer con for	El índice <code>i</code> va de 0 a <code>length - 1</code> .	<pre>int[] n = {8, 6, 9, 7}; int suma = 0; for (int i = 0; i < n.length; i++) { suma += n[i]; } double prom = suma / (double) n.length; System.out.println(prom);</pre>	<pre>7.5</pre>
Arrays.toString	Convierte el arreglo en texto legible para imprimir.	<pre>int[] n = {5, 2, 9}; System.out.println(Arrays.toString(n));</pre>	<pre>[5, 2, 9]</pre>
Arrays.sort	Ordena de menor a mayor.	<pre>int[] n = {5, 2, 9, 1}; Arrays.sort(n); System.out.println(Arrays.toString(n));</pre>	<pre>[1, 2, 5, 9]</pre>
Arrays.fill y copyOf	Llena con un valor y copia con otro tamaño.	<pre>int[] a = new int[4]; Arrays.fill(a, 7); int[] b = Arrays.copyOf(a, 6); System.out.println(Arrays.toString(b));</pre>	<pre>[7, 7, 7, 7, 0, 0]</pre>
Asignar no copia	<code>b = a</code> apunta al mismo arreglo, no hace una copia.	<pre>int[] a = {1, 2, 3}; int[] b = a; b[0] = 99; System.out.println(a[0]);</pre>	<pre>99</pre>
Arreglo 2D	Un arreglo de arreglos: <code>m[fila][columna]</code> .	<pre>int[][] m = {{1, 2, 3}, {4, 5, 6}}; System.out.println(m.length); System.out.println(m[0].length); System.out.println(m[1][2]);</pre>	<pre>2 3 6</pre>

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Recorrer un 2D	Un for-each por filas y otro por columnas.	<pre>int[][] m = {{1, 2, 3}, {4, 5, 6}}; for (int[] fila : m) { for (int v : fila) { System.out.print(v); } System.out.println(); }</pre>	123 456

TEMA 6

Métodos

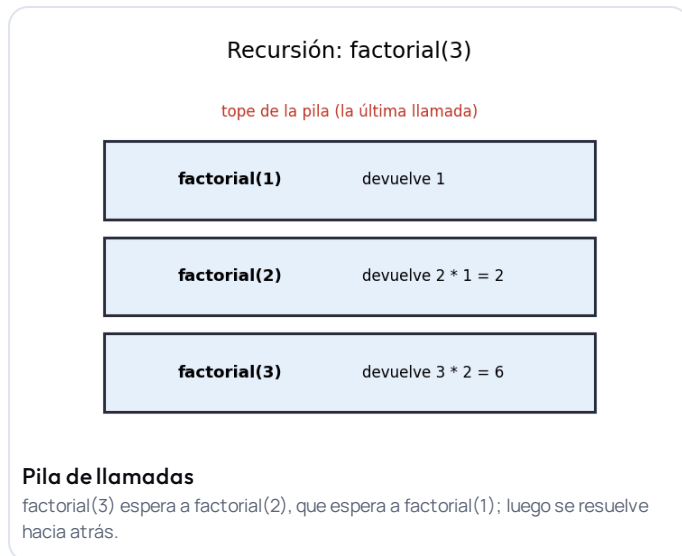
Un método es un bloque de código con nombre que recibe parámetros y puede devolver un valor con return. Java pasa los argumentos por valor: el método recibe una copia. Un método static se llama sin crear objetos.

RECUERDA Un método es una máquina de café: metes ingredientes (parámetros), pulsas el botón (llamada) y sale el café (return).

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Método void	Hace algo y no devuelve nada.	<pre>static void saludar(String nom) { System.out.println("Hola " + nom); }</pre>	Hola Ana Hola Luis
Parámetros y return	Recibe datos y devuelve un resultado con return.	<pre>static double area(double b, double h) { return b * h; }</pre>	10.0
Sobrecarga	Mismo nombre, distintos parámetros: Java elige el correcto.	<pre>static int suma(int a, int b) { return a + b; } static double suma(double a, double b) { return a + b; }</pre>	5 5.5
Varargs	Con ... recibe cuántos argumentos quieras (llega como arreglo).	<pre>static int total(int... nums) { int t = 0; for (int n : nums) { t += n; } return t; }</pre>	0 5 10
Método static	Pertenece a la clase: se llama con Clase.metodo().	<pre>class Util { static int doble(int n) { return n * 2; } }</pre>	42
Métodos de Math	Java trae métodos static listos para cálculos.	<pre>System.out.println(Math.max(3, 8)); System.out.println(Math.sqrt(49)); System.out.println(Math.pow(2, 10)); System.out.println(Math.abs(-5));</pre>	8 7.0 1024.0 5
Paso por valor	El método recibe una copia: la variable original no cambia.	<pre>static void cambiar(int x) { x = 99; }</pre>	5
Arreglos como argumento	Se copia la referencia: el método sí puede cambiar el contenido.	<pre>static void llenar(int[] a) { a[0] = 99; }</pre>	99
Recursión	Un método que se llama a sí mismo hasta un caso base.	<pre>static int factorial(int n) { if (n <= 1) { return 1; } return n * factorial(n - 1); }</pre>	120

En estos ejemplos se omite public delante de static para abreviar; funciona igual dentro de una clase.

Diagramas y ventanas de ejemplo



TEMA 7

Errores comunes del principiante

Hay dos clases de error. Los de compilación los detecta javac antes de ejecutar y el programa no arranca. Los de ejecución (excepciones) aparecen con el programa corriendo y lo detienen si nadie los atiende.

RECUERDA El compilador es el corrector que revisa tu examen antes de entregarlo; una excepción es un problema que surge en plena exposición.

ERROR	DESCRIPCIÓN	EJEMPLO	RESULTADO
Falta el punto y coma	Cada sentencia termina en ;. Error de compilación.	<code>int x = 5 System.out.println(x);</code>	error: ';' expected
Tipo incompatible	No se puede guardar un texto en un int.	<code>int edad = "veinte";</code>	error: incompatible types: String cannot be converted to int
Variable no declarada	Usar un nombre que no existe (o mal escrito).	<code>System.out.println(total);</code>	error: cannot find symbol symbol: variable total
Pérdida de precisión	Un double no entra solo en un int: hay que castear.	<code>int n = 3.7;</code>	error: incompatible types: possible lossy conversion from double to int
Variable sin valor	Se debe asignar un valor antes de usar la variable.	<code>int x; System.out.println(x);</code>	error: variable x might not have been initialized
División entre cero	Con enteros lanza ArithmeticException al ejecutar.	<code>int a = 10; int b = 0; System.out.println(a / b);</code>	ArithmeticException: / by zero
Índice fuera de rango	Un arreglo de 3 solo tiene posiciones 0, 1 y 2.	<code>int[] a = new int[3]; a[3] = 1;</code>	ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3
Usar un null	Llamar un método sobre una referencia null.	<code>String s = null; System.out.println(s.length());</code>	NullPointerException: Cannot invoke "String.length()" because "s" is null
Texto que no es número	parseInt falla si el texto no es un entero.	<code>int n = Integer.parseInt("abc");</code>	NumberFormatException: For input string: "abc"

En la fila del null, el nombre s aparece al compilar con javac -g o desde un IDE; con javac a secas muestra <local>.

Clases y objetos

Una clase es el molde: dice qué datos (atributos) y qué acciones (métodos) tendrán sus objetos. Un objeto es una copia concreta que se fabrica con new. De un molde salen muchos objetos, cada uno con sus propios datos.

RECUERDA Clase = molde de galletas; objeto = cada galleta. Hay un solo molde, pero puedes hacer muchas galletas y cada una con su sabor.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

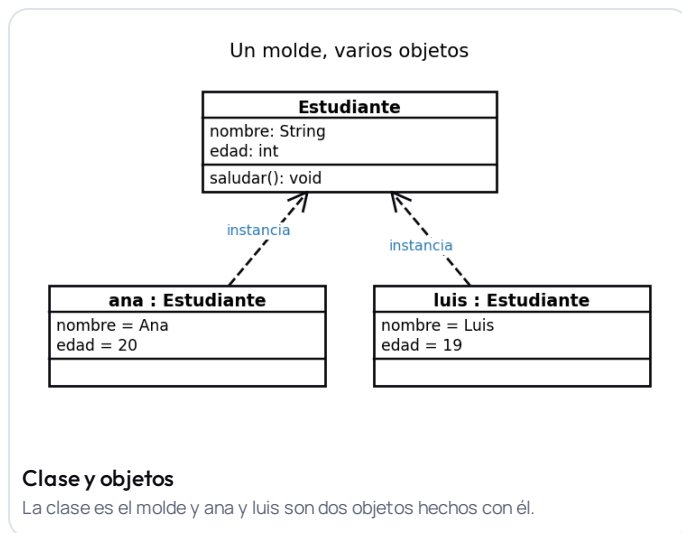
```
class Estudiante {
    String nombre;
    int edad;
    double promedio;

    void saludar() {
        System.out.println("Hola " + nombre);
    }
}
```

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
class	Declara el molde. Los atributos llevan su tipo, como en una variable.	<pre>class Libro { String titulo; int paginas; }</pre>	Java 1100
new	Fabrica un objeto de la clase. Devuelve una referencia a él.	<pre>Estudiante e = new Estudiante(); e.nombre = "Ana"; System.out.println(e.nombre);</pre>	Ana
Operador punto	Con el punto entras a un atributo del objeto, para leer o cambiar.	<pre>Estudiante e = new Estudiante(); e.edad = 20; e.edad = e.edad + 1; System.out.println(e.edad);</pre>	21
Valores por defecto	Un atributo sin dar valor se inicia solo: null, 0, 0.0 o false.	<pre>Estudiante e = new Estudiante(); System.out.println(e.nombre); System.out.println(e.edad); System.out.println(e.promedio);</pre>	null 0 0.0
Método	Una acción del objeto. Se llama con el punto y paréntesis.	<pre>Estudiante e = new Estudiante(); e.nombre = "Luis"; e.saludar();</pre>	Hola Luis
Objetos independientes	Cada new crea un objeto con sus propios datos.	<pre>Estudiante a = new Estudiante(); Estudiante b = new Estudiante(); a.nombre = "Ana"; b.nombre = "Luis"; System.out.println(a.nombre);</pre>	Ana
Referencias	La variable guarda una flecha al objeto. Copiarla no copia el objeto.	<pre>Estudiante a = new Estudiante(); Estudiante b = a; b.nombre = "Luis"; System.out.println(a.nombre);</pre>	Luis
null	Una referencia que no apunta a ningún objeto. Se puede comparar con ==.	<pre>Estudiante e = null; System.out.println(e == null);</pre>	true
Usar null	Llamar a un método de null falla con NullPointerException (se captura aquí).	<pre>Estudiante e = null; try { e.saludar(); } catch (NullPointerException x) { System.out.println("Era null"); }</pre>	Era null

En Python el `__init__` crea los atributos; en Java se declaran en la clase, con su tipo.

Diagramas y ventanas de ejemplo



TEMA 9

Constructores y la palabra this

El constructor es un método especial que se ejecuta al hacer new y deja el objeto listo. Tiene el nombre de la clase y no lleva tipo de retorno. La palabra this es el objeto actual: sirve para distinguir atributo de parámetro.

RECUERDA El constructor es el formulario de inscripción: lo llenas al crear al estudiante. this es decir "yo mismo".

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class Producto {
    String nombre;
    double precio;

    Producto(String nombre, double precio) {
        this.nombre = nombre;
        this.precio = precio;
    }
}
```

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Constructor	Mismo nombre de la clase, sin tipo de retorno. Recibe los datos iniciales.	<code>Producto p = new Producto("Lapiz", 2.5);</code> <code>System.out.println(p.nombre);</code> <code>System.out.println(p.precio);</code>	Lapiz 2.5
this.atributo	Si el parámetro se llama igual que el atributo, this.x es el atributo.	<pre>class Libro { String titulo; Libro(String titulo) { this.titulo = titulo; } }</pre>	Java
Olvidar this	Sin this el parámetro se asigna a sí mismo y el atributo queda en null.	<pre>class Libro { String titulo; Libro(String titulo) { titulo = titulo; } }</pre>	null
Constructor por defecto	Si no escribes ninguno, Java crea uno vacío que no recibe datos.	<pre>class Curso { String nombre; }</pre>	null
Se pierde el vacío	Al escribir un constructor propio, el vacío ya no existe.	<code>Producto p = new Producto();</code>	error: constructor Producto in class Producto cannot be applied to given types;

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Sobrecarga	Varios constructores con distintos parámetros. Python usa valores por defecto.	<pre> class Cuenta { String titular; double saldo; Cuenta(String t) { titular = t; } Cuenta(String t, double s) { titular = t; saldo = s; } } </pre>	0.0 50.0
this(...)	Un constructor llama a otro de la misma clase. Debe ser la primera línea.	<pre> class Cuenta { String titular; double saldo; Cuenta(String t) { this(t, 10.0); } Cuenta(String t, double s) { titular = t; saldo = s; } } </pre>	10.0
Constructor de copia	Recibe un objeto del mismo tipo y crea otro con los mismos datos.	<pre> class Libro { String titulo; Libro(String t) { titulo = t; } Libro(Libro otro) { this(otro.titulo); } } </pre>	Java P00
return this	this también se puede devolver para encadenar llamadas.	<pre> class Contador { int n; Contador sumar() { n++; return this; } } </pre>	3

Encapsulamiento: private, getters y setters

Encapsular es esconder los atributos con `private` y dejar entrar solo por métodos: `getX` para leer y `setX` para cambiar. Así el setter puede validar y el objeto nunca queda con datos absurdos, como un saldo negativo.

RECUERDA Como el cajero del banco: no metes la mano a la bóveda (`private`); pides el dinero por la ventanilla (métodos) y él revisa.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class Cuenta {
    private String titular;
    private double saldo;

    Cuenta(String titular, double saldo) {
        this.titular = titular;
        this.saldo = saldo;
    }
    String getTitular() { return titular; }
    double getSaldo() { return saldo; }
    void setSaldo(double saldo) {
        if (saldo >= 0) {
            this.saldo = saldo;
        }
    }
    void depositar(double monto) {
        if (monto > 0) {
            saldo += monto;
        }
    }
}
```

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
private	Solo la propia clase ve el atributo. Desde fuera, el compilador lo impide.	<code>Cuenta c = new Cuenta("Ana", 100);</code> <code>System.out.println(c.saldo);</code>	error: saldo has private access in Cuenta
Getter	Método público que devuelve el valor. Se llama <code>getNombre()</code> .	<code>Cuenta c = new Cuenta("Ana", 100);</code> <code>System.out.println(c.getSaldo());</code>	100.0
Setter válido	Método que cambia el atributo. Aquí acepta valores desde 0.	<code>Cuenta c = new Cuenta("Ana", 100);</code> <code>c.setSaldo(250);</code> <code>System.out.println(c.getSaldo());</code>	250.0
Setter que valida	Si el valor no sirve, el setter lo ignora y el saldo no cambia.	<code>Cuenta c = new Cuenta("Ana", 100);</code> <code>c.setSaldo(-50);</code> <code>System.out.println(c.getSaldo());</code>	100.0
Solo lectura	Sin setter, el valor solo se puede consultar. Aquí el titular.	<code>Cuenta c = new Cuenta("Ana", 100);</code> <code>System.out.println(c.getTitular());</code>	Ana
Operación en vez de setter	Un método con nombre de acción protege mejor la regla del negocio.	<code>Cuenta c = new Cuenta("Ana", 100);</code> <code>c.depositar(50);</code> <code>c.depositar(-20);</code> <code>System.out.println(c.getSaldo());</code>	150.0
isX para boolean	Si el atributo es boolean, el getter se suele llamar <code>isX</code> .	<pre>class Curso { private boolean abierto = true; boolean isAbierto() { return abierto; } }</pre>	true

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Validar en el constructor	El constructor también debe respetar las reglas del objeto.	<pre>class Nota { private double valor; Nota(double v) { valor = v < 0 ? 0 : v; } double getValor() { return valor; } }</pre>	0.0
Getter calculado	Un getter puede calcular el dato; no hace falta guardarlo.	<pre>class Rectangulo { private double base = 4; private double altura = 3; double getArea() { return base * altura; } }</pre>	12.0

En Python el guion bajo (`_saldo`) es solo una costumbre; en Java `private` lo obliga el compilador.

Diagramas y ventanas de ejemplo

Datos privados, métodos públicos

Cuenta
- titular: String
- saldo: double
+ getSaldo(): double
+ setSaldo(double): void
+ depositar(double): void

Cuenta encapsulada

Menos (-) es private, más (+) es público.

Métodos de instancia

Un método de instancia trabaja con los datos de un objeto concreto. Puede recibir parámetros y devolver un valor con `return`, o ser `void` si no devuelve nada. Hay métodos que cambian el estado y otros que solo consultan.

RECUERDA Dos tipos de botón en una máquina: unos hacen algo (depositar, cambia el saldo) y otros solo muestran (consultar saldo).

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class Alumno {
    String nombre;
    double nota1, nota2;
    int faltas;

    Alumno(String n, double a, double b) {
        nombre = n;
        nota1 = a;
        nota2 = b;
    }
    double promedio() {
        return (nota1 + nota2) / 2;
    }
    void registrarFalta() {
        faltas++;
    }
    String resumen() {
        return nombre + ": " + promedio();
    }
}
```

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Con retorno	Devuelve un valor con <code>return</code> . El tipo va antes del nombre.	<code>Alumno e = new Alumno("Ana", 8, 9);</code> <code>System.out.println(e.promedio());</code>	8.5
void (cambia el estado)	No devuelve nada; modifica los atributos del objeto.	<code>Alumno e = new Alumno("Ana", 8, 9);</code> <code>e.registrarFalta();</code> <code>e.registrarFalta();</code> <code>System.out.println(e.faltas);</code>	2
Consulta	Solo lee: llamarlo dos veces da lo mismo y el objeto no cambia.	<code>Alumno e = new Alumno("Luis", 6, 9);</code> <code>System.out.println(e.promedio());</code> <code>System.out.println(e.promedio());</code>	7.5 7.5
Con parámetros	Los parámetros llevan su tipo. El orden de los argumentos importa.	<pre>class Cuenta { double saldo; void depositar(double monto) { saldo += monto; } }</pre>	125.5
Devuelve un String	Un método puede armar y devolver texto con datos del objeto.	<code>Alumno e = new Alumno("Ana", 8, 9);</code> <code>System.out.println(e.resumen());</code>	Ana: 8.5
Un método llama a otro	Dentro de la clase se llama a otro método sin punto.	<pre>class Nota { double a = 8, b = 6; double prom() { return (a + b) / 2; } boolean aprobo() { return prom() >= 7; } }</pre>	7.0 true

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Sobrecarga	Mismo nombre y distintos parámetros: Java elige según los argumentos.	<pre>class Calc { int sumar(int a, int b) { return a + b; } double sumar(double a, double b) { return a + b; } }</pre>	5 4.0
Paso por valor	Un parámetro int es una copia: cambiarlo no toca la variable original.	<pre>class Prueba { void subir(int x) { x = x + 5; } }</pre>	10
void no se imprime	Un método void no da valor: usarlo como valor no compila.	<pre>Alumno e = new Alumno("Ana", 8, 9); System.out.println(e.registrarFalta());</pre>	error: 'void' type not allowed here

TEMA 12

Miembros static y constantes

Un miembro static pertenece a la clase, no a cada objeto: hay una sola copia compartida. Sirve para contadores, constantes (static final) y métodos de utilidad que se llaman con el nombre de la clase, sin new.

RECUERDA static es el letrero de la entrada del edificio: es uno solo y lo ven todos. Los atributos normales son la puerta de cada oficina.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class Producto {
    static int cantidad = 0;
    static final double IVA = 0.15;
    String nombre;
    double precio;

    Producto(String nombre, double precio) {
        this.nombre = nombre;
        this.precio = precio;
        cantidad++;
    }
    static double conIva(double p) {
        return p + p * IVA;
    }
}
```

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Atributo static	Una sola copia compartida por todos los objetos. Aquí cuenta productos.	<pre>new Producto("Lapiz", 0.5); new Producto("Regla", 1.2); new Producto("Libro", 20); System.out.println(Producto.cantidad);</pre>	3
Acceso con la clase	Un miembro static se usa con NombreClase.miembro, sin crear objetos.	<pre>System.out.println(Producto.IVA);</pre>	0.15
static final	Constante: un valor fijo. Por convención va EN_MAYUSCULAS.	<pre>Producto.IVA = 0.12;</pre>	error: cannot assign a value to static final variable IVA
Método static	Se llama sin objeto. Aquí suma el IVA a un precio.	<pre>double t = Producto.conIva(100); System.out.println(t);</pre>	115.0

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
static no ve lo de instancia	Un método static no tiene this, así que no puede usar atributos normales.	<pre>class Util { String x = "a"; static void f() { System.out.println(x); } }</pre>	error: non-static variable x cannot be referenced from a static context
Métodos de utilidad	Math, Integer y String ofrecen métodos static listos para usar.	<pre>System.out.println(Math.max(3, 9)); System.out.println(Math.sqrt(49)); int n = Integer.parseInt("42"); System.out.println(n + 1);</pre>	9 7.0 43
Constantes de la librería	Math.PI e Integer.MAX_VALUE son static final.	<pre>System.out.println(Math.PI); System.out.println(Integer.MAX_VALUE);</pre>	3.141592653589793 2147483647
Contador de ids	Un static da números seguidos: cada objeto toma el siguiente.	<pre>class Alumno { static int siguiente = 1; int id; Alumno() { id = siguiente++; } }</pre>	1 2 3
Bloque static	Se ejecuta una sola vez, cuando la clase se carga por primera vez.	<pre>class Config { static String nombre; static { nombre = "ESPOL"; } }</pre>	ESPOL

En Python, un atributo definido en la clase (fuera de `__init__`) y `@staticmethod` se parecen a static.

Los métodos toString, equals y hashCode

Todos los objetos heredan toString, equals y hashCode, pero las versiones por defecto no sirven: imprimen un código raro y comparan referencias. Se sobrescriben (@Override) para mostrar y comparar por contenido.

RECUERDA toString es la tarjeta de presentación del objeto. equals decide si dos tarjetas son de la misma persona. hashCode es su número de casillero.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class Libro {
    String titulo;
    Libro(String t) { titulo = t; }
    @Override
    public String toString() {
        return titulo;
    }
}

classCodigo {
    String valor;
    Codigo(String v) { valor = v; }
    @Override
    public boolean equals(Object o) {
        return o instanceof Codigo c
            && c.valor.equals(valor);
    }
    @Override
    public int hashCode() {
        return Objects.hash(valor);
    }
}
```

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
toString por defecto	Imprime NombreClase@codigo: no sirve para leer. Evítalo.	<code>Object o = new Object(); System.out.println(o);</code>	java.lang.Object@ seguido de un código hexadecimal que cambia
toString propio	Con @Override devuelves el texto que quieras mostrar.	<pre>class Curso { String nombre = "P00"; @Override public String toString() { return "Curso: " + nombre; } }</pre>	Curso: P00
Se llama solo	println y la concatenación con + usan toString automáticamente.	<code>Libro l = new Libro("Java"); System.out.println(l); System.out.println("Leo: " + l);</code>	Java Leo: Java
== y equals por defecto	== compara referencias; el equals heredado hace lo mismo.	<code>Libro a = new Libro("Java"); Libro b = new Libro("Java"); Libro c = a; System.out.println(a == b); System.out.println(a.equals(b)); System.out.println(a == c);</code>	false false true
equals propio	Se sobrescribe para comparar el contenido (aquí el título).	<pre>class Libro { String titulo; Libro(String t) { titulo = t; } @Override public boolean equals(Object o) { return o instanceof Libro l && l.titulo.equals(titulo); } }</pre>	true false

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
String: == contra equals	Con textos, == compara referencias; equals compara letras. No es como Python.	<pre>String a = new String("hola"); String b = new String("hola"); System.out.println(a == b); System.out.println(a.equals(b));</pre>	false true
hashCode	Objetos iguales deben dar el mismo hashCode. Objects.hash lo calcula.	<pre>Codigo a = new Codigo("A1"); Codigo b = new Codigo("A1"); System.out.println(a.equals(b)); int h1 = a.hashCode(); int h2 = b.hashCode(); System.out.println(h1 == h2);</pre>	true true
equals + hashCode juntos	Con ambos, HashSet detecta los repetidos.	<pre>Set<Codigo> s = new HashSet<>(); s.add(new Codigo("A1")); s.add(new Codigo("A1")); System.out.println(s.size());</pre>	1
Solo equals: error típico	Sin hashCode, un HashSet suele guardar los dos "iguales".	<pre>class Libro { String titulo; Libro(String t) { titulo = t; } @Override public boolean equals(Object o) { return o instanceof Libro l && l.titulo.equals(titulo); } }</pre>	2

TEMA 14

Composición: objetos dentro de objetos

Un objeto puede tener otros objetos como atributos: es la relación "tiene un". Un Alumno tiene una Dirección. Los objetos se pasan por referencia: un método que recibe un objeto puede cambiar el original.

RECUERDA Un auto tiene un motor, pero no es un motor. Cuando una pieza pertenece a otra, se arma un objeto con objetos adentro.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class Direccion {
    String ciudad;
    Direccion(String c) { ciudad = c; }
}

class Alumno {
    String nombre;
    Direccion direccion;

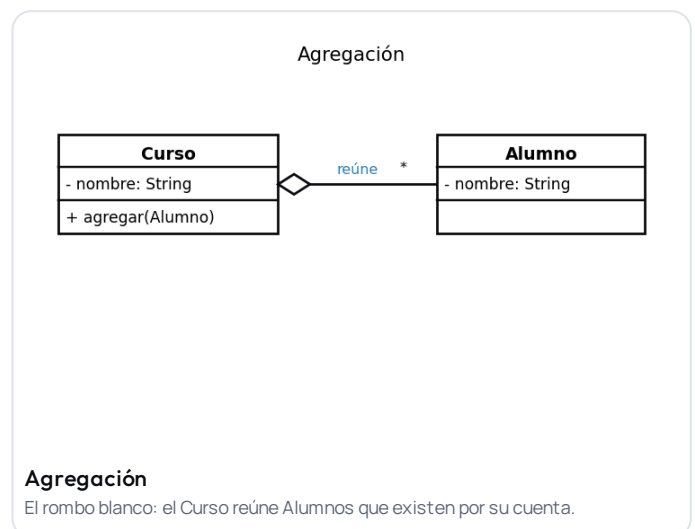
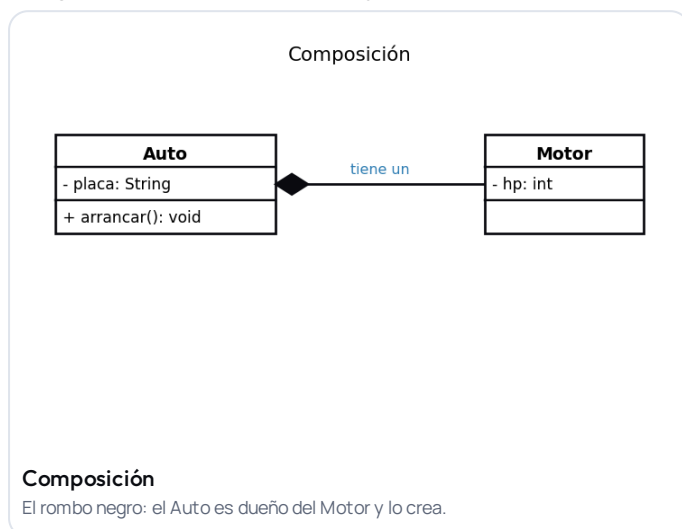
    Alumno(String nombre, Direccion d) {
        this.nombre = nombre;
        direccion = d;
    }
}
```

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Tiene un	Un atributo cuyo tipo es otra clase. Se arma primero la pieza.	<pre>Direccion d = new Direccion("Gye"); Alumno e = new Alumno("Ana", d); System.out.println(e.nombre); System.out.println(e.direccion.ciudad);</pre>	Ana Gye
Navegar con puntos	Se encadenan puntos: estudiante, su dirección y su ciudad.	<pre>Alumno e = new Alumno("Ana", new Direccion("Guayaquil")); System.out.println(e.direccion.ciudad); int n = e.direccion.ciudad.length(); System.out.println(n);</pre>	Guayaquil 9

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Pieza creada adentro	Composición fuerte: el Auto crea su propio Motor con new.	<pre>class Motor { int hp = 120; } class Auto { Motor motor = new Motor(); }</pre>	120
Componente null	Si no se asigna la pieza, vale null. Revisalo antes de usarla.	<pre>Alumno e = new Alumno("Ana", null); if (e.direccion == null) { System.out.println("Sin direccion"); }</pre>	Sin direccion
Arreglo de objetos	Se crea lleno de null: faltan los new de cada posición.	<pre>Alumno[] g = new Alumno[3]; System.out.println(g[0]); System.out.println(g.length);</pre>	null 3
Llenar y recorrer	Se crean los objetos y se recorre con for-each.	<pre>Direccion d = new Direccion("Gye"); Alumno[] g = { new Alumno("Ana", d), new Alumno("Luis", d) }; for (Alumno e : g) { System.out.println(e.nombre); }</pre>	Ana Luis
Objeto como parámetro	El método recibe la referencia: cambia el objeto original.	<pre>static void mudar(Alumno e) { e.direccion.ciudad = "Quito"; }</pre>	Quito
Reasignar el parámetro	Apuntar el parámetro a otro objeto no cambia la variable de fuera.	<pre>static void cambiar(Alumno e) { e = new Alumno("Otro", null); }</pre>	Ana
Pieza compartida	Dos objetos pueden apuntar a la misma pieza: un cambio lo ven ambos.	<pre>Direccion d = new Direccion("Gye"); Alumno a = new Alumno("Ana", d); Alumno b = new Alumno("Luis", d); a.direccion.ciudad = "Duran"; System.out.println(b.direccion.ciudad);</pre>	Duran

Composición: la pieza nace y muere con el todo (Auto-Motor). Agregación: la pieza existe aparte (Curso-Alumno).

Diagramas y ventanas de ejemplo



enum: constantes con nombre

Un enum es un tipo con un conjunto fijo de valores: los días, los niveles, las categorías. Es más seguro que usar números o textos sueltos. Puede tener campos, constructor y métodos, y funciona muy bien con switch.

RECUERDA Un enum es el menú desplegable de un formulario: solo puedes elegir las opciones que ya están en la lista.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
enum Dia {
    LUNES, MARTES, MIERCOLES
}
```

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Declarar enum	Se listan las constantes en mayúsculas. Se usan como Tipo.CONSTANTE.	<pre>enum Semaforo { ROJO, AMARILLO, VERDE }</pre>	ROJO
Comparar con ==	Cada constante es única, así que se compara con == sin problema.	<pre>Dia d = Dia.LUNES; System.out.println(d == Dia.LUNES); System.out.println(d == Dia.MARTES);</pre>	true false
values()	Devuelve un arreglo con todas las constantes, en el orden declarado.	<pre>for (Dia d : Dia.values()) { System.out.println(d); }</pre>	LUNES MARTES MIERCOLES
valueOf()	Convierte un texto exacto en la constante del mismo nombre.	<pre>Dia d = Dia.valueOf("MARTES"); System.out.println(d);</pre>	MARTES
valueOf inválido	Si el nombre no existe lanza IllegalArgumentException.	<pre>try { Dia.valueOf("DOMINGO"); } catch (IllegalArgumentException e) { System.out.println("No existe"); }</pre>	No existe
name() y ordinal()	name() da el texto; ordinal() la posición, que empieza en 0.	<pre>Dia d = Dia.MIERCOLES; System.out.println(d.name()); System.out.println(d.ordinal());</pre>	MIERCOLES 2
Campos y constructor	Cada constante pasa datos al constructor del enum.	<pre>enum Categoria { BASICA(0), SOCIO(10), VIP(25); final int pct; Categoria(int p) { pct = p; } }</pre>	10 VIP
Métodos en el enum	Un enum puede tener métodos que usan sus campos.	<pre>enum Categoria { BASICA(0), SOCIO(10), VIP(25); final int pct; Categoria(int p) { pct = p; } double aplicar(double p) { return p - p * pct / 100; } }</pre>	60.0 80.0
switch con enum	En los case se escribe solo el nombre de la constante, sin el tipo.	<pre>Dia d = Dia.MARTES; String t = switch (d) { case LUNES -> "Inicio"; case MARTES -> "Clases"; default -> "Otro"; }; System.out.println(t);</pre>	Clases

record: clases de datos (Java 16+)

Un record es una clase corta para guardar datos que no cambian. Con una sola línea Java crea el constructor, los accesores, toString, equals y hashCode. Se parece a un dataclass(frozen=True) de Python.

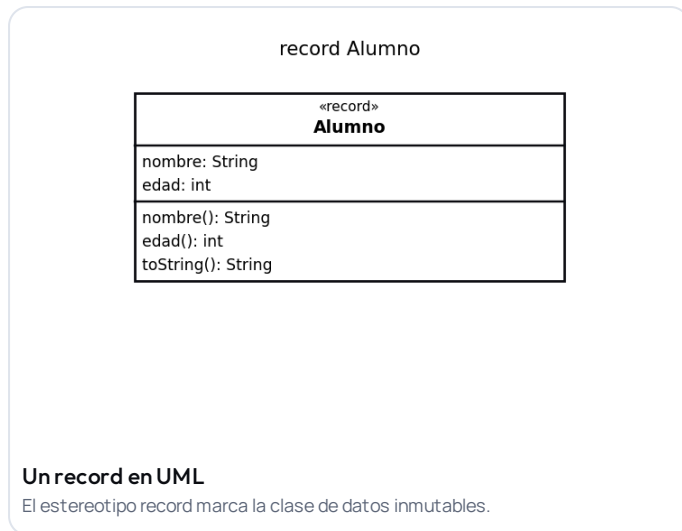
RECUERDA Un record es una ficha de datos sellada: la llenas una vez al crearla y ya no se modifica. Ahorra escribir toda la rutina.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
record Alumno(String nombre, int edad) {
}
```

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Declarar y crear	Los componentes van entre paréntesis. toString viene hecho.	<pre>Alumno e = new Alumno("Ana", 20); System.out.println(e);</pre>	Alumno[nombre=Ana, edad=20]
Accesores	Se llaman como el componente: nombre(), sin la palabra get.	<pre>Alumno e = new Alumno("Ana", 20); System.out.println(e.nombre()); System.out.println(e.edad());</pre>	Ana 20
equals y hashCode	Dos records con los mismos valores son iguales, sin escribir nada.	<pre>Alumno a = new Alumno("Ana", 20); Alumno b = new Alumno("Ana", 20); System.out.println(a.equals(b)); System.out.println(a == b); int h1 = a.hashCode(); int h2 = b.hashCode(); System.out.println(h1 == h2);</pre>	true false true
En un HashSet	Como equals y hashCode ya funcionan, el HashSet quita repetidos.	<pre>Set<Alumno> s = new HashSet<>(); s.add(new Alumno("Ana", 20)); s.add(new Alumno("Ana", 20)); System.out.println(s.size());</pre>	1
Immutable	No hay setters. Los componentes se fijan al crear y no cambian.	<pre>Alumno e = new Alumno("Ana", 20); e.setEdad(21);</pre>	error: cannot find symbol symbol: method setEdad(int) location: variable e of type Alumno
Constructor compacto	Sin paréntesis ni this: valida o ajusta los datos antes de guardarlos.	<pre>public record Nota(double valor) { public Nota { if (valor < 0) { valor = 0; } } }</pre>	0.0 8.5
Métodos propios	Un record puede tener métodos que usan sus componentes.	<pre>record Rect(double base, double alto) { double area() { return base * alto; } }</pre>	12.0 Rect[base=3.0, alto=4.0]
Constructor adicional	Un constructor extra debe llamar al canónico con this(...).	<pre>record Libro(String titulo, int pags) { Libro(String titulo) { this(titulo, 0); } }</pre>	Libro[titulo=Java, pags=0] Libro[titulo=P00, pags=300]
Record frente a clase	Una línea de record reemplaza unas 20 líneas de una clase.	<pre>record Punto(int x, int y) { }</pre>	Java crea por ti: constructor, x(), y(), toString, equals y hashCode

Diagramas y ventanas de ejemplo



TEMA 17

Paquetes, modificadores de acceso y la clase Object

Los paquetes agrupan clases como carpetas, y import trae las que están en otro paquete. Los modificadores public, protected, sin modificador y private deciden quién ve cada miembro. Toda clase hereda de Object.

RECUERDA Paquete = carpeta de la facultad. Los modificadores son las llaves: public abre a todos, private solo al dueño. Object es el abuelo de todos.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class Estudiante {
    String nombre;
    Estudiante(String n) { nombre = n; }
    @Override
    public String toString() {
        return "Estudiante " + nombre;
    }
}
```

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
package	Primera línea del archivo: dice a qué paquete pertenece. Sigue las carpetas.	<code>package ec.espol.poo;</code>	Archivo en la carpeta ec/espol/poo/Cuenta.java
import	Trae una clase de otro paquete. java.lang (String, Math) viene incluido.	<code>import java.util.ArrayList; ArrayList<String> l = new ArrayList<>(); l.add("Ana"); System.out.println(l);</code>	[Ana]
public	Lo ve cualquier clase de cualquier paquete.	<code>public String nombre;</code>	Lo ve: todas las clases
protected	Lo ve su paquete y las subclases (herencia), aunque estén en otro paquete.	<code>protected String nombre;</code>	Lo ve: su paquete y las subclases
Sin modificador	Es el acceso de paquete (package-private): solo clases del mismo paquete.	<code>String nombre;</code>	Lo ve: las clases de su paquete
private	Lo ve solo la propia clase. Es lo normal para los atributos.	<code>private String nombre;</code>	Lo ve: solo su propia clase

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Todo hereda de Object	Sin extends, tu clase hereda de Object: toString, equals, hashCode, getClass.	<pre>class Perro { }</pre>	true Perro
Object guarda de todo	Una variable Object acepta cualquier objeto, aun números y textos.	<pre>Object[] cosas = {"hola", 5, 2.5, new Estudiante("Ana")}; for (Object o : cosas) { System.out.println(o); }</pre>	hola 5 2.5 Estudiante Ana
instanceof	Pregunta el tipo real del objeto y, si coincide, lo convierte en una variable.	<pre>Object o = new Estudiante("Ana"); if (o instanceof Estudiante e) { System.out.println(e.nombre); }</pre>	Ana

Resumen: *public* = todos · *protected* = paquete y subclases · *sin modificador* = paquete · *private* = solo la clase.

Introducción al manejo de colecciones

Guardar, buscar y ordenar datos con listas, conjuntos, mapas y colas.

TEMA 18

Genéricos, diamante y clases envoltorio

Un genérico `<T>` es un tipo que se completa al usarlo: `List<String>` es una lista que solo guarda `String`. El compilador revisa los tipos y evita errores al ejecutar. Las colecciones solo guardan objetos, por eso existen las clases envoltorio (`Integer`, `Double`...).

RECUERDA Un genérico es una caja con etiqueta: la caja de `String` solo recibe `String`, y el compilador te avisa si intentas meter otra cosa.

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
List<String>	Lista que solo acepta <code>String</code> ; no hace falta convertir al sacar.	<pre>List<String> lista = new ArrayList<>(); lista.add("Ana"); lista.add("Luis"); String primero = lista.get(0); System.out.println(primero);</pre>	Ana
Error de tipo	Con genéricos el error sale al compilar, no al ejecutar.	<pre>List<String> lista = new ArrayList<>(); lista.add(5);</pre>	error: incompatible types: int cannot be converted to String
List crudo (sin <T>)	Sin tipo todo entra y el error aparece al ejecutar. Evítalo.	<pre>List crudo = new ArrayList(); crudo.add("Ana"); crudo.add(5); String s = (String) crudo.get(1);</pre>	ClassCastException: Integer no se puede convertir a String
Operador diamante <>	A la derecha basta <code><></code> : Java deduce el tipo de la izquierda.	<pre>Map<String, List<Integer>> notas = new HashMap<>(); notas.put("Ana", new ArrayList<>()); notas.get("Ana").add(9); System.out.println(notas);</pre>	{Ana=[9]}
Clase genérica	Una clase con <code><T></code> sirve para cualquier tipo.	<pre>class Caja<T> { private T contenido; void guardar(T x) { contenido = x; } T obtener() { return contenido; } }</pre>	Ana 8
Método genérico	<code><T></code> antes del tipo de retorno: el método acepta listas de cualquier tipo.	<pre>static <T> void mostrar(List<T> l) { for (T x : l) { System.out.println(x); } }</pre>	Ana Luis 1 2
Clases envoltorio	Cada primitivo tiene su clase: <code>int</code> - <code>Integer</code> , <code>double</code> - <code>Double</code> , <code>char</code> - <code>Character</code> .	<pre>Integer a = Integer.valueOf(7); Double d = Double.parseDouble("8.5"); System.out.println(a + " " + d); int x = Integer.parseInt("42"); System.out.println(x + 1);</pre>	7 8.5 43
Autoboxing	Java convierte solo entre <code>int</code> e <code>Integer</code> al meter y al sacar.	<pre>List<Integer> notas = new ArrayList<>(); notas.add(9); int n = notas.get(0); System.out.println(n + 1);</pre>	10
No con primitivos	<code>List<int></code> no existe: se usa <code>List<Integer></code> .	<pre>List<int> notas = new ArrayList<>();</pre>	error: unexpected type required: reference found: int

Ojo: con `Integer` compara con `equals()`, no con `==`. En Python no hay genéricos: la lista acepta cualquier cosa.

ArrayList

ArrayList es la lista más usada: guarda elementos en orden, permite repetidos y crece sola. Se accede por posición (empieza en 0). Es la lista de Python, pero con un solo tipo de dato.

RECUERDA Un ArrayList es una fila de casilleros numerados desde 0 que se agrandan solos cuando se llenan.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class Curso {
    static ArrayList<String> nombres() {
        var l = new ArrayList<String>();
        l.add("Ana");
        l.add("Luis");
        l.add("Eva");
        return l;
    }
}
```

MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
new ArrayList<>() y add	Crea la lista vacía y agrega al final. En Python: lista.append(x).	<pre>ArrayList<String> l = new ArrayList<>(); l.add("Ana"); l.add("Luis"); System.out.println(l); System.out.println(l.size());</pre>	[Ana, Luis] 2
add(i, x)	Inserta en la posición i y corre el resto a la derecha.	<pre>ArrayList<String> l = Curso.nombres(); l.add(1, "Zoe"); System.out.println(l);</pre>	[Ana, Zoe, Luis, Eva]
get(i)	Lee la posición i. En Python: lista[i].	<pre>ArrayList<String> l = Curso.nombres(); System.out.println(l.get(0)); System.out.println(l.get(2));</pre>	Ana Eva
set(i, x)	Reemplaza la posición i y devuelve el valor anterior.	<pre>ArrayList<String> l = Curso.nombres(); String antes = l.set(1, "Beto"); System.out.println(antes); System.out.println(l);</pre>	Luis [Ana, Beto, Eva]
remove(i) y remove(x)	Con int quita por posición; con objeto, quita ese valor.	<pre>ArrayList<Integer> n = new ArrayList<>(); n.add(10); n.add(20); n.add(30); n.remove(1); n.remove(Integer.valueOf(10)); System.out.println(n);</pre>	[30]
size() e isEmpty()	Cantidad de elementos. En Python: len(lista).	<pre>ArrayList<String> l = Curso.nombres(); System.out.println(l.size()); System.out.println(l.isEmpty());</pre>	3 false
contains e indexOf	Busca un valor. indexOf devuelve -1 si no está.	<pre>ArrayList<String> l = Curso.nombres(); System.out.println(l.contains("Eva")); System.out.println(l.indexOf("Luis")); System.out.println(l.indexOf("Zoe"));</pre>	true 1 -1
clear()	Vacía la lista. En Python: lista.clear().	<pre>ArrayList<String> l = Curso.nombres(); l.clear(); System.out.println(l); System.out.println(l.isEmpty());</pre>	[] true
Posición inexistente	get(5) en una lista de 3 lanza excepción.	<pre>ArrayList<String> l = Curso.nombres(); System.out.println(l.get(5));</pre>	IndexOutOfBoundsException: Index 5 out of bounds for length 3

Curso.nombres() devuelve una lista nueva [Ana, Luis, Eva] en cada fila.

Recorrer colecciones

Hay varias formas de recorrer una colección: for-each (la más simple), for con índice, Iterator (permite borrar sin error) y forEach con lambda. Si modificas la lista mientras la recorres con for-each, Java lanza ConcurrentModificationException.

RECUERDA Es como pasar lista en clase: no puedes echar a un alumno a mitad del conteo sin avisar; el Iterator sí sabe sacarlo bien.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class Curso {
    static ArrayList<String> nombres() {
        var l = new ArrayList<String>();
        l.add("Ana");
        l.add("Luis");
        l.add("Eva");
        return l;
    }
}
```

FORMA	DESCRIPCIÓN	EJEMPLO	RESULTADO
for-each	Recorre todos los elementos. En Python: for n in lista.	<pre>for (String n : Curso.nombres()) { System.out.println(n); }</pre>	Ana Luis Eva
for con índice	Cuando necesitas la posición. En Python: enumerate.	<pre>ArrayList<String> l = Curso.nombres(); for (int i = 0; i < l.size(); i++) { String n = l.get(i); System.out.println(i + " " + n); }</pre>	0 Ana 1 Luis 2 Eva
Al revés	Se empieza en size() - 1 y se baja hasta 0.	<pre>ArrayList<String> l = Curso.nombres(); int ult = l.size() - 1; for (int i = ult; i >= 0; i--) { System.out.println(l.get(i)); }</pre>	Eva Luis Ana
Iterator	hasNext() pregunta si hay más; next() entrega el siguiente.	<pre>ArrayList<String> l = Curso.nombres(); Iterator<String> it = l.iterator(); while (it.hasNext()) { System.out.println(it.next()); }</pre>	Ana Luis Eva
Iterator.remove()	Borra con seguridad mientras recorres.	<pre>ArrayList<String> l = Curso.nombres(); Iterator<String> it = l.iterator(); while (it.hasNext()) { if (it.next().startsWith("L")) { it.remove(); } } System.out.println(l);</pre>	[Ana, Eva]
removeIf	Borra todo lo que cumpla la condición (lambda).	<pre>ArrayList<String> l = Curso.nombres(); l.removeIf(n -> n.length() == 3); System.out.println(l);</pre>	[Luis]
forEach con lambda	Ejecuta una acción por cada elemento.	<pre>ArrayList<String> l = Curso.nombres(); l.forEach(n -> System.out.println("Hola " + n));</pre>	Hola Ana Hola Luis Hola Eva
Referencia a método	System.out::println es un atajo de la lambda.	<pre>ArrayList<String> l = Curso.nombres(); l.forEach(System.out::println);</pre>	Ana Luis Eva
ConcurrentModificationException	Borrar dentro de un for-each rompe el recorrido.	<pre>ArrayList<String> l = Curso.nombres(); for (String n : l) { if (n.equals("Ana")) { l.remove(n); } }</pre>	ConcurrentModificationException

Curso.nombres() devuelve [Ana, Luis, Eva]. Para borrar mientras recorres usa Iterator.remove() o removeIf.

LinkedList, ArrayDeque, pilas y colas

Una cola (Queue) atiende en orden de llegada: el primero que entra es el primero que sale (FIFO). Una pila (Deque con push y pop) es al revés: el último que entra sale primero (LIFO). ArrayDeque y LinkedList implementan ambas.

RECUERDA Cola = fila del banco: sale el que llegó primero. Pila = platos apilados: usas el de arriba, el último que pusiste.

OPERACIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
Queue: offer, poll, peek	offer agrega al final; poll saca el primero; peek lo mira.	<pre>Queue<String> cola = new LinkedList<>(); cola.offer("Ana"); cola.offer("Luis"); cola.offer("Eva"); System.out.println(cola.poll()); System.out.println(cola.peek()); System.out.println(cola);</pre>	Ana Luis [Luis, Eva]
Cola vacía	poll y peek devuelven null, no lanzan error.	<pre>Queue<String> cola = new ArrayDeque<>(); System.out.println(cola.poll()); System.out.println(cola.peek()); System.out.println(cola.isEmpty());</pre>	null null true
Deque como pila	push apila arriba, pop saca la de arriba. En Python: append y pop.	<pre>Deque<String> pila = new ArrayDeque<>(); pila.push("Ana"); pila.push("Luis"); pila.push("Eva"); System.out.println(pila.pop()); System.out.println(pila.peek()); System.out.println(pila);</pre>	Eva Luis [Luis, Ana]
pop en pila vacía	pop lanza excepción si no hay nada.	<pre>Deque<String> pila = new ArrayDeque<>(); System.out.println(pila.pop());</pre>	NoSuchElementException
LinkedList	Es List y Deque a la vez: agrega y quita en ambos extremos.	<pre>LinkedList<String> l = new LinkedList<>(); l.add("Luis"); l.addFirst("Ana"); l.addLast("Eva"); System.out.println(l); System.out.println(l.removeFirst());</pre>	[Ana, Luis, Eva] Ana
ArrayDeque por ambos lados	offerFirst/offerLast y pollFirst/pollLast.	<pre>Deque<Integer> d = new ArrayDeque<>(); d.offerLast(2); d.offerFirst(1); d.offerLast(3); System.out.println(d); System.out.println(d.pollLast());</pre>	[1, 2, 3] 3
Stack (clase antigua)	Funciona, pero se prefiere ArrayDeque. Imprime de abajo a arriba.	<pre>Stack<String> s = new Stack<>(); s.push("Ana"); s.push("Luis"); System.out.println(s); System.out.println(s.pop());</pre>	[Ana, Luis] Luis
Uso de la pila	Revisar paréntesis: se apila (, se desapila).	<pre>Deque<Character> p = new ArrayDeque<>(); for (char c : "(()())".toCharArray()) { if (c == '(') { p.push(c); } else { p.pop(); } } System.out.println(p.isEmpty());</pre>	true
¿Cuál uso?	Regla práctica para escoger.	<pre>Cola o pila -> ArrayDeque List y Deque -> LinkedList Pila de Java 1 -> Stack (evitar)</pre>	Casi siempre: ArrayDeque

Queue: offer, poll, peek. Deque como pila: push, pop, peek. Con ArrayDeque no se permite null.

HashSet, LinkedHashSet y TreeSet

Un Set es un conjunto: no admite repetidos. HashSet es el más rápido pero sin orden; LinkedHashSet recuerda el orden de inserción; TreeSet mantiene los elementos ordenados. Con ellos se hacen unión, intersección y diferencia.

RECUERDA Un Set es la lista de asistencia de un evento: cada persona aparece una sola vez, por más que se registre dos veces.

CLASE / MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
HashSet	Sin repetidos. add devuelve false si ya estaba. Sin orden fijo.	<pre>Set<String> s = new HashSet<>(); s.add("Ana"); s.add("Luis"); System.out.println(s.add("Ana")); System.out.println(s.size());</pre>	false 2
LinkedHashSet	Sin repetidos y conserva el orden en que llegaron.	<pre>Set<String> s = new LinkedHashSet<>(); s.add("Luis"); s.add("Ana"); s.add("Luis"); System.out.println(s);</pre>	[Luis, Ana]
TreeSet	Sin repetidos y siempre ordenado. Tiene first() y last().	<pre>TreeSet<String> s = new TreeSet<>(); s.add("Luis"); s.add("Ana"); s.add("Eva"); System.out.println(s); System.out.println(s.first());</pre>	[Ana, Eva, Luis] Ana
contains y remove	Buscar y quitar. En Python: x in s y s.remove(x).	<pre>Set<String> s = new TreeSet<>(); s.add("Ana"); s.add("Luis"); System.out.println(s.contains("Ana")); s.remove("Ana"); System.out.println(s);</pre>	true [Luis]
addAll (unión)	Une dos conjuntos. En Python: a b.	<pre>Set<Integer> a = new TreeSet<>(); a.addAll(List.of(1, 2, 3)); Set<Integer> b = new TreeSet<>(); b.addAll(List.of(3, 4)); a.addAll(b); System.out.println(a);</pre>	[1, 2, 3, 4]
retainAll (intersección)	Deja solo lo común. En Python: a & b.	<pre>Set<Integer> a = new TreeSet<>(); a.addAll(List.of(1, 2, 3)); Set<Integer> b = new TreeSet<>(); b.addAll(List.of(2, 3, 4)); a.retainAll(b); System.out.println(a);</pre>	[2, 3]
removeAll (diferencia)	Quita lo que está en el otro. En Python: a - b.	<pre>Set<Integer> a = new TreeSet<>(); a.addAll(List.of(1, 2, 3)); Set<Integer> b = new TreeSet<>(); b.addAll(List.of(2, 3, 4)); a.removeAll(b); System.out.println(a);</pre>	[1]
containsAll (subconjunto)	true si todos los de b están en a.	<pre>Set<Integer> a = new TreeSet<>(); a.addAll(List.of(1, 2, 3)); List<Integer> b = List.of(1, 3); List<Integer> c = List.of(1, 9); System.out.println(a.containsAll(b)); System.out.println(a.containsAll(c));</pre>	true false
Quitar repetidos	Pasar una lista a un set. En Python: set(lista).	<pre>List<Integer> l = List.of(3, 1, 3, 2, 1); Set<Integer> s = new LinkedHashSet<>(l); System.out.println(s);</pre>	[3, 1, 2]

Con tus propias clases, HashSet detecta repetidos usando equals() y hashCode(): hay que redefinirlos.

HashMap, LinkedHashMap y TreeMap

Un Map guarda pares clave-valor y busca por clave, como el diccionario de Python. Las claves no se repiten. HashMap no garantiza orden; LinkedHashMap respeta el orden de inserción; TreeMap ordena por clave.

RECUERDA Un Map es una agenda de contactos: buscas por el nombre (la clave) y obtienes el teléfono (el valor).

MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
put y get	Guarda y lee por clave. Repetir la clave reemplaza el valor.	<pre>Map<String, Double> m = new HashMap<>(); m.put("Ana", 9.0); m.put("Luis", 7.5); m.put("Ana", 9.5); System.out.println(m.get("Ana")); System.out.println(m.size());</pre>	9.5 2
getOrDefault	Si la clave no existe, get da null; esta da un valor por defecto.	<pre>Map<String, Double> m = new HashMap<>(); m.put("Ana", 9.0); System.out.println(m.get("Zoe")); double d = m.getOrDefault("Zoe", 0.0); System.out.println(d);</pre>	null 0.0
containsKey y remove	Preguntar por una clave y borrar un par.	<pre>Map<String, Double> m = new TreeMap<>(); m.put("Ana", 9.0); m.put("Luis", 7.5); boolean b = m.containsKey("Ana"); System.out.println(b); System.out.println(m.remove("Ana")); System.out.println(m);</pre>	true 9.0 {Luis=7.5}
putIfAbsent	Agrega solo si la clave aún no existe.	<pre>Map<String, Double> m = new TreeMap<>(); m.put("Ana", 9.0); m.putIfAbsent("Ana", 5.0); m.putIfAbsent("Eva", 8.0); System.out.println(m);</pre>	{Ana=9.0, Eva=8.0}
merge	Cuenta o acumula: si existe combina, si no, pone el valor.	<pre>Map<String, Integer> c = new TreeMap<>(); String[] ps = "a b a c a b".split(" "); for (String p : ps) { c.merge(p, 1, Integer::sum); } System.out.println(c);</pre>	{a=3, b=2, c=1}
LinkedHashMap	Imprime en el orden en que se agregó.	<pre>Map<String, Double> m = new LinkedHashMap<>(); m.put("Luis", 7.5); m.put("Ana", 9.0); m.put("Eva", 8.0); System.out.println(m);</pre>	{Luis=7.5, Ana=9.0, Eva=8.0}
TreeMap	Ordena por clave (aquí alfabético).	<pre>Map<String, Double> m = new TreeMap<>(); m.put("Luis", 7.5); m.put("Ana", 9.0); m.put("Eva", 8.0); System.out.println(m);</pre>	{Ana=9.0, Eva=8.0, Luis=7.5}
Recorrer con entrySet	Cada Entry trae clave y valor. En Python: items().	<pre>Map<String, Double> m = new TreeMap<>(); m.put("Ana", 9.0); m.put("Luis", 7.5); for (var e : m.entrySet()) { String k = e.getKey(); double v = e.getValue(); System.out.println(k + " " + v); }</pre>	Ana 9.0 Luis 7.5
keySet y values	Solo las claves o solo los valores.	<pre>Map<String, Double> m = new TreeMap<>(); m.put("Ana", 9.0); m.put("Luis", 7.5); for (String k : m.keySet()) { System.out.println(k); } System.out.println(m.values());</pre>	Ana Luis [9.0, 7.5]

TEMA 24

Ordenar: Comparable y Comparator

Collections.sort y List.sort ordenan una lista. Los números y String ya saben compararse. Para tus clases hay dos caminos: Comparable (la clase define su orden natural con compareTo) o Comparator (un criterio aparte, que puedes cambiar).

RECUERDA Comparable es el orden que la clase trae de fábrica (como la lista por apellido); Comparator es un criterio que eliges al momento.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class Estudiante {
    private String nombre;
    private double nota;
    Estudiante(String n, double x) {
        nombre = n;
        nota = x;
    }
    String getNombre() { return nombre; }
    double getNota() { return nota; }
    public String toString() {
        return nombre + ":" + nota;
    }
}

class Grupo {
    static List<Estudiante> lista() {
        var l = new ArrayList<Estudiante>();
        l.add(new Estudiante("Luis", 7.5));
        l.add(new Estudiante("Ana", 9.0));
        l.add(new Estudiante("Eva", 7.5));
        l.add(new Estudiante("Beto", 6.0));
        return l;
    }
}
```

FORMA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Collections.sort	Ordena de menor a mayor el orden natural. En Python: sort().	<pre>List<Integer> n = new ArrayList<>(); n.addAll(List.of(5, 2, 9, 1)); Collections.sort(n); System.out.println(n);</pre>	[1, 2, 5, 9]
Comparable y compareTo	La clase define su orden natural (aquí, por puntos).	<pre>record Alumno(String nombre, int puntos) implements Comparable<Alumno> { public int compareTo(Alumno o) { return puntos - o.puntos; } }</pre>	Beto 12 Luis 40 Ana 85
Comparator.comparing	Orden por un atributo, sin tocar la clase.	<pre>List<Estudiante> l = Grupo.lista(); l.sort(Comparator.comparing(Estudiante::getNombre)); System.out.println(l);</pre>	[Ana:9.0, Beto:6.0, Eva:7.5, Luis:7.5]
reversed()	Invierte el criterio: de mayor a menor.	<pre>List<Estudiante> l = Grupo.lista(); l.sort(Comparator.comparingDouble(Estudiante::getNota).reversed()); System.out.println(l);</pre>	[Ana:9.0, Luis:7.5, Eva:7.5, Beto:6.0]
thenComparing	Desempata: nota de mayor a menor y luego nombre.	<pre>List<Estudiante> l = Grupo.lista(); Comparator<Estudiante> c = Comparator.comparingDouble(Estudiante::getNota).reversed(); l.sort(c.thenComparing(Estudiante::getNombre)); System.out.println(l);</pre>	[Ana:9.0, Eva:7.5, Luis:7.5, Beto:6.0]

FORMA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Comparator con lambda	Criterio escrito a mano: más corto primero.	<pre>List<String> l = new ArrayList<>(); l.addAll(List.of("Luis", "Ana", "Eva")); l.sort((a, b) -> a.length() - b.length()); System.out.println(l);</pre>	[Ana, Eva, Luis]
List.sort	Método de la lista. Comparator.reverseOrder()) invierte.	<pre>List<String> l = new ArrayList<>(); l.addAll(List.of("Luis", "Ana", "Eva")); l.sort(Comparator.reverseOrder()); System.out.println(l);</pre>	[Luis, Eva, Ana]
Mayúsculas primero	El orden natural pone las mayúsculas antes.	<pre>List<String> l = new ArrayList<>(); l.addAll(List.of("ana", "Beto", "Ana")); Collections.sort(l); System.out.println(l); l.sort(String.CASE_INSENSITIVE_ORDER); System.out.println(l);</pre>	[Ana, Beto, ana] [Ana, ana, Beto]

Grupo.lista() devuelve Luis 7.5, Ana 9.0, Eva 7.5 y Beto 6.0. El orden es estable: empates mantienen su orden.

TEMA 25

Las clases Collections y Arrays

Collections y Arrays son clases con métodos estáticos listos para usar: invertir, mezclar, buscar el mayor, contar apariciones o crear listas fijas. Collections trabaja con colecciones; Arrays, con arreglos.

RECUERDA Son la caja de herramientas de las colecciones: no las construyes tú, solo escoges la herramienta y la usas.

MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Collections.reverse	Invierte el orden de la lista. En Python: reverse().	<pre>List<Integer> n = new ArrayList<>(); n.addAll(List.of(1, 2, 3, 4)); Collections.reverse(n); System.out.println(n);</pre>	[4, 3, 2, 1]
Collections.shuffle	Mezcla. Con semilla el resultado se repite siempre.	<pre>List<Integer> n = new ArrayList<>(); n.addAll(List.of(1, 2, 3, 4, 5)); Collections.shuffle(n, new Random(42)); System.out.println(n);</pre>	[2, 3, 4, 5, 1]
max y min	El mayor y el menor. En Python: max() y min().	<pre>List<Integer> n = List.of(4, 9, 2); System.out.println(Collections.max(n)); System.out.println(Collections.min(n));</pre>	9 2
max con Comparator	El criterio lo eliges tú: el nombre más largo.	<pre>List<String> l = List.of("Ana", "Beto"); Comparator<String> c = (a, b) -> a.length() - b.length(); String m = Collections.max(l, c); System.out.println(m);</pre>	Beto
frequency	Cuántas veces aparece un valor. En Python: count().	<pre>List<String> l = List.of("a", "b", "a", "a"); System.out.println(Collections.frequency(l, "a"));</pre>	3
unmodifiableList	Vista de solo lectura: no se puede modificar.	<pre>List<String> l = new ArrayList<>(); l.add("Ana"); List<String> v = Collections.unmodifiableList(l); System.out.println(v); v.add("Luis");</pre>	[Ana] UnsupportedOperationException
List.of	Lista inmutable rápida de escribir.	<pre>List<String> l = List.of("Ana", "Luis"); System.out.println(l); l.add("Eva");</pre>	[Ana, Luis] UnsupportedOperationException
Arrays.asList	Lista de tamaño fijo ligada al arreglo: set sí, add no.	<pre>String[] v = {"Ana", "Luis", "Eva"}; List<String> l = Arrays.asList(v); l.set(0, "Zoe"); System.out.println(v[0]); System.out.println(l);</pre>	Zoe [Zoe, Luis, Eva]

MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Arrays.sort y toString	Ordena un arreglo y lo convierte en texto.	<pre>int[] a = {5, 2, 9, 1}; Arrays.sort(a); System.out.println(Arrays.toString(a)); int pos = Arrays.binarySearch(a, 9); System.out.println(pos);</pre>	<pre>[1, 2, 5, 9] 3</pre>

Para modificar una lista de List of o Arrays.asList, cópiala: new ArrayList<> (lista).

TEMA 26

Lambdas y Streams básicos

Una lambda es una función corta sin nombre: $(x) \rightarrow x * 2$. Un Stream recorre una colección en una cadena de pasos: filter filtra, map transforma y al final se cierra con collect, sum, count... No modifica la lista original.

RECUERDA Un stream es una línea de producción: la lista entra, pasa por estaciones (filtrar, transformar) y al final sale el resultado.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class Estudiante {
    private String nombre;
    private double nota;
    Estudiante(String n, double x) {
        nombre = n;
        nota = x;
    }
    String getNombre() { return nombre; }
    double getNota() { return nota; }
    public String toString() {
        return nombre + ":" + nota;
    }
}

class Grupo {
    static List<Estudiante> lista() {
        var l = new ArrayList<Estudiante>();
        l.add(new Estudiante("Luis", 7.5));
        l.add(new Estudiante("Ana", 9.0));
        l.add(new Estudiante("Eva", 7.5));
        l.add(new Estudiante("Beto", 6.0));
        return l;
    }
}
```

OPERACIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
Lambda	Función corta: parámetros $\rightarrow$ resultado. En Python: lambda.	<pre>Predicate<Double> pasa = n -> n >= 7; Function<Double, Double> sube = n -> n + 1; System.out.println(pasa.test(6.5)); System.out.println(sube.apply(6.5));</pre>	<pre>false 7.5</pre>
filter	Se queda con lo que cumple. En Python: if en la comprensión.	<pre>List<Estudiante> l = Grupo.lista(); l.stream() .filter(e -> e.getNota() >= 7) .forEach(System.out::println);</pre>	<pre>Luis:7.5 Ana:9.0 Eva:7.5</pre>
map y collect(toList)	Transforma cada elemento y junta el resultado en lista.	<pre>List<Estudiante> l = Grupo.lista(); List<String> nombres = l.stream() .map(Estudiante::getNombre) .collect(Collectors.toList()); System.out.println(nombres);</pre>	<pre>[Luis, Ana, Eva, Beto]</pre>
Collectors.joining	Une textos con un separador. En Python: join.	<pre>List<Estudiante> l = Grupo.lista(); String s = l.stream() .map(Estudiante::getNombre) .collect(Collectors.joining(", ")); System.out.println(s);</pre>	<pre>Luis, Ana, Eva, Beto</pre>

OPERACIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
sum	mapToDouble pasa a números y sum() los suma.	<pre>List<Estudiante> l = Grupo.lista(); double total = l.stream() .mapToDouble(Estudiante::getNota) .sum(); System.out.println(total);</pre>	30.0
average	Promedio; orElse da un valor si la lista está vacía.	<pre>List<Estudiante> l = Grupo.lista(); double prom = l.stream() .mapToDouble(Estudiante::getNota) .average().orElse(0); System.out.println(prom);</pre>	7.5
sorted	Ordena dentro del stream con un Comparator.	<pre>List<Estudiante> l = Grupo.lista(); l.stream() .sorted(Comparator.comparingDouble(Estudiante::getNota)) .forEach(System.out::println);</pre>	Beto:6.0 Luis:7.5 Eva:7.5 Ana:9.0
count	Cuenta cuántos pasan el filtro.	<pre>List<Estudiante> l = Grupo.lista(); long n = l.stream() .filter(e -> e.getNota() >= 7) .count(); System.out.println(n);</pre>	3
groupingBy	Agrupar en un Map. Aquí en TreeMap para ver el orden.	<pre>List<Estudiante> l = Grupo.lista(); var g = l.stream().collect(Collectors.groupingBy(Estudiante::getNota, TreeMap::new, Collectors.toList())); g.forEach((k, v) -> System.out.println(k + " " + v));</pre>	6.0 [Beto:6.0] 7.5 [Luis:7.5, Eva:7.5] 9.0 [Ana:9.0]

Grupo.lista() devuelve Luis 7.5, Ana 9.0, Eva 7.5 y Beto 6.0. El stream no cambia la lista original.

PriorityQueue y elección de la colección

PriorityQueue es una cola donde sale primero el de mayor prioridad (por defecto, el menor valor), no el que llegó antes. Para escoger la colección: List si importa el orden y hay repetidos, Set si no hay repetidos, Map si buscas por clave, Queue si atiendes por turnos.

RECUERDA PriorityQueue es la sala de urgencias: no atiende por orden de llegada, sino al paciente más grave primero.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

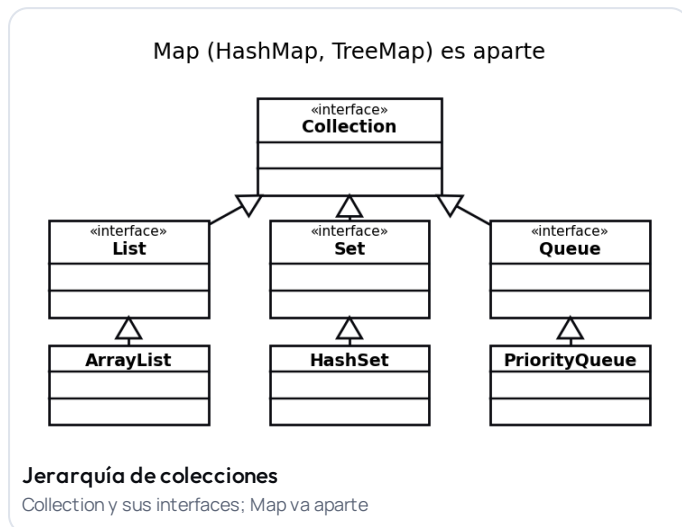
```
class Estudiante {
    private String nombre;
    private double nota;
    Estudiante(String n, double x) {
        nombre = n;
        nota = x;
    }
    String getNombre() { return nombre; }
    double getNota() { return nota; }
    public String toString() {
        return nombre + ":" + nota;
    }
}
class Grupo {
    static List<Estudiante> lista() {
        var l = new ArrayList<Estudiante>();
        l.add(new Estudiante("Luis", 7.5));
        l.add(new Estudiante("Ana", 9.0));
        l.add(new Estudiante("Eva", 7.5));
        l.add(new Estudiante("Beto", 6.0));
        return l;
    }
}
```

COLECCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
PriorityQueue	poll saca primero el menor, sin importar cómo entró.	<pre>PriorityQueue<Integer> pq = new PriorityQueue<>(); pq.offer(5); pq.offer(1); pq.offer(3); System.out.println(pq.peek()); System.out.println(pq.poll()); System.out.println(pq.poll());</pre>	1 1 3
Vaciar en orden	Con poll en un while sale todo ordenado.	<pre>PriorityQueue<Integer> pq = new PriorityQueue<>(); pq.addAll(List.of(4, 8, 1, 6)); while (!pq.isEmpty()) { System.out.println(pq.poll()); }</pre>	1 4 6 8
Mayor primero	Comparator.reverseOrder() invierte la prioridad.	<pre>PriorityQueue<Integer> pq = new PriorityQueue<>(Comparator.reverseOrder()); pq.addAll(List.of(4, 8, 1, 6)); System.out.println(pq.poll()); System.out.println(pq.poll());</pre>	8 6
Con objetos	Se le da un Comparator: sale la mejor nota.	<pre>Comparator<Estudiante> c = Comparator.comparingDouble(Estudiante::getNota).reversed(); PriorityQueue<Estudiante> pq = new PriorityQueue<>(c); pq.addAll(Grupo.lista()); System.out.println(pq.poll());</pre>	Ana:9.0
No imprimas la cola	println(pq) no sale ordenado: solo poll lo es.	<pre>PriorityQueue<Integer> pq = new PriorityQueue<>(); pq.addAll(List.of(5, 1, 4, 2, 3)); System.out.println(pq); System.out.println(pq.poll());</pre>	[1, 2, 4, 5, 3] 1

COLECCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
Elegir List	Orden de llegada y repetidos: notas de un alumno.	<pre>List<Double> notas = new ArrayList<>(); notas.add(8.0); notas.add(8.0); System.out.println(notas);</pre>	[8.0, 8.0]
Elegir Set	Sin repetidos: paralelos distintos.	<pre>Set<String> p = new TreeSet<>(); p.add("P1"); p.add("P2"); p.add("P1"); System.out.println(p);</pre>	[P1, P2]
Elegir Map	Buscar por clave: nota por matrícula.	<pre>Map<String, Double> m = new HashMap<>(); m.put("2024001", 9.0); m.put("2024002", 7.5); System.out.println(m.get("2024002"));</pre>	7.5
Elegir Queue	Turnos: el primero en llegar es el primero en salir.	<pre>Queue<String> cola = new ArrayDeque<>(); cola.offer("Ana"); cola.offer("Luis"); System.out.println(cola.poll());</pre>	Ana

Costo: *ArrayList* get $O(1)$, contains $O(n)$. *HashSet* y *HashMap* $O(1)$. *TreeSet* y *TreeMap* $O(\log n)$. *ArrayDeque* offer y poll $O(1)$. *PriorityQueue* $O(\log n)$.

Diagramas y ventanas de ejemplo



Diseño orientado a objetos y UML

Cómo pensar un programa antes de escribirlo: clases, diagramas UML y buenas prácticas.

TEMA 28

Del problema al diseño: clases, atributos y responsabilidades

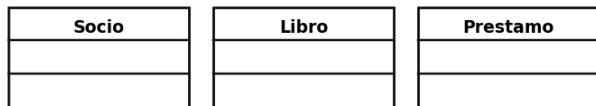
Diseñar es pasar de un enunciado en lenguaje común a clases. Los sustantivos suelen ser clases o atributos, los verbos son comportamientos (métodos) y cada clase debe tener una responsabilidad clara. Abstractar es modelar solo lo que el problema necesita.

RECUERDA Subraya el enunciado con dos colores: sustantivos en azul (clases y datos) y verbos en rojo (métodos).

PASO DEL DISEÑO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Leer el enunciado	Todo diseño parte de una descripción del problema.	<i>// "Un socio pide prestado un libro // de la biblioteca por 7 días."</i>	Sustantivos: socio, libro, biblioteca Verbos: pedir, prestar, devolver
Sustantivos → clases	Cada cosa importante del problema pasa a ser una clase.	<code>class Socio { } class Libro { } class Prestamo { }</code>	Socio Libro Prestamo
Sustantivos → atributos	Los datos que describen a una clase son sus atributos.	<code>class Libro { String titulo; String autor; int paginas; }</code>	Java - Deitel
Verbos → métodos	Lo que la clase sabe hacer son sus métodos.	<code>class Socio { String nombre = "Ana"; String pedir(String t) { return nombre + " pide " + t; } }</code>	Ana pide Java
Responsabilidad	Cada clase hace una tarea clara: el libro sabe si está libre.	<code>class Libro { boolean disponible = true; void prestar() { disponible = false; } }</code>	false
Colaboradores	Una clase pide ayuda a otras para cumplir su tarea.	<code>class Libro { String titulo = "P00"; } class Prestamo { Libro libro = new Libro(); String resumen() { return "Presta " + libro.titulo; } }</code>	Presta P00
Abstracción	Modelar solo lo que importa: no hace falta el color de la portada.	<code>class Libro { String titulo = "P00"; String isbn = "978-1"; // sin color, peso ni precio }</code>	P00 978-1
Tarjeta CRC	Una ficha por clase: Clase, Responsabilidades, Colaboradores.	<i>// Clase: Prestamo // Hace: registrar y cerrar el préstamo // Con: Socio, Libro</i>	Una ficha por clase ayuda a repartir bien el trabajo
Probar la idea	Antes de seguir, comprueba que el modelo cuenta la historia.	<code>class Libro { boolean libre = true; } class Socio { void pedir(Libro l) { l.libre = false; } }</code>	Libre: false

Diagramas y ventanas de ejemplo

Paso 1: identificar clases

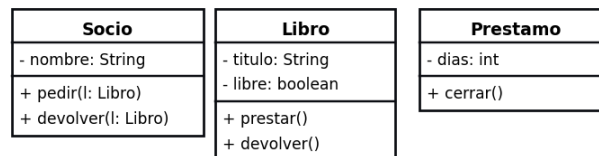


socio, libro y préstamo
son los sustantivos clave

Sustantivos: las clases

Del enunciado salen tres clases candidatas.

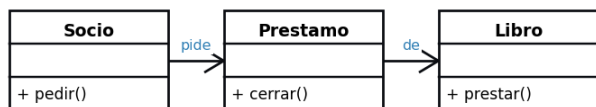
Paso 2: atributos y métodos



Atributos y métodos

Los datos pasan a atributos y los verbos a métodos.

Paso 3: quién colabora con quién



Prestamo conecta a Socio con Libro

Colaboración

Las clases se conocen para cumplir la historia.

TEMA 29

Diagrama de clases: la caja y su traducción a Java

Una clase en UML es una caja con tres partes: nombre, atributos y métodos. Cada miembro lleva un signo de visibilidad (+ público, - privado, # protegido, ~ paquete). Cada parte de la caja se traduce directo a una línea de código Java.

RECUERDA La caja UML es el plano de la casa; la clase Java es la casa ya construida con esos mismos cuartos.

PARTE UML	DESCRIPCIÓN	EJEMPLO	RESULTADO
Caja de 3 partes	Nombre / atributos / métodos. UML: - saldo: double = 0 y + getSaldo(): double	<pre> class Cuenta { private double saldo = 0; public double getSaldo() { return saldo; } } </pre>	0.0
+ público y - privado	+ se ve desde afuera; - solo dentro de la clase.	<pre> class Alumno { public String nombre = "Ana"; private int cedula = 123; public int getCedula() { return cedula; } } </pre>	Ana 123
# protegido y ~ paquete	# lo ven las subclases; ~ (sin modificador) todo el paquete.	<pre> class Base { protected int nivel = 2; int codigo = 7; } class Hija extends Base { int doble() { return nivel * 2; } } </pre>	4 7

PARTE UML	DESCRIPCIÓN	EJEMPLO	RESULTADO
Método con parámetros	UML: + depositar(m: double): void	<pre>class Cuenta { private double saldo; public void depositar(double m) { saldo += m; } public double getSaldo() { return saldo; } }</pre>	50.0
static (subrayado)	Lo subrayado en UML es de la clase, no de cada objeto.	<pre>class Cuenta { private static int total = 0; Cuenta() { total++; } static int getTotal() { return total; } }</pre>	2
Constante {readOnly}	UML: + TASA: double = 0.05 {readOnly} (subrayada).	<pre>class Banco { static final double TASA = 0.05; }</pre>	0.05
Clase abstracta	Nombre en cursiva (o {abstract}): no se puede instanciar.	<pre>abstract class Figura { abstract double area(); }</pre>	4.0
«interface»	Estereotipo entre « » que marca una interfaz.	<pre>interface Prestable { boolean prestar(); }</pre>	true
«enumeration»	Estereotipo para un conjunto fijo de valores.	<pre>enum Estado { DISPONIBLE, PRESTADO }</pre>	[DISPONIBLE, PRESTADO]

En UML primero va el nombre del atributo y luego el tipo: saldo: double. En Java es al revés.

Diagramas y ventanas de ejemplo

Las 3 partes y la visibilidad

CuentaBancaria	
- titular: String	+ público
- saldo: double	- privado
# tasa: double	# protegido
~ codigo: int	~ paquete
+ depositar(m: double)	
+ getSaldo(): double	
- validar(m): boolean	

Visibilidad en la caja

Cada miembro lleva su signo.

static y abstract

Contador	Figura
<u>total: int</u>	- nombre: String
<u>+ getTotal(): int</u>	+ area(): double

subrayado = static

cursiva = abstracta

static y abstracta

Subrayado = static; cursiva = abstracta.

Estereotipos

«interface» Prestable	«enumeration» Estado
+ prestar(): boolean	DISPONIBLE
+ devolver(): void	PRESTADO
	RESERVADO

Interfaz y enumeración

Los estereotipos van entre « ».

Relaciones: asociación, multiplicidad, agregación y composición

Las clases se relacionan. La asociación es una línea (un atributo que apunta a otra clase) y la multiplicidad dice cuántos objetos participan: 1, 0..1, *. La agregación (rombo vacío) y la composición (rombo lleno) son asociaciones del tipo todo-parte.

RECUERDA Agregación: el curso y sus estudiantes (si se acaba el curso, los estudiantes siguen). Composición: el auto y su motor (sin auto, ese motor ya no tiene sentido).

RELACIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
Asociación	Línea entre clases: un objeto guarda una referencia a otro.	<pre>class Carrera { String nombre = "Computación"; } class Estudiante { Carrera carrera = new Carrera(); }</pre>	Computación
Multiplicidad 1	Exactamente uno: el constructor lo exige.	<pre>class Docente { String nombre = "Luis"; } class Curso { final Docente docente; Curso(Docente d) { docente = d; } }</pre>	Luis
Multiplicidad 0..1	Cero o uno: la referencia puede ser null.	<pre>class Socio { String nombre = "Ana"; } class Libro { Socio prestadoA; }</pre>	true Ana
Multiplicidad *	Muchos (0..*): se guarda una lista.	<pre>class Estudiante { String nombre; Estudiante(String n) { nombre = n; } } class Curso { List<Estudiante> lista = new ArrayList<>(); }</pre>	2
Bidireccional	Ambas clases se conocen: hay que enlazar los dos lados.	<pre>class Docente { List<Curso> cursos = new ArrayList<>(); } class Curso { Docente docente; }</pre>	1 true
Agregación (rombo vacío)	La parte llega de afuera y vive aparte del todo.	<pre>class Estudiante { } class Curso { List<Estudiante> lista = new ArrayList<>(); void agregar(Estudiante e) { lista.add(e); } }</pre>	Sigue vivo: true
Composición (rombo lleno)	El todo crea a su parte y la parte no existe sin él.	<pre>class Motor { } class Auto { private Motor motor = new Motor(); boolean tieneMotor() { return motor != null; } }</pre>	true
¿Quién crea la parte?	La pista más rápida en el código.	<pre>// Agregación: Curso(Estudiante e) // Composición: motor = new Motor()</pre>	Agregación: llega de afuera Composición: la crea el todo

Una pista: si la clase hace new de la parte, es composición; si la recibe, es agregación.

Diagramas y ventanas de ejemplo

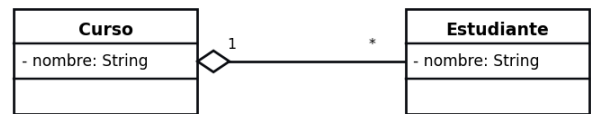
Asociación 1 a 0..*



Asociación con multiplicidad

Un socio tiene de 0 a muchos préstamos.

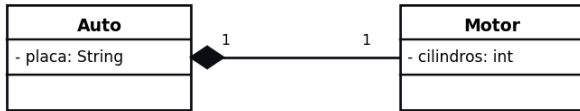
Agregación (rombo vacío)



Agregación

Un curso agrupa estudiantes que existen aparte.

Composición (rombo lleno)



Composición

El motor forma parte del auto.

Multiplicidad

1	exactamente uno
0..1	cero o uno
* o 0..*	cero o muchos
1..*	uno o muchos
2..5	entre dos y cinco

Multiplicidades comunes

Se escriben en los extremos de la línea.

TEMA 31

Herencia, realización y dependencia en UML

La herencia (flecha con triángulo vacío) significa «es un» y se escribe `extends`. La realización (línea punteada con triángulo) es `implements`. La dependencia (flecha punteada) significa «usa»: una clase necesita a otra solo de pasada, por ejemplo como parámetro.

RECUERDA Herencia: un estudiante ES una persona. Realización: un libro CUMPLE el contrato Prestable. Dependencia: la impresora USA una cuenta solo mientras imprime.

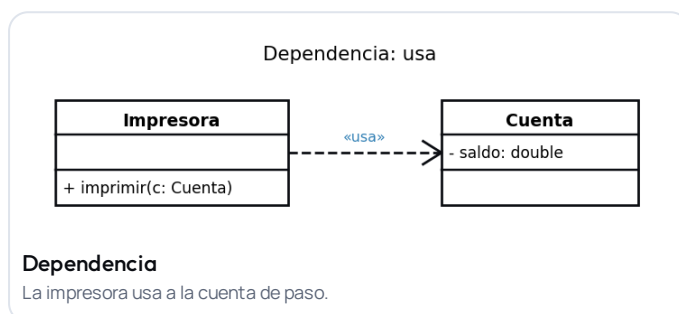
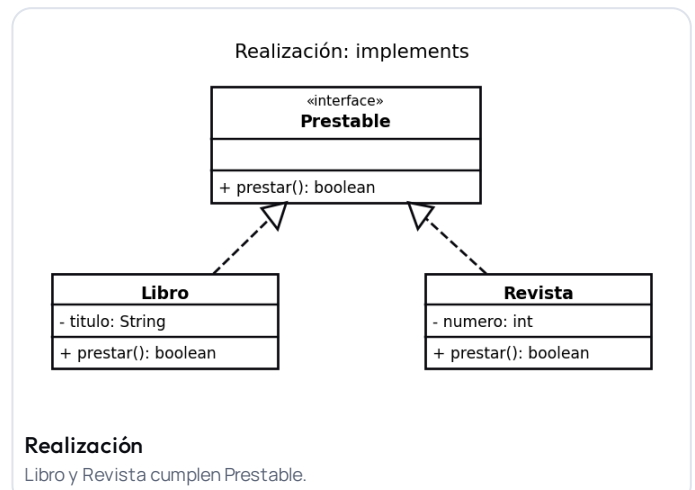
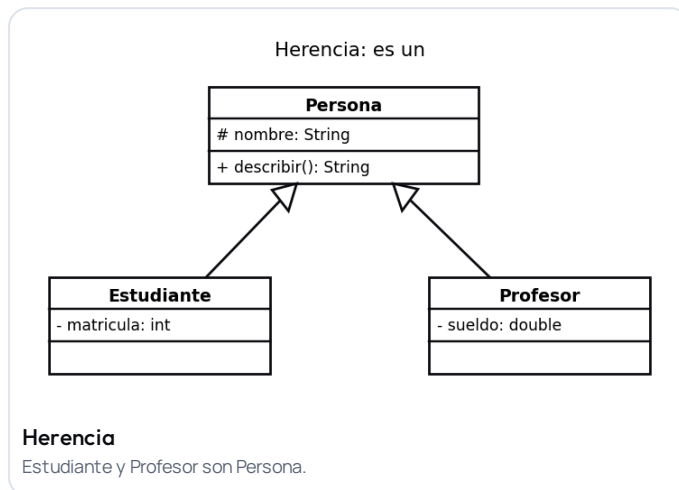
CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class Persona {
    protected String nombre;
    Persona(String nombre) {
        this.nombre = nombre;
    }
    String describir() {
        return "Persona " + nombre;
    }
}
```

RELACIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
Herencia: extends	El hijo recibe atributos y métodos del padre.	<pre>class Estudiante extends Persona { int matricula = 2024; Estudiante(String n) { super(n); } }</pre>	Ana 2024
Sobrescribir un método	El hijo cambia lo que hereda con <code>@Override</code> .	<pre>class Estudiante extends Persona { Estudiante(String n) { super(n); } @Override String describir() { return "Estudiante " + nombre; } }</pre>	Estudiante Ana

RELACIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
Padre abstracto	Nombre en cursiva: el padre solo define el molde.	<pre>abstract class Figura { abstract double area(); } class Cuadrado extends Figura { double area() { return 4.0; } }</pre>	4.0
Realización: implements	Línea punteada con triángulo hacia la interfaz.	<pre>interface Prestable { boolean prestar(); } class Libro implements Prestable { public boolean prestar() { return true; } }</pre>	true
Varias interfaces	Una clase puede cumplir muchos contratos.	<pre>interface Prestable { } interface Reservable { } class Libro implements Prestable, Reservable { }</pre>	true true
Una sola superclase	Con extends solo se hereda de una clase.	<pre>class Hija extends Padre, Madre { }</pre>	error: '{' expected
Dependencia: «usa»	Flecha punteada: la usa como parámetro, sin guardarla.	<pre>class Impresora { String imprimir(Persona p) { return "Imp: " + p.describir(); } }</pre>	Imp: Persona Ana
Dependencia o asociación	Atributo = asociación. Parámetro o local = dependencia.	<pre>class A { B b; } class C { void m() { B x = new B(); } }</pre>	A guarda un B: asociación C usa un B de paso: dependencia

Diagramas y ventanas de ejemplo



De UML a Java: la biblioteca completa

Un diagrama de clases se traduce pieza por pieza: cada caja es una class, cada atributo un campo, cada asociación una referencia (o una lista si la multiplicidad es *) y cada operación un método. Aquí, Socio 1 - * Prestamo * - 1 Libro.

RECUERDA Traducir UML a Java es como pasar un plano a ladrillos: no inventas nada, solo sigues lo dibujado.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class Libro {
    private String titulo;
    private boolean disponible = true;
    Libro(String t) { titulo = t; }
    String getTitulo() { return titulo; }
    boolean estaLibre() { return disponible; }
    void prestar() { disponible = false; }
}

class Prestamo {
    private Libro libro;
    Prestamo(Libro l) { libro = l; }
    Libro getLibro() { return libro; }
}

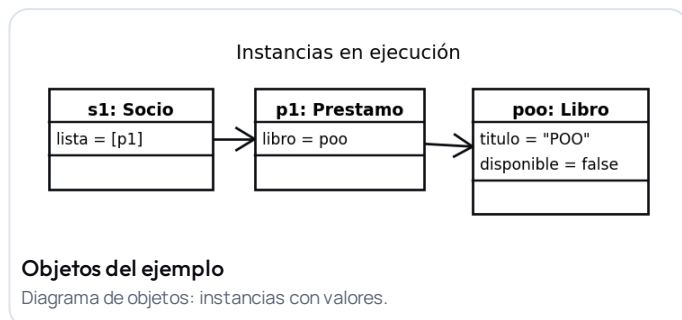
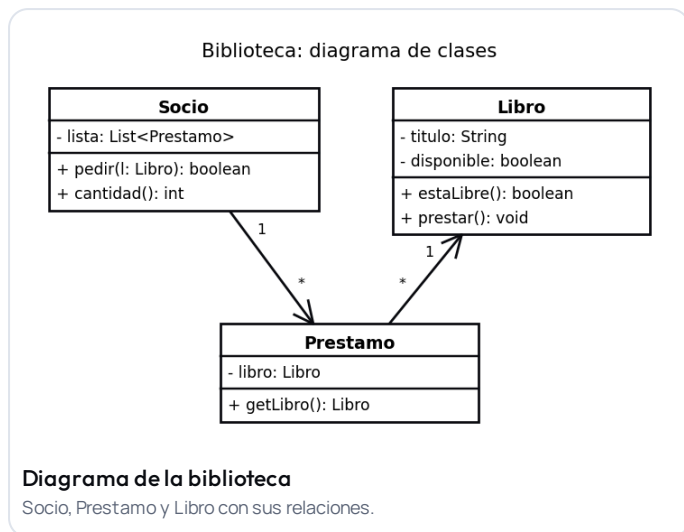
class Socio {
    private List<Prestamo> lista =
        new ArrayList<>();
    boolean pedir(Libro l) {
        if (!l.estaLibre()) { return false; }
        l.prestar();
        lista.add(new Prestamo(l));
        return true;
    }
    int cantidad() { return lista.size(); }
}
```

PIEZA DEL DIAGRAMA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Clase Libro	Cada caja se convierte en una class con su constructor.	<code>Libro l = new Libro("P00");</code> <code>System.out.println(l.getTitulo());</code>	P00
- disponible: boolean = true	Atributo privado con valor inicial y su getter.	<code>Libro l = new Libro("P00");</code> <code>System.out.println(l.estaLibre());</code>	true
Asociación Prestamo → Libro	El préstamo guarda el libro que se prestó.	<code>Libro l = new Libro("Java");</code> <code>Prestamo p = new Prestamo(l);</code> <code>String t = p.getLibro().getTitulo();</code> <code>System.out.println(t);</code>	Java
Efecto del préstamo	Al pedir el libro, deja de estar libre.	<code>Libro l = new Libro("Java");</code> <code>Socio s = new Socio();</code> <code>System.out.println(l.estaLibre());</code> <code>s.pedir(l);</code> <code>System.out.println(l.estaLibre());</code>	true false
Socio 1 - * Prestamo	Multiplicidad *: el socio guarda una lista de préstamos.	<code>Socio s = new Socio();</code> <code>s.pedir(new Libro("P00"));</code> <code>s.pedir(new Libro("Java"));</code> <code>System.out.println(s.cantidad());</code>	2
Regla de negocio	pedir() devuelve false si el libro no está libre.	<code>Socio s = new Socio();</code> <code>Libro l = new Libro("P00");</code> <code>System.out.println(s.pedir(l));</code> <code>System.out.println(s.pedir(l));</code>	true false
Encapsulación	El atributo es privado: solo se llega con el getter.	<code>Libro l = new Libro("P00");</code> <code>String t = l.titulo;</code>	error: titulo has private access in Libro

PIEZA DEL DIAGRAMA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Programa completo	Las tres clases trabajando juntas.	<pre> Libro a = new Libro("P00"); Libro b = new Libro("Java"); Socio s = new Socio(); System.out.println(s.pedir(a)); System.out.println(s.pedir(a)); System.out.println(s.pedir(b)); System.out.println(s.cantidad()); </pre>	<pre> true false true 2 </pre>

Las clases Libro, Prestamo y Socio de arriba son la traducción del diagrama de la galería.

Diagramas y ventanas de ejemplo



TEMA 33

Casos de uso y diagramas de secuencia

El diagrama de casos de uso muestra qué hace el sistema para cada actor (óvalos y muñecos). El de secuencia muestra cómo se hablan los objetos en el tiempo: líneas de vida verticales, flechas horizontales y se lee de arriba hacia abajo, igual que el código.

RECUERDA Casos de uso: el menú del restaurante (qué se puede pedir). Secuencia: la conversación entre mesero y cocina, en orden.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```

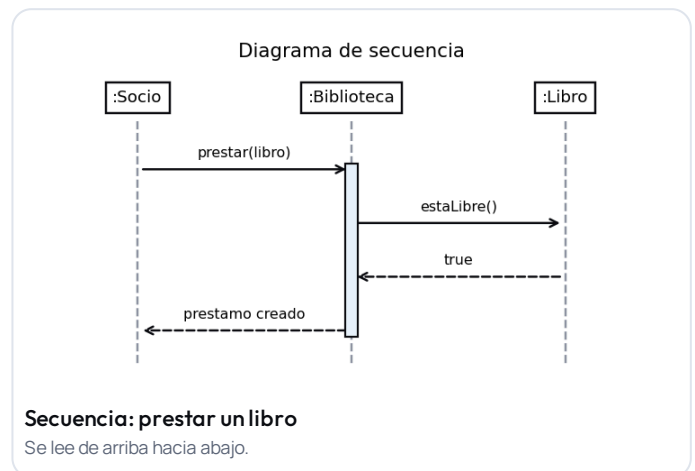
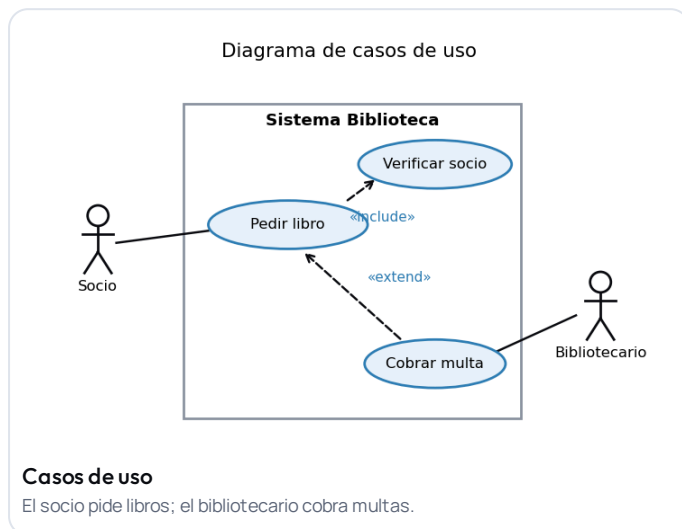
class Libro {
    boolean estaLibre() {
        System.out.println(" estaLibre()");
        return true;
    }
}
class Biblioteca {
    Libro libro = new Libro();
    boolean prestar() {
        System.out.println("prestar()");
        return libro.estaLibre();
    }
}

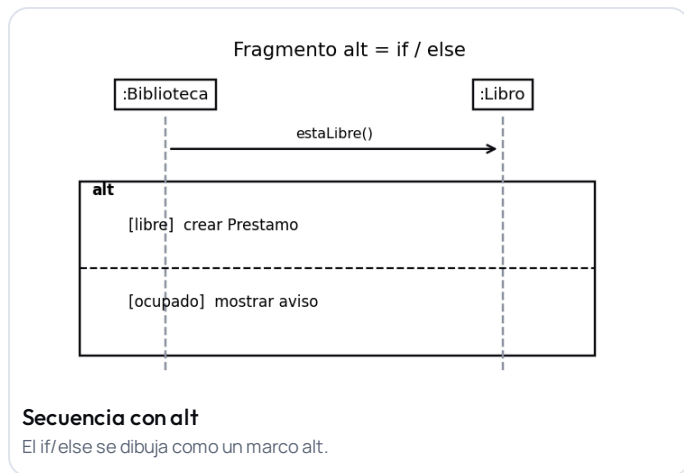
```

ELEMENTO UML	DESCRIPCIÓN	EJEMPLO	RESULTADO
Actor	Quien usa el sistema desde afuera; no es una clase del sistema.	<i>// Actores: Socio, Bibliotecario</i>	Un actor es un rol, no una clase del sistema
Caso de uso	Un óvalo con algo que el sistema hace para el actor.	<pre> static void pedirLibro() { System.out.println("Prestado"); } </pre>	Prestado

ELEMENTO UML	DESCRIPCIÓN	EJEMPLO	RESULTADO
«include»	El caso base SIEMPRE usa a otro caso.	<pre>static void verificarSocio() { System.out.println("Verificado"); } static void pedirLibro() { verificarSocio(); System.out.println("Prestado"); }</pre>	Verificado Prestado
«extend»	El caso extra ocurre solo si se cumple una condición.	<pre>static void pedirLibro(boolean tarde) { System.out.println("Prestado"); if (tarde) { System.out.println("Multa"); } }</pre>	Prestado Prestado Multa
Línea de vida	Línea punteada vertical: el objeto existe en el tiempo.	// :Socio :Biblioteca :Libro	Cada objeto tiene su línea vertical hacia abajo
Mensaje (flecha)	Una flecha es una llamada a un método.	<pre>Biblioteca b = new Biblioteca(); b.prestar();</pre>	prestar() estaLibre()
Mensaje de retorno	Flecha punteada: lo que devuelve el método.	<pre>boolean ok = new Biblioteca().prestar(); System.out.println("retorna " + ok);</pre>	prestar() estaLibre() retorna true
Orden: arriba a abajo	El diagrama y el código se leen en el mismo orden.	<pre>Biblioteca b = new Biblioteca(); System.out.println("1 socio pide"); b.prestar(); System.out.println("2 socio recibe");</pre>	1 socio pide prestar() estaLibre() 2 socio recibe
Fragmento alt (si / si no)	Se traduce como un if con else.	<pre>Libro l = new Libro(); boolean ok = l.estaLibre(); if (ok) { System.out.println("Prestado"); } else { System.out.println("Ocupado"); }</pre>	estaLibre() Prestado

Diagramas y ventanas de ejemplo





TEMA 34

Principios de buen diseño: cohesión, acoplamiento y SOLID

Un buen diseño tiene clases cohesivas (hacen una cosa) y poco acopladas (dependen de lo mínimo). Se encapsula lo que cambia y se prefiere componer antes que heredar. SOLID resume cinco reglas: S, O, L, I, D.

RECUERDA Piensa en una caja de herramientas: cada herramienta sirve para una cosa (cohesión) y se cambia sin desarmar todo (bajo acoplamiento).

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```

interface Pago {
    void pagar(double m);
}
class Tarjeta implements Pago {
    public void pagar(double m) {
        System.out.println("Tarjeta " + m);
    }
}
interface Tarifa {
    double calcular(double kg);
}
interface Figura { double area(); }
  
```

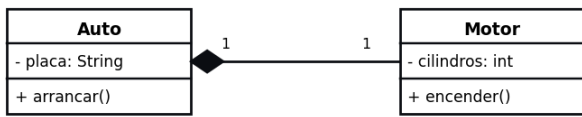
PRINCIPIO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Cohesión alta	Antes: Cuenta también enviaba correos. Ahora solo hace lo suyo.	<pre> class Cuenta { private double saldo; void depositar(double m) { saldo += m; } double getSaldo() { return saldo; } } </pre>	30.0
Acoplamiento bajo	Antes: promedio(Estudiante e). Ahora recibe solo las notas.	<pre> static double promedio(List<Double> n) { double s = 0; for (double x : n) { s += x; } return s / n.size(); } </pre>	8.0
Encapsular lo que varía	La tarifa cambia: se aísla en su propio tipo.	<pre> class Envio { private Tarifa t; Envio(Tarifa t) { this.t = t; } double costo(double kg) { return t.calcular(kg); } } </pre>	6.0 5.0

PRINCIPIO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Composición sobre herencia	Antes: Auto extends Motor (falso). Ahora: Auto tiene Motor.	<pre> class Motor { String encender() { return "on"; } } class Auto { private Motor motor = new Motor(); String arrancar() { return motor.encender(); } } </pre>	on
S: una sola razón de cambio	Antes: Factura calculaba e imprimía. Ahora se separan.	<pre> class Factura { double total = 50; } class ImpresorFactura { String texto(Factura f) { return "Total: " + f.total; } } </pre>	Total: 50.0
O: abierto a extensión	Antes: un if por cada figura. Ahora, una clase nueva.	<pre> record Cuadrado(double l) implements Figura { public double area() { return l * l; } } // total(figuras) no se modifica </pre>	13.0
L: sustitución de Liskov	Un hijo sirve donde se espera el padre; no promete lo que no cumple.	<pre> class Ave { } interface Volador { void volar(); } class Gorrión extends Ave implements Volador { public void volar() { System.out.println("vuela"); } } class Pinguino extends Ave { } </pre>	vuela false
I: interfaces pequeñas	Antes: una interfaz gigante. Ahora, contratos chicos.	<pre> interface Imprime { String imprimir(); } interface Escanea { String escanear(); } class Simple implements Imprime { public String imprimir() { return "imprime"; } } </pre>	imprime
D: depender de abstracciones	Antes: Tienda hacía new Tarjeta(). Ahora recibe un Pago.	<pre> class Tienda { private Pago pago; Tienda(Pago p) { pago = p; } void cobrar(double m) { pago.pagar(m); } } </pre>	Tarjeta 20.0 Efectivo 5.0

SOLID: S responsabilidad única, O abierto/cerrado, L Liskov, I segregación de interfaces, D inversión de dependencias.

Diagramas y ventanas de ejemplo

Composición sobre herencia

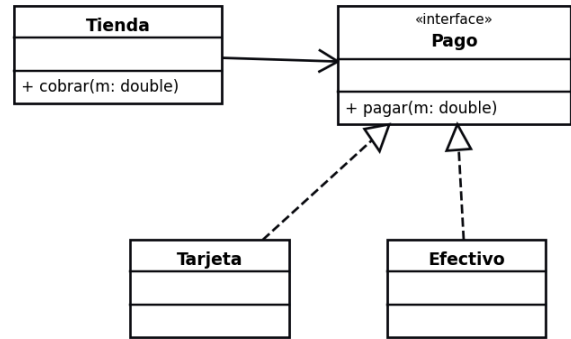


Auto tiene un Motor

Composición sobre herencia

Un auto tiene un motor; no es un motor.

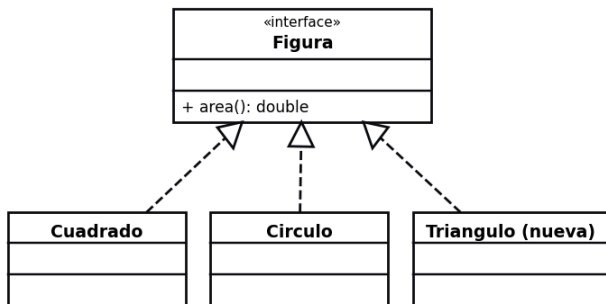
Principio D



Dependencia de abstracciones (D)

La tienda conoce la interfaz, no la clase concreta.

Principio O



Abierto/cerrado (O)

Nueva figura = nueva clase, nada se modifica.

TEMA 35

Patrones de diseño I: Singleton y Factory Method

Un patrón es una solución probada a un problema que se repite. Singleton garantiza una sola instancia con acceso global. Factory Method crea objetos sin que quien los pide conozca la clase concreta.

RECUERDA Singleton: la alcaldía de una ciudad, solo hay una. Fábrica: pides «un pan» y la panadería decide cuál hornear.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```

interface Cuenta {
    String tipo();
}
class Ahorro implements Cuenta {
    public String tipo() { return "Ahorro"; }
}
class Corriente implements Cuenta {
    public String tipo() {
        return "Corriente";
    }
}
  
```

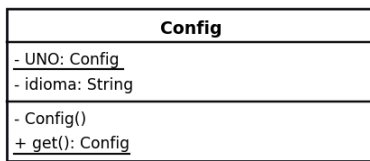
PATRÓN Y PIEZA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Singleton: constructor privado	Nadie más puede hacer new de la clase.	<pre> class Config { private Config() { } } // en otra clase: Config c = new Config(); </pre>	error: Config() has private access in Config

PATRÓN Y PIEZA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Singleton: instancia única	Un campo static guarda la única instancia.	<pre>class Config { private static final Config UNO = new Config(); private Config() { } static Config get() { return UNO; } }</pre>	true
Singleton: estado compartido	Todos ven los mismos datos.	<pre>class Sesion { private static final Sesion S = new Sesion(); String usuario = "nadie"; private Sesion() { } static Sesion get() { return S; } }</pre>	Ana
Singleton con enum	La forma más corta y segura en Java.	<pre>enum Reloj { UNICO; int ticks = 0; int marcar() { return ++ticks; } }</pre>	2
Sin fábrica	El cliente tiene que conocer cada clase concreta.	<pre>Cuenta c1 = new Ahorro(); Cuenta c2 = new Corriente(); System.out.println(c1.tipo()); System.out.println(c2.tipo());</pre>	Ahorro Corriente
Fábrica con if	Un método static decide qué objeto crear.	<pre>class Fabrica { static Cuenta crear(String t) { if (t.equals("ahorro")) { return new Ahorro(); } return new Corriente(); } }</pre>	Ahorro Corriente
Fábrica con switch	Versión más corta; el cliente no cambia.	<pre>static Cuenta crear(String t) { return switch (t) { case "ahorro" -> new Ahorro(); default -> new Corriente(); }; }</pre>	Ahorro Corriente
Factory Method clásico	La subclase decide qué producto se crea.	<pre>abstract class Banco { abstract Cuenta abrir(); String saludo() { return "Abre " + abrir().tipo(); } }</pre>	Abre Ahorro
Producto nuevo	Solo cambia la fábrica; el cliente queda igual.	<pre>case "plazo" -> new PlazoFijo();</pre>	El cliente sigue pidiendo una Cuenta, sin cambios

Variante perezosa del Singleton: crear la instancia dentro de get() solo si todavía es null.

Diagramas y ventanas de ejemplo

Singleton: constructor privado



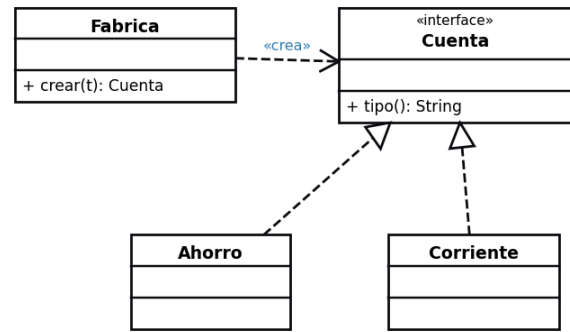
- Config(): nadie más hace new

subrayado = static: único y global

Singleton

Un solo objeto, accesible con getInstance.

Factory Method



Factory Method

La fábrica crea cuentas sin que el cliente conozca la clase.

TEMA 36

Patrones de diseño II: Strategy y Observer

Strategy encapsula un algoritmo detrás de una interfaz para poder cambiarlo mientras el programa corre. Observer avisa automáticamente a una lista de interesados cuando algo ocurre, sin que el emisor los conozca uno por uno.

RECUERDA Strategy: elegir la ruta del GPS (a pie, en bus, en carro). Observer: suscribirte a un canal para que te avisen cuando suba un video.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```

interface Descuento {
    double aplicar(double precio);
}

class Carrito {
    private Descuento d;
    Carrito(Descuento d) { this.d = d; }
    void cambiar(Descuento d) { this.d = d; }
    double total(double p) {
        return d.aplicar(p);
    }
}

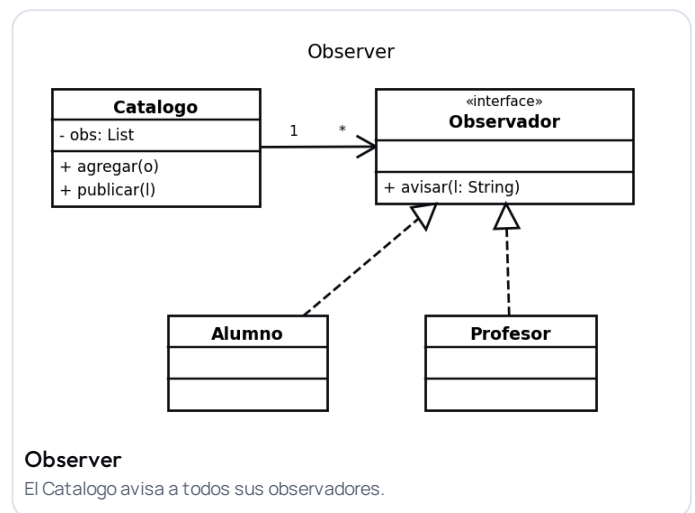
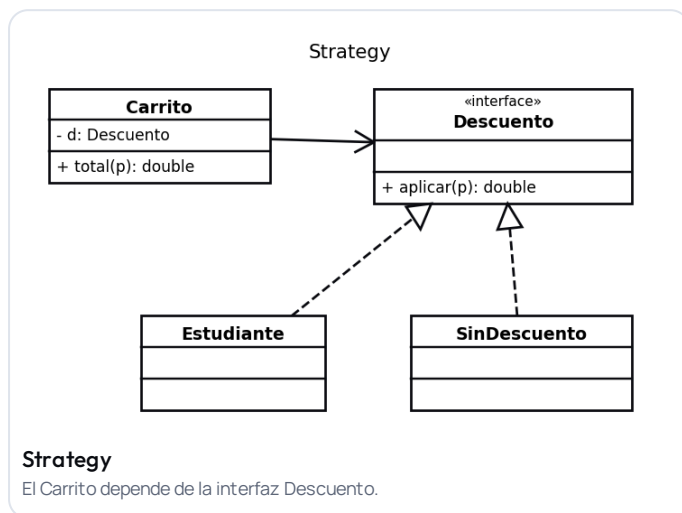
interface Observador {
    void avisar(String libro);
}

class Catalogo {
    List<Observador> obs = new ArrayList<>();
    void agregar(Observador o) { obs.add(o); }
    void quitar(Observador o) {
        obs.remove(o);
    }
    void publicar(String libro) {
        obs.forEach(o -> o.avisar(libro));
    }
}
  
```

PATRÓN Y PIEZA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Strategy: la interfaz	Cada descuento es una estrategia; aquí, una lambda.	<code>Descuento d = p -> p * 0.9;</code> <code>System.out.println(d.aplicar(100));</code>	90.0
Estrategia concreta	Una clase que cumple la interfaz.	<pre>class Estudiante implements Descuento { public double aplicar(double p) { return p * 0.8; } }</pre>	40.0
Contexto	El Carrito usa la estrategia sin saber cuáles.	<code>Carrito c = new Carrito(p -> p * 0.5);</code> <code>System.out.println(c.total(80));</code>	40.0

PATRÓN Y PIEZA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Cambiar en ejecución	Se cambia el algoritmo sin tocar el Carrito.	<pre>Carrito c = new Carrito(p -> p); System.out.println(c.total(100)); c.cambiar(p -> p * 0.9); System.out.println(c.total(100));</pre>	100.0 90.0
Strategy en la API	Comparator es una estrategia de orden.	<pre>List<String> n = new ArrayList<>(List.of("Pedro", "Ana", "Luis")); n.sort(Comparator.naturalOrder()); System.out.println(n); n.sort(Comparator.comparing(String::length).reversed()); System.out.println(n);</pre>	[Ana, Luis, Pedro] [Pedro, Luis, Ana]
Observer: observador concreto	Quien quiere enterarse implementa la interfaz.	<pre>class Alumno implements Observador { public void avisar(String l) { System.out.println("Hay " + l); } }</pre>	Hay Java
Observer: sujeto	El Catalogo guarda la lista y avisa a todos.	<pre>Catalogo c = new Catalogo(); c.agregar(l -> System.out.println(l)); c.publicar("P00");</pre>	P00
Varios observadores	Un solo aviso llega a todos los suscritos.	<pre>var o = System.out; Catalogo c = new Catalogo(); c.agregar(l -> o.println("Ana " + l)); c.agregar(l -> o.println("Luis " + l)); c.publicar("Java");</pre>	Ana Java Luis Java
Quitar un observador	Quien se da de baja ya no recibe avisos.	<pre>Catalogo c = new Catalogo(); Observador o = System.out::println; c.agregar(o); c.publicar("uno"); c.quitar(o); c.publicar("dos");</pre>	uno

Diagramas y ventanas de ejemplo



Herencia, polimorfismo e interfaces

Reutilizar código con jerarquías de clases y tratar objetos distintos de forma uniforme.

TEMA 37

Herencia: extends, superclase y subclase

La herencia crea una clase nueva (subclase) a partir de otra (superclase) con la palabra `extends`. La subclase recibe los atributos y métodos no privados y puede agregar los suyos. Se usa cuando existe la relación "es un": un Perro es un Animal.

RECUERDA Herencia = rasgos de familia: recibes lo de tus padres, pero además tienes lo tuyo.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

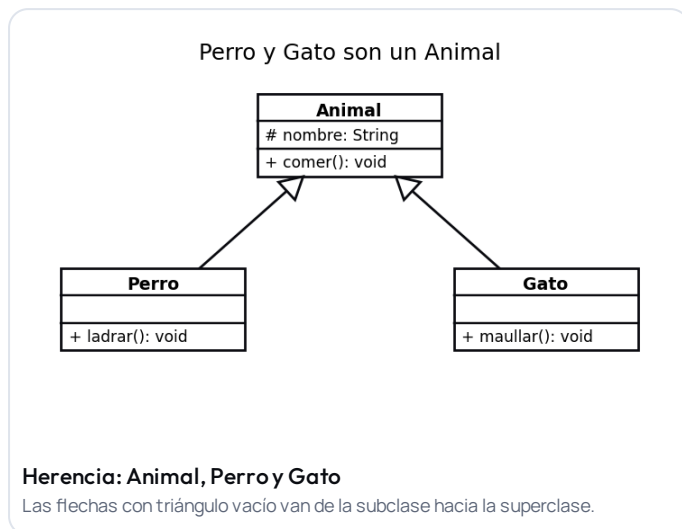
```
class Animal {
    String nombre;
    Animal(String nombre) {
        this.nombre = nombre;
    }
    void comer() {
        System.out.println(nombre + " come");
    }
}
class Perro extends Animal {
    Perro(String n) {
        super(n);
    }
    void ladrar() {
        System.out.println("Guau");
    }
}
```

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
extends	Declara que una clase hereda de otra.	<pre>class Gato extends Animal { Gato(String n) { super(n); } }</pre>	Misu come
Usar lo heredado	La subclase usa los métodos de su superclase.	<pre>Perro p = new Perro("Rex"); p.comer(); p.ladrar();</pre>	Rex come Guau
Relación "es un"	Un Perro cabe en una variable de tipo Animal.	<pre>Perro p = new Perro("Rex"); Animal a = p; System.out.println(a.nombre); System.out.println(p instanceof Animal);</pre>	Rex true
Atributos heredados	Los atributos no privados se usan como propios.	<pre>Perro p = new Perro("Luna"); p.nombre = "Nina"; p.comer();</pre>	Nina come
Cadena de herencia	Labrador hereda de Perro y este de Animal.	<pre>class Labrador extends Perro { String color = "miel"; Labrador(String n) { super(n); } }</pre>	Toby come Guau miel
Lo propio de la subclase	<code>ladrar()</code> existe en Perro, no en Animal.	<pre>Animal a = new Animal("Bicho"); a.ladrar();</pre>	error: cannot find symbol symbol: method ladrar() location: variable a of type Animal

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
private no se hereda	Un atributo privado no se ve desde la subclase.	<pre> class Cuenta { private double saldo; } class Ahorro extends Cuenta { double ver() { return saldo; } } </pre>	error: saldo has private access in Cuenta
Una sola superclase	Java no permite extender dos clases a la vez.	<pre> class A { } class B { } class C extends A, B { } </pre>	error: '{' expected
La raíz es Object	Toda clase termina heredando de Object.	<pre> Class<?> c = Perro.class; while (c != null) { String n = c.getSimpleName(); System.out.println(n); c = c.getSuperclass(); } </pre>	Perro Animal Object

En la guía, Animal y Perro (arriba de la tabla) existen en todas las filas.

Diagramas y ventanas de ejemplo



super y constructores en herencia

Al crear un objeto de la subclase, primero se ejecuta el constructor de la superclase y después el de la subclase. `super(...)` llama al constructor de arriba y debe ser la primera línea. Si la superclase no tiene constructor sin parámetros, `super(...)` es obligatorio.

RECUERDA Primero se construye el cimiento (superclase) y luego el piso de arriba (subclase).

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class Vehiculo {
    String placa;
    Vehiculo(String placa) {
        this.placa = placa;
        System.out.println("Placa " + placa);
    }
}
```

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>super(...)</code>	Llama al constructor de la superclase.	<pre>class Auto extends Vehiculo { Auto(String p) { super(p); System.out.println("Auto"); } }</pre>	Placa GAB-123 Auto
Orden de ejecución	Se construye de arriba hacia abajo: A, B, C.	<pre>class A { A() { System.out.println("A"); } } class B extends A { B() { System.out.println("B"); } } class C extends B { C() { System.out.println("C"); } }</pre>	A B C
<code>super()</code> implícito	Si no escribes <code>super</code> , Java pone <code>super()</code> solo.	<pre>class Base { Base() { System.out.println("Base"); } } class Hija extends Base { Hija() { } }</pre>	Base
Constructor por defecto	Sin constructores, Java crea uno que llama a <code>super()</code> .	<pre>class Base { Base() { System.out.println("Base"); } } class Sola extends Base { }</pre>	Base
<code>super(...)</code> obligatorio	Vehiculo exige una placa: falta llamar a <code>super(p)</code> .	<pre>class Auto extends Vehiculo { Auto() { System.out.println("Auto"); } }</pre>	error: constructor Vehiculo in class Vehiculo cannot be applied to given types;
super va primero	<code>super(...)</code> debe ser la primera sentencia.	<pre>class Auto extends Vehiculo { Auto(String p) { System.out.println("Auto"); super(p); } }</pre>	error: call to super must be first statement in constructor

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
this(...)	Un constructor llama a otro de la misma clase.	<pre>class Taxi extends Vehiculo { Taxi() { this("TAX-000"); } Taxi(String placa) { super(placa); } }</pre>	Placa TAX-000
Atributos propios	La subclase inicializa lo suyo después de super.	<pre>class Moto extends Vehiculo { int cc; Moto(String p, int cc) { super(p); this.cc = cc; } }</pre>	Placa MOT-9 MOT-9 150

Vehiculo (arriba) solo tiene un constructor con parámetro: por eso sus hijas deben llamar a super(...).

TEMA 39

Sobrescritura de métodos y @Override

Sobrescribir es volver a escribir en la subclase un método heredado, con el mismo nombre y los mismos parámetros, para cambiar su comportamiento. @Override pide al compilador que verifique que de verdad sobrescribes. Con super.metodo() también usas la versión original.

RECUERDA Sobrescribir es reescribir la receta de la abuela a tu manera; sobrecargar es tener dos recetas con otros ingredientes.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class Empleado {
    double sueldo = 1000;
    public double pagar() {
        return sueldo;
    }
    public void presentar() {
        System.out.println("Empleado");
    }
}
```

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
@Override	La subclase cambia lo que hace pagar().	<pre>class Gerente extends Empleado { @Override public double pagar() { return sueldo * 1.5; } }</pre>	1500.0 1000.0
super.metodo()	Reutiliza la versión de la superclase y agrega más.	<pre>class Gerente extends Empleado { @Override public void presentar() { super.presentar(); System.out.println("Gerente"); } }</pre>	Empleado Gerente
Sobrecarga (no es lo mismo)	Mismo nombre pero otros parámetros: es otro método.	<pre>class Operario extends Empleado { double pagar(int horas) { return horas * 5.0; } }</pre>	1000.0 50.0
Heredado sin cambios	Si no sobrescribes, se usa el de la superclase.	<pre>class Operario extends Empleado { }</pre>	1000.0

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
@Override detecta errores	Un nombre mal escrito no compila.	<pre>class Gerente extends Empleado { @Override public double pagar() { return sueldo; } }</pre>	error: method does not override or implement a method from a supertype
Retorno compatible	El tipo de retorno debe ser el mismo.	<pre>class Gerente extends Empleado { public int pagar() { return 1500; } }</pre>	error: pagar() in Gerente cannot override pagar() in Empleado return type int is not compatible with double
Acceso no más débil	No puedes quitar el public al sobrescribir.	<pre>class Gerente extends Empleado { @Override void presentar() { System.out.println("G"); } }</pre>	error: presentar() in Gerente cannot override presentar() in Empleado attempting to assign weaker access privileges; was public

TEMA 40

Control de acceso en herencia: protected y final

private solo se ve dentro de la clase; protected también en el paquete y en las subclases; public en todas partes. final en un método impide sobrescribirlo y en una clase impide heredarla. Conviene preferir private y dar getters, para no romper el encapsulamiento.

RECUERDA private = tu diario, protected = el álbum de la familia, public = tu foto en la calle.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class Cuenta {
    private double saldo = 100;
    protected String titular = "Ana";
    double getSaldo() {
        return saldo;
    }
}
```

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
private en la subclase	El atributo privado no se puede usar directamente.	<pre>class Ahorro extends Cuenta { double ver() { return saldo; } }</pre>	error: saldo has private access in Cuenta
protected	La subclase sí ve un atributo protected.	<pre>class Ahorro extends Cuenta { String ver() { return titular; } }</pre>	Ana
Getter de lo privado	La subclase usa el getter público de la superclase.	<pre>class Ahorro extends Cuenta { double ver() { return getSaldo(); } }</pre>	100.0
Niveles de acceso	Quién puede ver cada modificador.	<pre>private double a; protected double b; public double c;</pre>	a: solo esta clase b: paquete y subclases c: todas las clases

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
final en un método	Un método final no se puede sobrescribir.	<pre>class Padre { final void saludar() { } } class Hijo extends Padre { void saludar() { } }</pre>	error: saludar() in Hijo cannot override saludar() in Padre overridden method is final
final en una clase	Una clase final no se puede heredar.	<pre>final class Sello { } class Falso extends Sello { }</pre>	error: cannot inherit from final Sello
final en un atributo	Una constante no cambia después de asignarse.	<pre>class Tienda { static final double IVA = 0.15; }</pre>	0.15
Por qué preferir private	Con protected cualquier hija puede dañar el dato.	<pre>class Banco { protected double saldo = 100; } class Hack extends Banco { void romper() { saldo = -999; } }</pre>	-999.0

TEMA 41

Polimorfismo: upcasting y enlace dinámico

Polimorfismo significa "muchas formas": una variable de la superclase puede apuntar a objetos de cualquier subclase (upcasting). Al llamar un método sobrescrito, Java elige en ejecución la versión del objeto real (enlace dinámico), no la del tipo de la variable.

RECUERDA Dices "abran" a un grupo con puertas distintas: cada puerta se abre a su manera.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class Figura {
    double area() {
        return 0;
    }
}
class Circulo extends Figura {
    double r = 2;
    @Override
    double area() {
        return Math.PI * r * r;
    }
}
class Rectangulo extends Figura {
    double b = 3, h = 4;
    @Override
    double area() {
        return b * h;
    }
}
```

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Upcasting	Una variable Figura guarda un Circulo.	<pre>Figura f = new Circulo(); System.out.printf("%.2f%n", f.area());</pre>	12.57
Enlace dinámico	La misma llamada f.area() cambia según el objeto.	<pre>Figura f = new Rectangulo(); System.out.println(f.area()); f = new Circulo(); System.out.printf("%.2f%n", f.area());</pre>	12.0 12.57
Arreglo de la superclase	Un solo arreglo guarda subclases distintas.	<pre>Figura[] fs = {new Circulo(), new Rectangulo()}; for (Figura f : fs) { double a = f.area(); System.out.printf("%.2f%n", a); }</pre>	12.57 12.00

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Lista de la superclase	List<Figura> acepta cualquier Figura.	<pre>List<Figura> lista = new ArrayList<>(); lista.add(new Circulo()); lista.add(new Rectangulo()); double t = 0; for (Figura f : lista) t += f.area(); System.out.printf("%.2f%n", t);</pre>	24.57
Parámetro polimórfico	Un método recibe Figura y sirve para todas.	<pre>static void mostrar(Figura f) { double a = f.area(); System.out.printf("%.1f%n", a); }</pre>	12.6 12.0
Método fábrica	Devuelve el tipo base pero crea la subclase.	<pre>static Figura crear(String t) { if (t.equals("c")) { return new Circulo(); } return new Rectangulo(); }</pre>	12.57 12.00
Lo que ves depende del tipo	Con tipo Figura solo ves lo de Figura.	<pre>Figura f = new Circulo(); System.out.println(f.r);</pre>	error: cannot find symbol symbol: variable r location: variable f of type Figura

Circulo tiene radio 2 y Rectangulo mide 3 por 4.

TEMA 42

instanceof, casting y pattern matching

Con una variable de la superclase solo ves los métodos de la superclase. Para usar los de la subclase hay que hacer downcasting con (Perro) a. Si el objeto no es de ese tipo, falla con ClassCastException; por eso se pregunta antes con instanceof o se usa el patrón instanceof Perro p.

RECUERDA Antes de abrir la caja y sacar un perro, mira la etiqueta: instanceof es mirar la etiqueta.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class Animal {
    void comer() {
        System.out.println("Comiendo");
    }
}
class Perro extends Animal {
    void ladrar() {
        System.out.println("Guau");
    }
}
class Gato extends Animal {
    void maullar() {
        System.out.println("Miau");
    }
}
```

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
instanceof	Pregunta si el objeto real es de ese tipo.	<pre>Animal a = new Perro(); System.out.println(a instanceof Perro); System.out.println(a instanceof Gato);</pre>	true false
Downcasting	Convierte hacia abajo para usar lo de la subclase.	<pre>Animal a = new Perro(); Perro p = (Perro) a; p.ladrar();</pre>	Guau
Sin cast no compila	De Animal a Perro hay que convertir a mano.	<pre>Animal a = new Perro(); Perro p = a;</pre>	error: incompatible types: Animal cannot be converted to Perro

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
ClassCastException	El objeto real es un Gato, no un Perro.	<pre>Animal a = new Gato(); Perro p = (Perro) a;</pre>	ClassCastException: class Gato cannot be cast to class Perro
Preguntar antes de castear	Evita la excepción con un if.	<pre>Animal a = new Gato(); if (a instanceof Perro) { ((Perro) a).ladrar(); } else { System.out.println("No es perro"); }</pre>	No es perro
Pattern matching	Pregunta y declara la variable en un solo paso.	<pre>Animal a = new Perro(); if (a instanceof Perro p) { p.ladrar(); }</pre>	Guau
Recorrer una mezcla	Cada elemento hace lo que le toca.	<pre>Animal[] zoo = {new Perro(), new Gato()}; for (Animal x : zoo) { if (x instanceof Perro p) { p.ladrar(); } else if (x instanceof Gato g) { g.maullar(); } }</pre>	Guau Miau
instanceof y null	Con null siempre da false.	<pre>Animal a = null; System.out.println(a instanceof Animal);</pre>	false
getClass() es exacto	instanceof acepta superclases; getClass() no.	<pre>Animal a = new Perro(); System.out.println(a instanceof Animal); Class<?> c = a.getClass(); System.out.println(c == Animal.class);</pre>	true false

Clases abstractas

Una clase abstracta (abstract class) es una plantilla que no se puede instanciar. Puede tener atributos, constructores y métodos con código, y métodos abstractos sin cuerpo que cada subclase concreta debe implementar. Se usan cuando varias clases comparten código pero el objeto "genérico" no tiene sentido.

RECUERDA Una figura "a secas" no existe: existen círculos y cuadrados. Figura es el molde, no el objeto.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

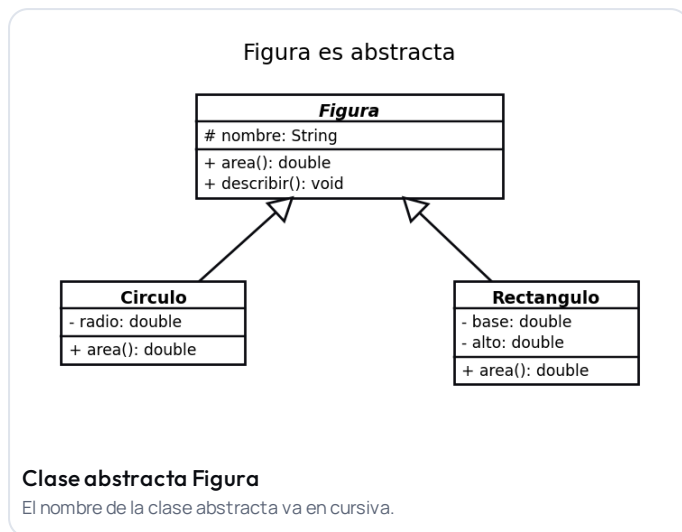
```
abstract class Figura {
    String nombre;
    Figura(String nombre) {
        this.nombre = nombre;
    }
    abstract double area();
    void describir() {
        double a = area();
        System.out.println(nombre + ": " + a);
    }
}

class Cuadrado extends Figura {
    Cuadrado() {
        super("Cuadrado");
    }
    @Override
    double area() {
        return 9;
    }
}
```

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
abstract class	No se puede crear un objeto de una abstracta.	<code>Figura f = new Figura("Cosa");</code>	error: Figura is abstract; cannot be instantiated
Implementar el método abstracto	La subclase concreta da el cuerpo de <code>area()</code> .	<pre>class Triangulo extends Figura { Triangulo() { super("Triangulo"); } @Override double area() { return 6; } }</pre>	Triangulo: 6.0
Olvidar implementarlo	Una subclase concreta debe implementar todo.	<pre>class Raro extends Figura { Raro() { super("Raro"); } }</pre>	error: Raro is not abstract and does not override abstract method <code>area()</code> in <code>Figura</code>
Constructor en abstracta	Existe y se ejecuta vía <code>super(...)</code> .	<pre>abstract class Base { Base() { System.out.println("Base"); } abstract void hacer(); } class Hija extends Base { void hacer() { }</pre>	Base
Referencia abstracta	El tipo es abstracto, el objeto es concreto.	<code>Figura f = new Cuadrado();</code> <code>f.describir();</code>	Cuadrado: 9.0
Método concreto que usa abstracto	<code>describir()</code> llama a <code>area()</code> de cada hija.	<pre>Figura[] fs = {new Cuadrado(), new Cuadrado()}; for (Figura f : fs) { f.describir(); }</pre>	Cuadrado: 9.0 Cuadrado: 9.0

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Clase anónima	Implementa la abstracta al instante, sin nombre.	<pre>Figura f = new Figura("Hoja") { double area() { return 2.5; } }; f.describir();</pre>	Hoja: 2.5
abstract y final juntas	Se contradicen: abstract pide heredar.	<pre>abstract final class X { }</pre>	error: illegal combination of modifiers: abstract and final

Diagramas y ventanas de ejemplo



TEMA 44

Interfaces: declarar, implementar y lambdas

Una interfaz es un contrato: dice qué métodos debe tener una clase, sin decir cómo. Una clase la implementa con implements y puede implementar varias. Sus métodos son public y abstractos y sus atributos son constantes; con default y static puede traer código. Con un solo método abstracto se usa con lambdas.

RECUERDA Una interfaz es el reglamento de un oficio: no hace el trabajo, solo dice qué debes saber hacer.

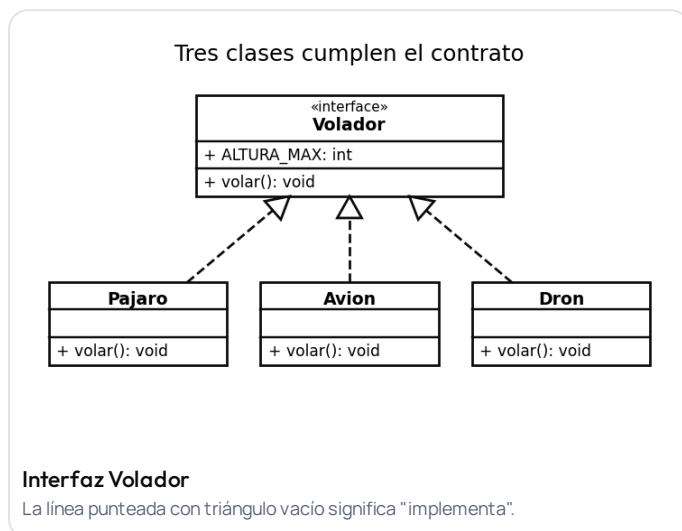
CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
interface Volador {
    int ALTURA_MAX = 100;
    void volar();
}
```

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
implements	La clase cumple el contrato de la interfaz.	<pre>class Pajaro implements Volador { public void volar() { System.out.println("Vuela"); } }</pre>	Vuela
Constantes	Los atributos de una interfaz son public static final.	<pre>System.out.println(Volador.ALTURA_MAX);</pre>	100
Métodos son public	Al implementar hay que escribir public.	<pre>class Piedra implements Volador { void volar() { }</pre>	error: volar() in Piedra cannot implement volar() in Volador attempting to assign weaker access privileges; was public

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Varias interfaces	Una clase puede implementar todas las que quiera.	<pre>class Pato implements Volador, Nadador { public void volar() { System.out.println("Vuela"); } public void nadar() { System.out.println("Nada"); } }</pre>	Vuela Nada
default y static	default trae código heredable; static se llama con la interfaz.	<pre>interface Util { default String hola() { return "Hola"; } static int doble(int x) { return x * 2; } } class Cli implements Util { }</pre>	Hola 42
Interfaz funcional y lambda	Un solo método abstracto: se usa con →.	<pre>interface Operacion { int aplicar(int a, int b); }</pre>	7
Comparable	Define el orden natural con compareTo.	<pre>class Alumno implements Comparable<Alumno> { int nota; Alumno(int n) { nota = n; } public int compareTo(Alumno o) { return nota - o.nota; } }</pre>	6 9
Runnable	Interfaz del JDK sin parámetros ni retorno.	<pre>Runnable r = () -> { System.out.println("Corre"); }; r.run();</pre>	Corre
Function	Recibe un valor y devuelve otro.	<pre>Function<String, Integer> largo = s -> s.length(); int n = largo.apply("ESPOL"); System.out.println(n);</pre>	5

Diagramas y ventanas de ejemplo



Abstracta vs interfaz y la clase Object

Usa clase abstracta cuando las clases son del mismo tipo y comparten estado y código; usa interfaz para añadir una capacidad a clases sin relación. Una clase extiende una sola superclase pero implementa muchas interfaces. Toda clase hereda de Object: toString, equals y getClass.

RECUERDA Abstracta = "qué eres" (solo uno). Interfaz = "qué sabes hacer" (puedes saber muchas cosas).

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
abstract class Empleado {
    String nombre = "Luis";
    abstract double pagar();
}
interface Imprimible {
    void imprimir();
}
```

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
extends una abstracta	Hereda estado y código común.	<pre>class Operario extends Empleado { double pagar() { return 800; } }</pre>	Luis 800.0
implements una interfaz	Solo promete un comportamiento.	<pre>class Reporte implements Imprimible { public void imprimir() { System.out.println("Reporte"); } }</pre>	Reporte
Las dos a la vez	Una superclase y varias interfaces.	<pre>class Gerente extends Empleado implements Imprimible { double pagar() { return 2000; } public void imprimir() { System.out.println("Gerente"); } }</pre>	Gerente 2000.0
Tabla de decisión	Cómo elegir entre las dos.	<pre>abstract class // es un + estado interface // sabe hacer algo</pre>	Mismo tipo y datos comunes: abstract Capacidad sin datos: interface Varias capacidades: interfaces Clases sin relación: interface
toString()	Texto que representa al objeto.	<pre>class Libro { String t = "Java"; @Override public String toString() { return "Libro: " + t; } }</pre>	Libro: Java
equals()	Compara contenido, no la referencia.	<pre>class Punto { int x; Punto(int x) { this.x = x; } @Override public boolean equals(Object o) { return o instanceof Punto p && p.x == x; } }</pre>	true false
getClass().getSimpleName()	Nombre de la clase real del objeto.	<pre>Object o = "hola"; String a = o.getClass().getSimpleName(); List<String> l = new ArrayList<>(); String b = l.getClass().getSimpleName(); System.out.println(a + " " + b);</pre>	String ArrayList

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Object como tipo universal	Una variable Object guarda cualquier cosa.	<pre>Object[] cosas = {"Ana", 5, 2.5}; for (Object o : cosas) { System.out.println(o); }</pre>	Ana 5 2.5

Al sobrescribir equals, también se sobrescribe hashCode (se verá con las colecciones).

TEMA 46

Interfaces selladas y diseño de jerarquías

sealed ... permits limita qué clases pueden heredar de un tipo. Así el compilador conoce todos los casos y un switch exhaustivo no necesita default (Java 21). Al diseñar, hereda solo si hay una relación "es un"; si es "tiene un", usa un atributo (composición).

RECUERDA sealed es una lista de invitados: solo entran los nombrados, y por eso sabes cuántos son.

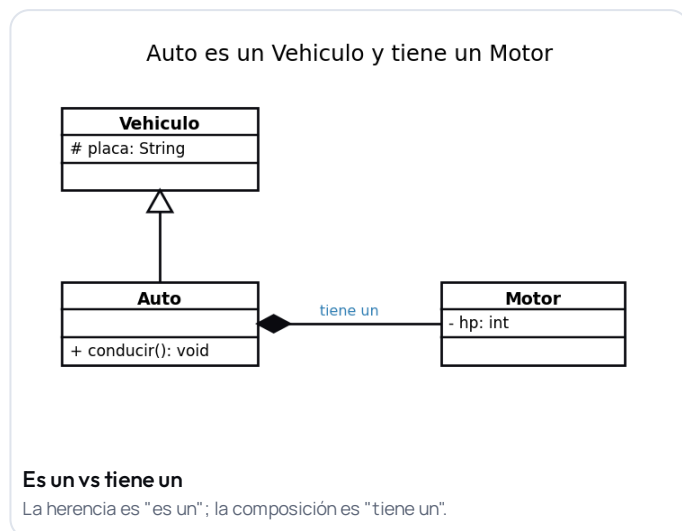
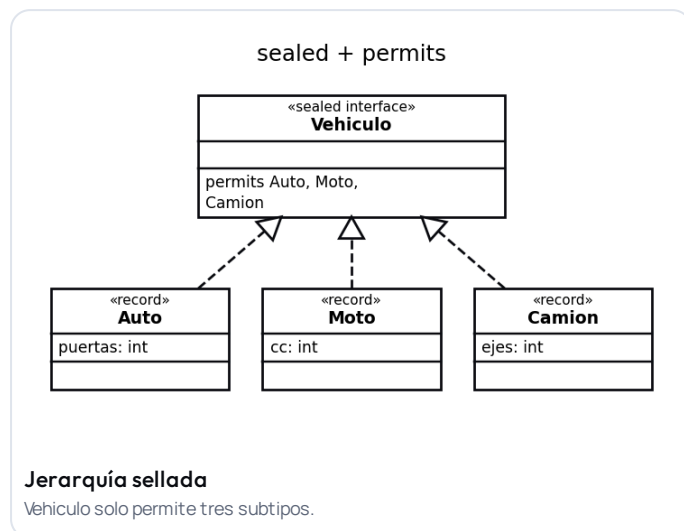
CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
sealed interface Vehiculo
    permits Auto, Moto, Camion {}
record Auto(int puertas)
    implements Vehiculo {}
record Moto(int cc)
    implements Vehiculo {}
record Camion(int ejes)
    implements Vehiculo {}
```

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
sealed / permits	Solo Perro y Gato pueden implementar Animal.	<pre>sealed interface Animal permits Perro, Gato {} record Perro() implements Animal {} record Gato() implements Animal {}</pre>	Perro[]
switch exhaustivo	Sin default: cubre todos los casos permitidos.	<pre>static int peaje(Vehiculo v) { return switch (v) { case Auto a -> 2; case Moto m -> 1; case Camion c -> 5; }; }</pre>	2 5
Falta un caso	El compilador avisa si no cubres todos.	<pre>return switch (v) { case Auto a -> 2; case Moto m -> 1; };</pre>	error: the switch expression does not cover all possible input values
Patrón de registro	Extrae los datos del record en el case.	<pre>Vehiculo v = new Auto(4); String s = switch (v) { case Auto(int p) -> "Auto " + p; case Moto m -> "Moto"; case Camion c -> "Camion"; }; System.out.println(s);</pre>	Auto 4
final, sealed o non-sealed	Cada hija debe elegir cómo seguir.	<pre>sealed class Empleado permits Gerente, Operario {} final class Gerente extends Empleado {} non-sealed class Operario extends Empleado {}</pre>	true
Clase no permitida	Quien no está en permits no puede heredar.	<pre>final class Intruso implements Vehiculo { }</pre>	error: class is not allowed to extend sealed class: Vehiculo (as it is not listed in its 'permits' clause)

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
"Tiene un" (composición)	Un Carro tiene un Motor: es un atributo.	<pre>class Motor { int hp = 120; } class Carro { Motor motor = new Motor(); }</pre>	120
Error: heredar sin "es un"	Una Pila no es una lista: se rompe la regla.	<pre>class Pila extends ArrayList<Integer> { void push(int x) { add(x); } }</pre>	[9, 1, 2]
Errores comunes	Lo que más se ve en jerarquías mal hechas.	<pre>class X extends ArrayList { }</pre>	Heredar solo para reutilizar Jerarquías de más de 3 niveles protected en lugar de private Olvidar @Override

Diagramas y ventanas de ejemplo



Manipulación de archivos y manejo de excepciones

Qué hacer cuando algo falla, y cómo leer, escribir y guardar datos en disco.

TEMA 47

Excepciones: try, catch y finally

Una excepción es un objeto que avisa que algo salió mal mientras el programa corría. Con try pones el código riesgoso, con catch lo atrapas y con finally haces la limpieza final.

RECUERDA try es intentar, catch es el plan B y finally es lo que haces siempre, como apagar la luz al salir.

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Excepción = objeto	Es un objeto normal; no pasa nada hasta que se lanza.	<pre>ArithmeticException e = new ArithmeticException("Aviso"); System.out.println(e.getMessage());</pre>	Aviso
try / catch sin falla	Si no hay error, el catch se salta.	<pre>try { int r = 10 / 2; System.out.println("r = " + r); } catch (ArithmeticException e) { System.out.println("Error"); }</pre>	r = 5
Sin catch: se detiene	Una excepción sin atrapar termina el programa.	<pre>int a = 10, b = 0; System.out.println("antes"); System.out.println(a / b); System.out.println("despues");</pre>	antes ArithmeticException: / by zero
catch atrapa y sigue	Se ejecuta el catch y el programa continúa.	<pre>int a = 10, b = 0; try { System.out.println(a / b); } catch (ArithmeticException e) { System.out.println("Dividir por 0"); } System.out.println("sigo vivo");</pre>	Dividir por 0 sigo vivo
finally sin falla	El bloque finally corre aunque todo salga bien.	<pre>try { System.out.println("trabajo"); } catch (Exception e) { System.out.println("error"); } finally { System.out.println("limpieza"); }</pre>	trabajo limpieza
Flujo cuando falla	Lo que sigue a la falla en el try se salta.	<pre>try { System.out.println("1 antes"); int x = 5 / 0; System.out.println("2 despues"); } catch (ArithmeticException e) { System.out.println("3 catch"); } finally { System.out.println("4 finally"); }</pre>	1 antes 3 catch 4 finally
e.getMessage()	Texto que explica qué pasó. Como str(e) en Python.	<pre>try { int x = 5 / 0; } catch (ArithmeticException e) { System.out.println(e.getMessage()); }</pre>	/ by zero

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Nombre del tipo	getSimpleName() da el tipo de la excepción.	<pre>try { Integer.parseInt("abc"); } catch (Exception e) { System.out.println(e.getClass().getSimpleName()); }</pre>	NumberFormatException

En Python: try/except/finally. En Java el tipo va dentro de catch (Tipo e), como except Tipo as e.

TEMA 48

Jerarquía de excepciones

Todas las excepciones heredan de Throwable, que se divide en Error (fallos graves de la JVM) y Exception. Las RuntimeException no son verificadas: el compilador no te obliga a manejarlas.

RECUERDA Throwable es la familia entera; Error es el incendio del edificio y Exception son los problemas que sí puedes arreglar.

CLASE	DESCRIPCIÓN	EJEMPLO	RESULTADO
Cadena de herencia	Sube por las superclases hasta la raíz.	<pre>Class<?> c = ArithmeticException.class; while (c != null) { System.out.println(c.getSimpleName()); c = c.getSuperclass(); }</pre>	ArithmeticException RuntimeException Exception Throwable Object
Error	Fallos graves de la JVM; no se suelen atrapar.	<pre>class Rec { static int f(int n) { return f(n + 1) + 1; } }</pre>	Error de la JVM
Verificada (checked)	Exception sin RuntimeException: debes manejarla.	<pre>new FileReader("notas.txt");</pre>	error: unreported exception FileNotFoundException; must be caught or declared to be thrown
No verificada (unchecked)	RuntimeException y sus hijas: sin obligación.	<pre>Exception a = new ArithmeticException(); if (a instanceof RuntimeException) { System.out.println("unchecked"); } else { System.out.println("checked"); }</pre>	unchecked
ArithmeticException	Dividir enteros entre cero.	<pre>int a = 10, b = 0; System.out.println(a / b);</pre>	ArithmeticException: / by zero
NullPointerException	Usar una referencia que vale null.	<pre>String s = null; System.out.println(s.length());</pre>	NullPointerException: Cannot invoke "String.length()" because "s" is null
ArrayIndexOutOfBoundsException	Índice fuera del arreglo.	<pre>int[] notas = new int[3]; notas[5] = 10;</pre>	ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 3
NumberFormatException	Texto que no es un número.	<pre>int n = Integer.parseInt("abc");</pre>	NumberFormatException: For input string: "abc"
InputMismatchException	Scanner recibe un tipo distinto del pedido.	<pre>Scanner sc = new Scanner(System.in); int edad = sc.nextInt();</pre>	InputMismatchException: null

Throwable → Exception → RuntimeException → ArithmeticException. Error es hermana de Exception.

Varios catch, multi-catch y finally

Un try puede tener varios catch: entra el primero que coincide. Pon primero los tipos específicos y al final los generales. Con | atrapas varios tipos a la vez, y finally corre siempre.

RECUERDA Los catch son como las ventanillas de un banco: cada una atiende un caso y la última recibe lo que sobra.

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Varios catch	Cada tipo tiene su propio manejo.	<pre>try { Integer.parseInt("x"); } catch (NumberFormatException e) { System.out.println("Número malo"); } catch (ArithmeticException e) { System.out.println("Aritmética"); }</pre>	Número malo
Orden: de lo específico	Lo específico primero, Exception al final.	<pre>try { System.out.println(5 / (1 - 1)); } catch (ArithmeticException e) { System.out.println("División"); } catch (Exception e) { System.out.println("Otro error"); }</pre>	División
Orden mal puesto	El general primero tapa al específico: no compila.	<pre>try { int x = 1 / 0; } catch (Exception e) { } catch (ArithmeticException e) { }</pre>	error: exception ArithmeticException has already been caught
Multi-catch con 	Un solo manejo para varios tipos. Como except (A, B).	<pre>try { int n = Integer.parseInt("12"); System.out.println(n / 0); } catch (NumberFormatException ArithmeticException e) { System.out.println(e.getMessage()); }</pre>	/ by zero
catch (Exception e)	Atrapa casi cualquier excepción.	<pre>try { Object o = "hola"; Integer n = (Integer) o; } catch (Exception e) { System.out.println(e.getClass().getSimpleName()); }</pre>	ClassCastException
finally con return	finally corre incluso si hay return.	<pre>static int probar() { try { return 1; } finally { System.out.println("finally"); } }</pre>	finally 1
try con finally	Sin catch: la excepción sigue, pero finally corre.	<pre>try { System.out.println(1 / 0); } finally { System.out.println("limpieza"); }</pre>	limpieza ArithmeticException: / by zero
try anidado	Si el catch interno no coincide, sube al externo.	<pre>try { try { Integer.parseInt("z"); } catch (ArithmeticException e) { System.out.println("interno"); } } catch (NumberFormatException e) { System.out.println("externo"); }</pre>	externo

Java no tiene else en try; finally equivale al finally de Python.

throw y throws

throw lanza una excepción en ese momento. throws, en la firma del método, avisa que el método puede lanzar una excepción verificada. Si nadie la atrapa, sube a quien llamó al método.

RECUERDA throw es gritar «¡incendio!»; throws es el letrero que avisa que el lugar puede tener incendios.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class Cuenta {
    private double saldo;
    Cuenta(double saldo) {
        this.saldo = saldo;
    }
    double getSaldo() { return saldo; }
    void retirar(double monto) {
        if (monto > saldo) {
            throw new IllegalStateException(
                "Sin fondos");
        }
        saldo -= monto;
    }
}
```

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
throw	Lanza una excepción a propósito. Es el raise de Python.	<code>throw new IllegalStateException("Cuenta cerrada");</code>	IllegalStateException: Cuenta cerrada
Validar y lanzar	El método revisa sus datos y lanza si algo está mal.	<code>Cuenta c = new Cuenta(100); c.retirar(150);</code>	IllegalStateException: Sin fondos
Atrapar al llamar	Quien llama decide qué hacer con el error.	<code>Cuenta c = new Cuenta(100); try { c.retirar(150); } catch (IllegalStateException e) { System.out.println(e.getMessage()); } System.out.println(c.getSaldo());</code>	Sin fondos 100.0
throws en la firma	Declara la excepción verificada que puede lanzar.	<code>static void abrir(String n) throws IOException { if (n.isEmpty()) { throw new IOException("Vacio"); } System.out.println("Abierto " + n); }</code>	Abierto notas.txt Falló: Vacio
Propagación	Sin catch, la excepción sube método por método.	<code>static void c() { System.out.println("en c"); int x = 1 / 0; } static void b() { c(); } static void a() { b(); }</code>	en c atrapada en main
Relanzar con throw e	Atrapas, haces algo y la vuelves a lanzar.	<code>try { new Cuenta(10).retirar(20); } catch (IllegalStateException e) { System.out.println("Registro"); throw e; }</code>	Registro IllegalStateException: Sin fondos
Falta throws	Lanzar una verificada sin manejarla no compila.	<code>static void f() { throw new Exception("x"); }</code>	error: unreported exception Exception; must be caught or declared to be thrown

Las no verificadas (como `IllegalStateException`) no necesitan `throws`.

Excepciones propias

Puedes crear tu propia excepción heredando de `Exception` (verificada) o de `RuntimeException` (no verificada). Se le pasa el mensaje a `super` y, si hace falta, la causa original.

RECUERDA Una excepción propia es una etiqueta con tu nombre: «SinFondos» dice más que «Error».

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class SinFondosException extends Exception {
    private final double falta;
    SinFondosException(double falta) {
        super("Faltan $" + falta);
        this.falta = falta;
    }
    double getFalta() { return falta; }
}

class Cuenta {
    private double saldo = 50;
    void retirar(double m)
        throws SinFondosException {
        if (m > saldo) {
            throw new SinFondosException(
                m - saldo);
        }
        saldo -= m;
    }
}
```

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
extends Exception	Verificada: el compilador exige manejarla.	<pre>class NotaInvalidaException extends Exception { NotaInvalidaException(String m) { super(m); } }</pre>	Nota > 10
extends RuntimeException	No verificada: no exige throws ni catch.	<pre>class EdadInvalidaException extends RuntimeException { EdadInvalidaException(int e) { super("Edad " + e); } }</pre>	EdadInvalidaException: Edad -3
Con causa	<code>super(m, causa)</code> guarda la excepción original.	<pre>class NotaException extends Exception { NotaException(String m, Throwable c) { super(m, c); } }</pre>	Ilegible NumberFormatException
Lanzar la propia	Se lanza y se atrapa como cualquier otra.	<pre>try { new Cuenta().retirar(80); } catch (SinFondosException e) { System.out.println(e.getMessage()); }</pre>	Faltan \$30.0
Dato extra	La clase puede guardar información del error.	<pre>try { new Cuenta().retirar(80); } catch (SinFondosException e) { System.out.println(e.getFalta()); }</pre>	30.0
println(e)	Imprime el tipo y el mensaje juntos.	<pre>try { new Cuenta().retirar(80); } catch (Exception e) { System.out.println(e); }</pre>	SinFondosException: Faltan \$30.0
Sin atrapar	Si es verificada, el método debe declarar throws.	<pre>new Cuenta().retirar(80);</pre>	SinFondosException: Faltan \$30.0

TEMA 52

try-with-resources y AutoCloseable

Un recurso (archivo, conexión) hay que cerrarlo siempre. Con try (recurso) Java llama a close() solo, aunque haya error. Funciona con cualquier clase que implemente AutoCloseable.

RECUERDA Es el with open() de Python: abres, usas, y Java cierra la puerta por ti al salir.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class Recurso implements AutoCloseable {
    private final String id;
    Recurso(String id) {
        this.id = id;
        System.out.println("abre " + id);
    }
    @Override
    public void close() {
        System.out.println("cierra " + id);
    }
}
```

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Cierre manual	Fácil de olvidar si algo falla antes.	<pre>Recurso r = new Recurso("A"); System.out.println("usa A"); r.close();</pre>	abre A usa A cierra A
try (recurso)	close() se llama solo al terminar el bloque.	<pre>try (Recurso r = new Recurso("A")) { System.out.println("usa A"); }</pre>	abre A usa A cierra A
Cierra aunque falle	close() corre antes del catch.	<pre>try (Recurso r = new Recurso("A")) { int x = 1 / 0; } catch (ArithmeticException e) { System.out.println("catch"); }</pre>	abre A cierra A catch
Varios recursos	Se cierran en orden inverso al de apertura.	<pre>try (Recurso a = new Recurso("A"); Recurso b = new Recurso("B")) { System.out.println("usa A y B"); }</pre>	abre A abre B usa A y B cierra B cierra A
Con catch y finally	Orden: close, catch y por último finally.	<pre>try (Recurso r = new Recurso("R")) { throw new RuntimeException("x"); } catch (RuntimeException e) { System.out.println("catch"); } finally { System.out.println("finally"); }</pre>	abre R cierra R catch finally
Clases de la API	Scanner, BufferedReader y PrintWriter ya lo son.	<pre>try (Scanner sc = new Scanner("12 7")) { int a = sc.nextInt(); int b = sc.nextInt(); System.out.println(a + b); }</pre>	19
try (variable existente)	Java 9: usar una variable ya declarada.	<pre>Recurso r = new Recurso("Z"); try (r) { System.out.println("usa Z"); }</pre>	abre Z usa Z cierra Z
Sin AutoCloseable	El tipo debe implementar AutoCloseable.	<pre>try (String s = "hola") { System.out.println(s); }</pre>	error: incompatible types: try-with-resources not applicable to variable type

close() de AutoCloseable puede lanzar Exception; aquí la sobrescribimos sin throws.

Archivos y rutas

Una ruta se representa con Path. La clase Files crea, consulta, lista y borra archivos y carpetas. La clase File es la versión antigua, pero aún se ve mucho en libros y código viejo.

RECUERDA Path es la dirección escrita en un papel; Files es el cartero que va y hace cosas en esa dirección.

MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Path.of(...)	Arma una ruta. Paths.get() hace lo mismo.	<pre>Path p = Path.of("datos", "notas.txt"); System.out.println(p);</pre>	datos/notas.txt
getFileName y getParent	Separan el nombre del archivo y su carpeta.	<pre>Path p = Path.of("datos", "notas.txt"); System.out.println(p.getFileName()); System.out.println(p.getParent());</pre>	notas.txt datos
Files.createDirectory	Crea una carpeta nueva.	<pre>Path dir = Path.of("datos"); Files.createDirectory(dir); System.out.println(Files.exists(dir));</pre>	true
Files.createFile	Crea un archivo vacío. Falla si ya existe.	<pre>Path p = Path.of("datos", "notas.txt"); Files.createFile(p); System.out.println(Files.exists(p));</pre>	true
Files.exists y size	Pregunta si existe y cuántos bytes pesa.	<pre>Path p = Path.of("datos/notas.txt"); System.out.println(Files.exists(p)); System.out.println(Files.size(p)); Path q = Path.of("x.txt"); System.out.println(Files.exists(q));</pre>	true 0 false
File (clase antigua)	Mismas preguntas con la clase File.	<pre>File f = new File("datos/notas.txt"); System.out.println(f.getName()); System.out.println(f.isFile()); System.out.println(f.length());</pre>	notas.txt true 0
Listar una carpeta	Files.list entrega el contenido; se ordena aparte.	<pre>Path d = Path.of("datos"); Files.createFile(d.resolve("b.txt")); Files.createFile(d.resolve("a.txt")); try (var s = Files.list(d)) { s.map(Path::getFileName).sorted() .forEach(System.out::println); }</pre>	a.txt b.txt notas.txt
Borrar	delete falla si no existe; deleteIfExists avisa con boolean.	<pre>Path p = Path.of("datos/a.txt"); Files.delete(p); System.out.println(Files.exists(p)); boolean b = Files.deleteIfExists(p); System.out.println(b);</pre>	false false
Archivo inexistente	Leer algo que no existe lanza una IOException.	<pre>Path p = Path.of("fantasma.txt"); Files.readString(p);</pre>	NoSuchFileException: fantasma.txt

Cada tema corre en una carpeta vacía. En Windows el separador de Path es \ en vez de /.

Leer y escribir texto

Para textos cortos, Files.writeString y readAllLines hacen todo en una línea. Para más control están FileWriter, PrintWriter, BufferedWriter, BufferedReader y Scanner, siempre dentro de try-with-resources.

RECUERDA Escribir es llenar un cuaderno; leer es pasar las páginas. Cierra el cuaderno al terminar, siempre.

CLASE O MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
writeString y readString	Escribe y lee todo el texto de una vez.	<pre>Path p = Path.of("notas.txt"); Files.writeString(p, "Ana,9\nLuis,7\n"); System.out.println(Files.readString(p));</pre>	Ana,9 Luis,7

CLASE O MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Files.readAllLines	Devuelve una lista con una línea por elemento.	<pre>Path p = Path.of("notas.txt"); List<String> l = Files.readAllLines(p); System.out.println(l.size()); System.out.println(l.get(0));</pre>	2 Ana,9
Files.write con lista	Escribe cada elemento de la lista como línea.	<pre>Path p = Path.of("lista.txt"); Files.write(p, List.of("a", "b", "c")); var l = Files.readAllLines(p); System.out.println(l);</pre>	[a, b, c]
FileWriter y PrintWriter	PrintWriter ofrece println y printf.	<pre>try (PrintWriter pw = new PrintWriter(new FileWriter("inv.txt"))) { pw.println("Mouse,15"); pw.printf("Teclado,%d%n", 8); } System.out.println(Files.readAllLines(Path.of("inv.txt")));</pre>	[Mouse,15, Teclado,8]
Agregar (append)	El segundo argumento true no borra lo anterior.	<pre>try (FileWriter fw = new FileWriter("inv.txt", true)) { fw.write("Monitor,3\n"); } System.out.println(Files.readAllLines(Path.of("inv.txt")));</pre>	[Mouse,15, Teclado,8, Monitor,3]
BufferedWriter	Escribe con buffer; newLine() separa líneas.	<pre>try (var bw = new BufferedWriter(new FileWriter("nom.txt"))) { bw.write("Ana"); bw.newLine(); bw.write("Luis"); bw.newLine(); } System.out.println("Archivo listo");</pre>	Archivo listo
BufferedReader	readLine() devuelve null al terminar el archivo.	<pre>try (var br = new BufferedReader(new FileReader("nom.txt"))) { String l = br.readLine(); while (l != null) { System.out.println(l); l = br.readLine(); } }</pre>	Ana Luis
Scanner(File)	Lee con hasNextLine y nextLine, como el teclado.	<pre>File f = new File("notas.txt"); try (Scanner sc = new Scanner(f)) { while (sc.hasNextLine()) { String l = sc.nextLine(); System.out.println(l); } }</pre>	Ana,9 Luis,7
Archivo faltante	Scanner(File) lanza FileNotFoundException.	<pre>try { new Scanner(new File("no.txt")); } catch (FileNotFoundException e) { System.out.println("Sin archivo"); }</pre>	Sin archivo

Equivale a `open(...).read()` y `readlines()` de Python. Aquí el `with` es `try-with-resources`.

Procesar datos: CSV de estudiantes

Un CSV guarda un registro por línea con los campos separados por coma. Para usarlo: split corta la línea, parseInt y parseDouble convierten el texto y con eso creas objetos.

RECUERDA Una línea CSV es una ficha en tira: la cortas con split y armas el objeto con cada pedazo.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class Estudiante {
    private String nombre;
    private int edad;
    private double nota;
    Estudiante(String nombre, int edad,
        double nota) {
        this.nombre = nombre;
        this.edad = edad;
        this.nota = nota;
    }
    String getNombre() { return nombre; }
    double getNota() { return nota; }
    String aCsv() {
        return nombre + "," + edad
            + "," + nota;
    }
    static Estudiante deCsv(String linea) {
        String[] p = linea.split(",");
        return new Estudiante(p[0],
            Integer.parseInt(p[1]),
            Double.parseDouble(p[2]));
    }
}
```

PASO	DESCRIPCIÓN	EJEMPLO	RESULTADO
split(",")	Parte el texto por cada coma.	<pre>String[] p = "Ana,19,9.5".split(","); System.out.println(p.length); System.out.println(p[0]);</pre>	3 Ana
parseInt y parseDouble	Convierten texto a número.	<pre>int e = Integer.parseInt("19"); double n = Double.parseDouble("9.5"); System.out.println(e + 1); System.out.println(n * 2);</pre>	20 19.0
trim()	Quita espacios que sobran en cada campo.	<pre>String campo = " 19 "; int e = Integer.parseInt(campo.trim()); System.out.println(e);</pre>	19
Línea a objeto	deCsv arma un Estudiante desde una línea.	<pre>String l = "Ana,19,9.5"; Estudiante e = Estudiante.deCsv(l); System.out.println(e.getNombre()); System.out.println(e.getNota());</pre>	Ana 9.5
Guardar la lista	aCsv de cada objeto se escribe como línea.	<pre>var lista = List.of(new Estudiante("Ana", 19, 9.5), new Estudiante("Luis", 20, 7.0)); var lineas = lista.stream() .map(Estudiante::aCsv).toList(); Files.write(Path.of("est.csv"), lineas); System.out.println(lineas);</pre>	[Ana,19,9.5, Luis,20,7.0]
Cargar la lista	Cada línea del archivo vuelve a ser un objeto.	<pre>var lineas = Files.readAllLines(Path.of("est.csv")); var lista = new ArrayList<Estudiante>(); for (String l : lineas) { lista.add(Estudiante.deCsv(l)); } System.out.println(lista.size()); Estudiante e = lista.get(1); System.out.println(e.getNombre());</pre>	2 Luis

PASO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Promedio de la columna	Recorrer los objetos y acumular.	<pre>var lineas = Files.readAllLines(Path.of("est.csv")); double suma = 0; for (String l : lineas) { Estudiante e = Estudiante.deCsv(l); suma += e.getNota(); } double prom = suma / lineas.size(); System.out.println(prom);</pre>	8.25
Encabezado y vacías	skip(1) salta el título; filter quita vacías.	<pre>var l = List.of("nombre,edad,nota", "Ana,19,9.5", "", "Luis,20,7.0"); l.stream().skip(1) .filter(s -> !s.isBlank()) .map(Estudiante::deCsv) .map(Estudiante::getNombre) .forEach(System.out::println);</pre>	Ana Luis
Línea mal formada	Atrapa el error y sigue con las demás.	<pre>String[] d = {"Ana,19,9.5", "Luis,x,7"}; for (String l : d) { try { var s = Estudiante.deCsv(l); System.out.println(s.aCsv()); } catch (NumberFormatException e) { System.out.println("Ignorada"); } }</pre>	Ana,19,9.5 Ignorada

Con un CSV real (con comillas o comas dentro del texto) conviene una librería de CSV.

TEMA 56

Serialización y flujos binarios

Serializar es convertir un objeto en bytes para guardarlo en un archivo y recuperarlo después. La clase debe implementar Serializable; los campos transient no se guardan. DataOutputStream escribe tipos básicos en binario.

RECUERDA Serializar es congelar el objeto en una caja; al leerlo lo descongelas igualito (menos lo marcado transient).

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class Estudiante implements Serializable {
    private String nombre;
    private double nota;
    private transient String clave;
    Estudiante(String nombre, double nota) {
        this.nombre = nombre;
        this.nota = nota;
    }
    String getNombre() { return nombre; }
    double getNota() { return nota; }
    String getClave() { return clave; }
    void setClave(String c) { clave = c; }
}
class Libro { }
```

CLASE O MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
writeObject	Guarda el objeto completo en un archivo.	<pre>var e = new Estudiante("Ana", 9.5); e.setClave("abc123"); String f = "est.dat"; try (var out = new ObjectOutputStream(new FileOutputStream(f))) { out.writeObject(e); } System.out.println("Guardado");</pre>	Guardado

CLASE O MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
readObject	Lo recupera; hay que hacer el cast.	<pre>String f = "est.dat"; try (var in = new ObjectInputStream(new FileInputStream(f))) { Object o = in.readObject(); Estudiante e = (Estudiante) o; System.out.println(e.getNombre()); System.out.println(e.getNota()); }</pre>	Ana 9.5
transient	Un campo transient no se guarda y vuelve en null.	<pre>String f = "est.dat"; try (var in = new ObjectInputStream(new FileInputStream(f))) { Object o = in.readObject(); Estudiante e = (Estudiante) o; System.out.println(e.getClave()); }</pre>	null
Guardar una lista	ArrayList ya es Serializable y guarda sus objetos.	<pre>var lista = new ArrayList<Estudiante>(); lista.add(new Estudiante("Ana", 9.5)); lista.add(new Estudiante("Luis", 7.0)); String f = "lista.dat"; try (var out = new ObjectOutputStream(new FileOutputStream(f))) { out.writeObject(lista); } System.out.println(lista.size());</pre>	2
Cargar la lista	Se lee de vuelta con readObject y un cast.	<pre>String f = "lista.dat"; try (var in = new ObjectInputStream(new FileInputStream(f))) { Object o = in.readObject(); var l = (ArrayList<Estudiante>) o; System.out.println(l.size()); Estudiante e = l.get(1); System.out.println(e.getNombre()); }</pre>	2 Luis
No Serializable	Sin implements Serializable, writeObject falla.	<pre>String f = "libro.dat"; try (var out = new ObjectOutputStream(new FileOutputStream(f))) { out.writeObject(new Libro()); }</pre>	NotSerializableException: Libro
DataOutputStream	Escribe int, double y texto en binario.	<pre>String f = "n.bin"; try (var out = new DataOutputStream(new FileOutputStream(f))) { out.writeUTF("Ana"); out.writeInt(19); out.writeDouble(9.5); } long n = Files.size(Path.of(f)); System.out.println(n);</pre>	17
DataInputStream	Se lee en el mismo orden en que se escribió.	<pre>String f = "n.bin"; try (var in = new DataInputStream(new FileInputStream(f))) { String n = in.readUTF(); int e = in.readInt(); double d = in.readDouble(); System.out.println(n + " " + e); System.out.println(d); }</pre>	Ana 19 9.5
Fin del archivo	Leer de más lanza EOFException.	<pre>String f = "n.bin"; try (var in = new DataInputStream(new FileInputStream(f))) { in.skipBytes(17); in.readInt(); } catch (EOFException e) { System.out.println("Fin de datos"); }</pre>	Fin de datos

Buena práctica: agregar `private static final long serialVersionUID = 1L;` a la clase.

Programación con interfaces gráficas y manejo de eventos

Ventanas, componentes y eventos con Swing: de la primera ventana al patrón MVC.

TEMA 57

Primera ventana Swing

Una ventana Swing es un objeto `JFrame`. Le pones título, tamaño y qué hacer al cerrarla, le agregas componentes y al final la haces visible. Todo eso se hace dentro de `SwingUtilities.invokeLater` para usar el hilo de eventos.

RECUERDA Armar una ventana es como montar un cuadro: primero marco y tamaño, luego la imagen, y al final lo cuelgas en la pared (`setVisible`).

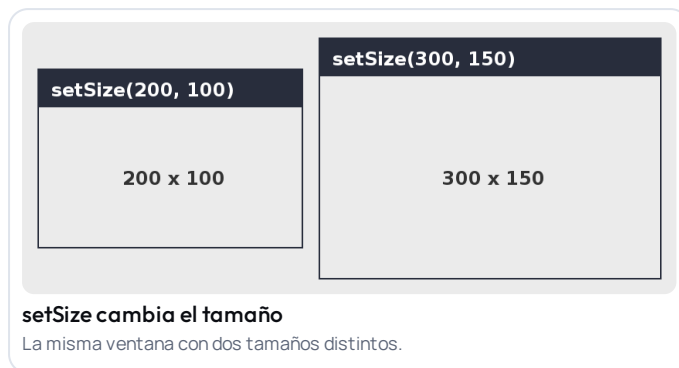
CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
import javax.swing.*;
```

MÉTODO O CLASE	DESCRIPCIÓN	EJEMPLO	RESULTADO
JFrame	Es la ventana principal; el título va en el constructor.	<code>JFrame f = new JFrame("Mi ventana");</code>	Crea la ventana (aún no se ve) Título: Mi ventana
setSize	Fija el ancho y el alto en píxeles.	<code>f.setSize(300, 150);</code>	Ventana de 300 px de ancho y 150 px de alto
setDefaultCloseOperation	Dice qué pasa al pulsar la X de la ventana.	<code>f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);</code>	Al cerrar la ventana, el programa termina
Constantes de cierre	Las cuatro opciones de cierre son números enteros.	<pre>int a = JFrame.DO_NOTHING_ON_CLOSE; int b = JFrame.HIDE_ON_CLOSE; int c = JFrame.DISPOSE_ON_CLOSE; int d = JFrame.EXIT_ON_CLOSE; System.out.println(a + " " + b); System.out.println(c + " " + d);</pre>	0 1 2 3
add	Agrega un componente al área de contenido.	<code>f.add(new JLabel("Hola ESPOL"));</code>	La etiqueta aparece dentro de la ventana
setLocationRelativeTo(null)	Centra la ventana en la pantalla.	<code>f.setLocationRelativeTo(null);</code>	La ventana sale centrada
setVisible(true)	Muestra la ventana; sin esto no se ve.	<code>f.setVisible(true);</code>	La ventana aparece en pantalla
SwingUtilities.invokeLater	Crea y muestra la GUI en el hilo de eventos.	<pre>SwingUtilities.invokeLater(() -> { JFrame f = new JFrame("Hola"); f.setSize(300, 150); f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); f.add(new JLabel("Hola ESPOL")); f.setVisible(true); });</pre>	Abre una ventana de 300 x 150 con el texto Hola ESPOL
isEventDispatchThread	Indica si el código corre en el hilo de eventos (EDT).	<pre>boolean a = SwingUtilities .isEventDispatchThread(); System.out.println("main: " + a); SwingUtilities.invokeAndWait(() -> { boolean b = SwingUtilities .isEventDispatchThread(); System.out.println("EDT: " + b); });</pre>	main: false EDT: true

JFrame necesita pantalla: aquí se muestran las ventanas como imágenes reales generadas con Swing.

Diagramas y ventanas de ejemplo



TEMA 58

Componentes básicos de Swing

Los componentes son las piezas de la ventana: etiquetas, botones, campos de texto, casillas, listas. Todos son objetos: los creas con new y lees su estado con métodos como getText, isSelected o getSelectedItem.

RECUERDA Los componentes son piezas de lego: cada uno tiene su forma y se acopla a un contenedor para armar la ventana.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)
`import javax.swing.*;`

COMPONENTE	DESCRIPCIÓN	EJEMPLO	RESULTADO
JLabel	Muestra un texto fijo que el usuario no edita.	<pre>JLabel l = new JLabel("Nombre:"); System.out.println(l.getText()); l.setText("Apellido:"); System.out.println(l.getText());</pre>	Nombre: Apellido:
JButton	Botón que se pulsa; se puede activar o desactivar.	<pre>JButton b = new JButton("Guardar"); System.out.println(b.getText()); b.setEnabled(false); System.out.println(b.isEnabled());</pre>	Guardar false
TextField	Campo de una línea para escribir texto.	<pre>TextField t = new TextField("Ana", 8); System.out.println(t.getText()); System.out.println(t.getColumns()); t.setText("Luis"); System.out.println(t.getText());</pre>	Ana 8 Luis
PasswordField	Campo que oculta lo escrito; se lee con getPassword.	<pre>PasswordField p = new PasswordField(); p.setText("abc123"); char[] c = p.getPassword(); System.out.println(c.length); System.out.println(new String(c));</pre>	6 abc123

COMPONENTE	DESCRIPCIÓN	EJEMPLO	RESULTADO
JTextArea y JScrollPane	Texto de varias líneas; el JScrollPane le da barras.	<pre> JTextArea a = new JTextArea(4, 20); a.append("Línea 1\n"); a.append("Línea 2"); System.out.println(a.getLineCount()); JScrollPane sp = new JScrollPane(a); System.out.println(sp.getViewport().getView() == a); </pre>	2 true
JCheckBox	Casilla que se marca o desmarca.	<pre> JCheckBox c = new JCheckBox("Acepto"); System.out.println(c.isSelected()); c.setSelected(true); System.out.println(c.isSelected()); System.out.println(c.getText()); </pre>	false true Acepto
JRadioButton y ButtonGroup	Opciones excluyentes: solo una queda marcada.	<pre> JRadioButton a = new JRadioButton("AM"); JRadioButton b = new JRadioButton("PM"); ButtonGroup g = new ButtonGroup(); g.add(a); g.add(b); a.setSelected(true); b.setSelected(true); System.out.println(a.isSelected()); System.out.println(b.isSelected()); </pre>	false true
JComboBox	Lista desplegable; se lee con <code>getSelectedItem</code> .	<pre> String[] n = {"Java", "Python", "C"}; JComboBox<String> c = new JComboBox<>(n); System.out.println(c.getSelectedItem()); c.setSelectedIndex(2); System.out.println(c.getSelectedItem()); System.out.println(c.getItemCount()); </pre>	Java C 3
JList	Lista visible de opciones; permite elegir un elemento.	<pre> String[] n = {"Ana", "Luis", "Eva"}; JList<String> l = new JList<>(n); l.setSelectedIndex(1); String v = l.getSelectedValue(); System.out.println(v); int k = l.getModel().getSize(); System.out.println(k); </pre>	Luis 3

Aquí se crean y se leen por código; en la galería se ven dibujados dentro de una ventana.

Diagramas y ventanas de ejemplo

Inicio de sesión

Nombre:

Clave:

Cancelar

Entrar

Etiquetas, campos y botones

JLabel, JTextField, JPasswordField y JButton.

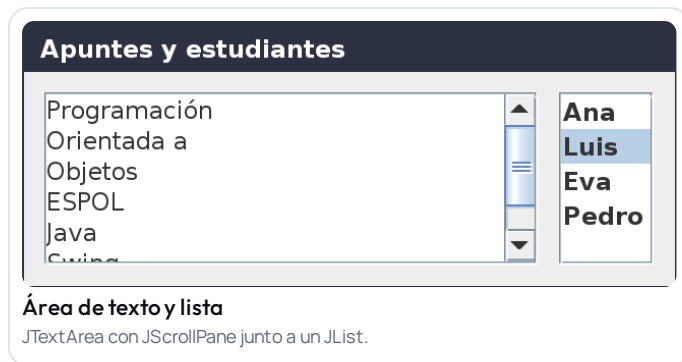
Matrícula

☒ Acepto los términos

☐ AM
☒ PM

Casillas, opciones y combo

JCheckBox, JRadioButton y JComboBox.



TEMA 59

Contenedores y paneles

Un JPanel es una caja que agrupa componentes y puede tener su propio diseño y borde. Los paneles se anidan unos dentro de otros. JScrollPane agrega barras de desplazamiento y JTabbedPane organiza el contenido en pestañas.

RECUERDA Los paneles son cajones con divisiones: cada cajón guarda sus cosas y todos juntos caben en el mueble (la ventana).

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
import javax.swing.*;
import javax.swing.border.TitledBorder;
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Insets;
```

CONTENEDOR O BORDE	DESCRIPCIÓN	EJEMPLO	RESULTADO
JPanel	Agrupar componentes; add los mete y getCount los cuenta.	<pre>JPanel p = new JPanel(); p.add(new JButton("OK")); p.add(new JLabel("Listo")); int n = p.getComponentCount(); System.out.println(n);</pre>	2
Diseño por defecto	Un JPanel nuevo usa FlowLayout.	<pre>JPanel p = new JPanel(); Object l = p.getLayout(); String s = l.getClass().getSimpleName(); System.out.println(s);</pre>	FlowLayout
Anidar paneles	Un panel puede ir dentro de otro panel.	<pre>JPanel f = new JPanel(); f.setLayout(new BorderLayout()); JPanel d = new JPanel(); d.add(new JButton("A")); f.add(d, BorderLayout.CENTER); System.out.println(d.getParent() == f); int n = f.getComponentCount(); System.out.println(n);</pre>	true 1
Borde con título	createTitledBorder rodea el panel con un título.	<pre>JPanel p = new JPanel(); p.setBorder(BorderFactory .createTitledBorder("Datos")); TitledBorder t = (TitledBorder) p.getBorder(); System.out.println(t.getTitle());</pre>	Datos
Borde vacío	createEmptyBorder deja un margen interior.	<pre>JPanel p = new JPanel(); p.setBorder(BorderFactory .createEmptyBorder(5, 10, 5, 10)); Insets i = p.getInsets(); System.out.println(i.top); System.out.println(i.left);</pre>	5 10
Borde de línea	createLineBorder dibuja un marco de color y grosor.	<pre>JPanel p = new JPanel(); p.setBorder(BorderFactory .createLineBorder(Color.BLUE, 3)); System.out.println(p.getInsets().top);</pre>	3

CONTENEDOR O BORDE	DESCRIPCIÓN	EJEMPLO	RESULTADO
JScrollPane	Envuelve un componente grande y le pone barras.	<pre> JTextArea a = new JTextArea(5, 20); JScrollPane sp = new JScrollPane(a); System.out.println(sp.getViewport().getView() == a); int v = sp.getVerticalScrollBarPolicy(); System.out.println(v); </pre>	true 20
JTabbedPane	Muestra varios paneles como pestañas.	<pre> JTabbedPane tp = new JTabbedPane(); tp.addTab("Datos", new JPanel()); tp.addTab("Notas", new JPanel()); System.out.println(tp.getTabCount()); tp.setSelectedIndex(1); System.out.println(tp.getTitleAt(tp.getSelectedIndex())); </pre>	2 Notas
remove	Quita un componente del panel.	<pre> JPanel p = new JPanel(); JButton b = new JButton("X"); p.add(b); p.add(new JLabel("Y")); p.remove(b); int n = p.getComponentCount(); System.out.println(n); </pre>	1

Diagramas y ventanas de ejemplo

Registro

Datos del estudiante

Nombre:

Carrera:

Acciones

Paneles anidados con bordes
Un panel principal con dos paneles titulados.

Ficha del estudiante

Datos

Notas

Horario

Nombre:

Edad:

Pestañas con JTabbedPane
Tres pestañas; se ve el contenido de una.

Registro de eventos

Línea de texto número 1

Línea de texto número 2

Línea de texto número 3

Línea de texto número 4

Línea de texto número 5

Línea de texto número 6

Línea de texto número 7

JScrollPane con barras
Texto largo dentro de un área con desplazamiento.

Bordes

Línea

Título

Con título

Compuesto

Biselado

Cuatro tipos de borde
Línea, título, vacío con línea y biselado.

Administradores de diseño (layouts)

Un layout decide dónde y de qué tamaño va cada componente. FlowLayout los pone en fila, BorderLayout usa cinco zonas, GridLayout una cuadrícula, BoxLayout una columna o fila y GridBagLayout una cuadrícula flexible.

RECUERDA El layout es el acomodador de sala: tú traes las sillas (componentes) y él decide en qué lugar va cada una.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
import javax.swing.*;
import java.awt.BorderLayout;
import java.awt.FlowLayout;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.GridLayout;
```

LAYOUT	DESCRIPCIÓN	EJEMPLO	RESULTADO
FlowLayout	Pone los componentes en fila y baja de línea al llenarse.	<pre>FlowLayout f = new FlowLayout(FlowLayout.LEFT); JPanel p = new JPanel(f); p.add(new JButton("A")); p.add(new JButton("B")); System.out.println(f.getAlignment()); System.out.println(f.getHgap());</pre>	0 5
BorderLayout	Cinco zonas: norte, sur, este, oeste y centro.	<pre>BorderLayout b = new BorderLayout(); JPanel p = new JPanel(b); JButton c = new JButton("Centro"); p.add(c, BorderLayout.CENTER); Object o = b.getLayoutComponent(BorderLayout.CENTER); System.out.println(o == c);</pre>	true
Zonas de BorderLayout	Son constantes de texto de la clase.	<pre>System.out.println(BorderLayout.NORTH); System.out.println(BorderLayout.SOUTH); System.out.println(BorderLayout.EAST); System.out.println(BorderLayout.WEST); System.out.println(BorderLayout.CENTER);</pre>	North South East West Center
GridLayout	Cuadrícula de celdas iguales (filas, columnas).	<pre>GridLayout g = new GridLayout(2, 3); JPanel p = new JPanel(g); for (int i = 1; i <= 6; i++) { p.add(new JButton("" + i)); } System.out.println(g.getRows()); System.out.println(g.getColumns());</pre>	2 3
BoxLayout	Apila componentes en vertical o los alinea en horizontal.	<pre>JPanel p = new JPanel(); BoxLayout b = new BoxLayout(p, BoxLayout.Y_AXIS); p.setLayout(b); p.add(new JButton("Uno")); p.add(new JButton("Dos")); System.out.println(b.getAxis());</pre>	1
GridBagLayout	Cuadrícula flexible; GridBagConstraints indica la celda.	<pre>GridBagConstraints c = new GridBagConstraints(); c.gridx = 1; c.gridy = 0; c.fill = GridBagConstraints.HORIZONTAL; JPanel p = new JPanel(); p.setLayout(new GridBagLayout()); p.add(new JTextField(8), c); System.out.println(c.gridx);</pre>	1
setLayout	Cambia el diseño de un contenedor ya creado.	<pre>JPanel p = new JPanel(); p.setLayout(new GridLayout(1, 2)); Object l = p.getLayout(); String s = l.getClass().getSimpleName(); System.out.println(s);</pre>	GridLayout

LAYOUT	DESCRIPCIÓN	EJEMPLO	RESULTADO
Layout nulo	Con null tú posicionas todo; se evita por ser frágil.	<pre>JPanel p = new JPanel(); p.setLayout(null); System.out.println(p.getLayout());</pre>	null
Combinar layouts	Paneles con distinto layout dentro de otro.	<pre>JPanel p = new JPanel(); p.setLayout(new BorderLayout()); JPanel sur = new JPanel(); sur.add(new JButton("Ok")); sur.add(new JButton("Salir")); p.add(new JTextArea(3, 10)); p.add(sur, BorderLayout.SOUTH); int n = sur.getComponentCount(); System.out.println(n);</pre>	2

Diagramas y ventanas de ejemplo

FlowLayout y BoxLayout

FlowLayout **BoxLayout (Y)**

FlowLayout y BoxLayout
Fila de botones a la izquierda; columna a la derecha.

BorderLayout

BorderLayout
Cinco zonas: norte, sur, este, oeste y centro.

GridLayout 3 x 3

GridLayout
Cuadrícula de 3 por 3 como un teclado numérico.

GridBagLayout

GridBagLayout
Formulario con campos que se estiran.

Eventos: el modelo de delegación

Cuando el usuario hace algo, la fuente (un botón) crea un evento (ActionEvent) y se lo entrega a sus oyentes (ActionListener). Tú solo escribes qué hace el oyente. Puede ser una clase, una clase anónima o una lambda.

RECUERDA Es como un timbre: el botón es el timbre, el evento es el sonido y el oyente es quien abre la puerta.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
import javax.swing.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

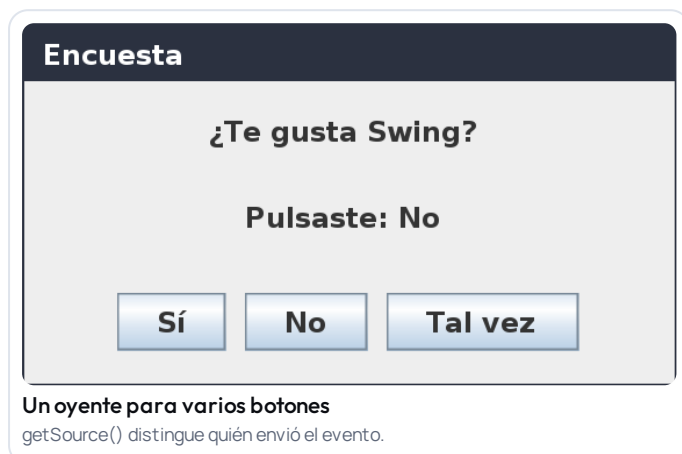
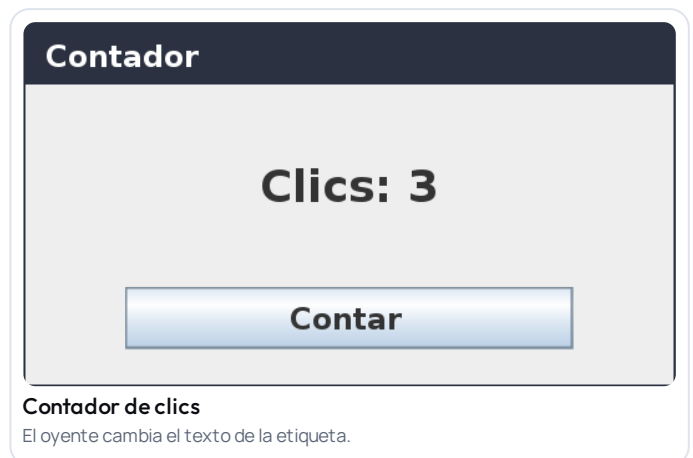
class Aviso implements ActionListener {
    private final String texto;
    Aviso(String texto) {
        this.texto = texto;
    }
    public void actionPerformed(
        ActionEvent e) {
        System.out.println(texto);
    }
}
```

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Fuente, evento y oyente	La fuente avisa al oyente cuando pasa algo.	<code>btn.addActionListener(oyente);</code>	fuente: btn (JButton) evento: ActionEvent oyente: el objeto registrado
Clase que implementa ActionListener	Un oyente propio, reutilizable.	<pre>class Saludo implements ActionListener { public void actionPerformed(ActionEvent e) { System.out.println("Hola!"); } }</pre>	Hola!
Clase anónima	Oyente sin nombre, escrito en el mismo lugar.	<pre>JButton b = new JButton("OK"); b.addActionListener(new ActionListener() { public void actionPerformed(ActionEvent e) { System.out.println("Guardado"); } }); b.doClick();</pre>	Guardado
Lambda	ActionListener tiene un solo método: cabe en una lambda.	<pre>JButton b = new JButton("Saludar"); b.addActionListener(e -> System.out.println("Hola")); b.doClick(); b.doClick();</pre>	Hola Hola
doClick()	Simula un clic por código: sirve para probar oyentes.	<pre>JButton b = new JButton("Contar"); b.addActionListener(new Aviso("clic")); System.out.println("antes"); b.doClick(); System.out.println("despues");</pre>	antes clic despues
getSource()	Dice cuál componente produjo el evento.	<pre>JButton a = new JButton("Si"); JButton n = new JButton("No"); ActionListener l = e -> { JButton f = (JButton) e.getSource(); System.out.println(f.getText()); }; a.addActionListener(l); n.addActionListener(l); a.doClick(); n.doClick();</pre>	Si No

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
getActionCommand()	Texto que identifica la acción (por defecto, el del botón).	<pre> JButton b = new JButton("Guardar"); b.addActionListener(e -> { String c = e.getActionCommand(); System.out.println(c); }); b.doClick(); b.setActionCommand("g1"); b.doClick(); </pre>	Guardar g1
Quitar un oyente	removeActionListener deja de avisarle al oyente.	<pre> JButton b = new JButton("X"); ActionListener l = new Aviso("clic"); b.addActionListener(l); b.doClick(); b.removeActionListener(l); b.doClick(); int n = b.getActionListeners().length; System.out.println(n); </pre>	clic 0
Oyente que cambia otro componente	Dentro del oyente se modifica la interfaz.	<pre> JLabel l = new JLabel("Clics: 0"); JButton b = new JButton("Contar"); int[] n = {0}; b.addActionListener(e -> l.setText("Clics: " + ++n[0])); b.doClick(); b.doClick(); System.out.println(l.getText()); </pre>	Clics: 2

doClick() ejecuta el oyente igual que un clic real del usuario.

Diagramas y ventanas de ejemplo



Más eventos: ratón, teclado, ítems, texto y ventana

Cada tipo de evento tiene su interfaz. Si la interfaz tiene varios métodos, un adapter (MouseListener, KeyAdapter, WindowAdapter) ya los trae vacíos y tú sobrescribes solo el que necesitas. Algunos eventos se pueden crear por código.

RECUERDA Una interfaz es un formulario con muchas casillas obligatorias; un adapter es el mismo formulario ya llenado en blanco.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
import javax.swing.*;
import javax.swing.event.*;
import java.awt.event.*;

class Espia implements DocumentListener {
    public void insertUpdate(DocumentEvent e) {
        System.out.println("insertUpdate");
    }
    public void removeUpdate(DocumentEvent e) {
        System.out.println("removeUpdate");
    }
    public void changedUpdate(DocumentEvent e) {
    }
}
```

INTERFAZ O ADAPTER	DESCRIPCIÓN	EJEMPLO	RESULTADO
MouseListener	Interfaz con 5 métodos: hay que escribirlos todos.	<pre>mouseClicked(MouseEvent e) mousePressed(MouseEvent e) mouseReleased(MouseEvent e) mouseEntered(MouseEvent e) mouseExited(MouseEvent e)</pre>	Si implementas la interfaz, debes escribir los 5 métodos
MouseAdapter	Sobrescribes solo mouseClicked; los demás vienen vacíos.	<pre>class Clic extends MouseAdapter { @Override public void mouseClicked(MouseEvent e) { System.out.println(e.getX() + "," + e.getY()); } }</pre>	15,20
KeyAdapter	keyTyped recibe cada tecla escrita.	<pre>class Tecla extends KeyAdapter { @Override public void keyTyped(KeyEvent e) { char c = e.getKeyChar(); System.out.println(c); } }</pre>	a
ItemListener con JCheckBox	Avisa cuando la casilla cambia de estado.	<pre>JCheckBox c = new JCheckBox("Acepto"); c.addItemListener(e -> { int s = e.getStateChange(); System.out.println(s == ItemEvent.SELECTED); }); c.doClick(); c.doClick();</pre>	true false
ItemListener con JComboBox	Un cambio dispara dos eventos: DESELECTED y SELECTED.	<pre>JComboBox<String> c = new JComboBox<>(new String[] {"Java", "Python"}); c.addItemListener(e -> { int s = e.getStateChange(); if (s == ItemEvent.SELECTED) { System.out.println(e.getItem()); } }); c.setSelectedIndex(1);</pre>	Python
DocumentListener	Interfaz de texto: 3 métodos y sin adapter.	<pre>insertUpdate(DocumentEvent e) removeUpdate(DocumentEvent e) changedUpdate(DocumentEvent e)</pre>	Se escriben siempre los 3 (no existe DocumentAdapter)

INTERFAZ O ADAPTER	DESCRIPCIÓN	EJEMPLO	RESULTADO
Escuchar cambios de texto	getDocument().addDocumentListener(...) detecta cada cambio.	<pre> JTextField t = new JTextField(); t.getDocument().addDocumentListener(new Espia()); t.setText("Ana"); t.setText("Eva"); </pre>	insertUpdate removeUpdate insertUpdate
WindowListener	Interfaz de ventana con 7 métodos.	<pre> windowOpened(WindowEvent e) windowClosing(WindowEvent e) windowClosed(WindowEvent e) windowIconified(WindowEvent e) windowDeiconified(WindowEvent e) windowActivated(WindowEvent e) windowDeactivated(WindowEvent e) </pre>	7 métodos obligatorios si no usas el adapter
WindowAdapter	Solo windowClosing: avisa antes de cerrar la ventana.	<pre> f.addWindowListener(new WindowAdapter() { @Override public void windowClosing(WindowEvent e) { System.out.println("Adiós"); } }); </pre>	Al pulsar la X de la ventana se imprime: Adiós

Los eventos de ratón y teclado se crean aquí por código; en la ventana real los genera el usuario.

Diagramas y ventanas de ejemplo

ItemListener

☒ Quiero recibir el boletín

Boletín: sí

Casilla con ItemListener
El oyente cambia la etiqueta al marcar.

DocumentListener

Caracteres: 12

Campo con DocumentListener
Cuenta los caracteres mientras se escribe.

MouseListener

Clic en (40, 25)

Panel con MouseListener
Muestra dónde se hizo el último clic.

Diálogos y menús

JOptionPane abre diálogos listos: mensaje, pregunta y entrada de texto. Los menús se arman con JMenuBar, JMenu y JMenuItem, y JFileChooser deja elegir archivos. Los diálogos necesitan pantalla; los menús se construyen y leen por código.

RECUERDA Un diálogo es como el mesero que interrumpe para preguntarte algo: la ventana espera tu respuesta antes de seguir.

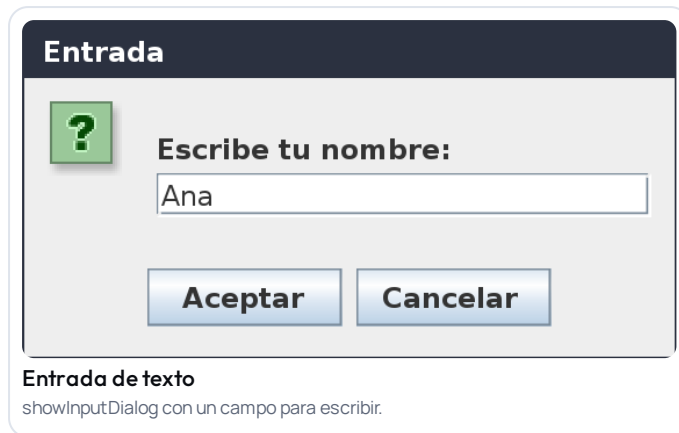
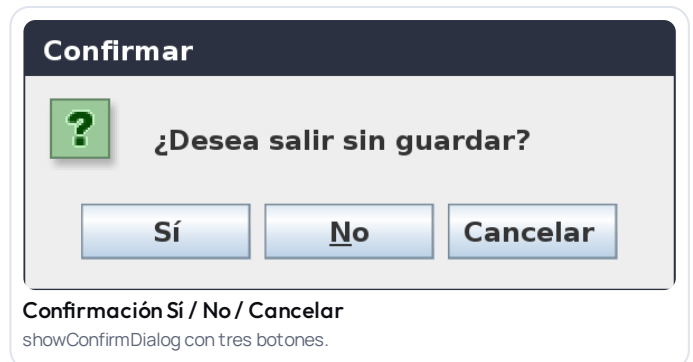
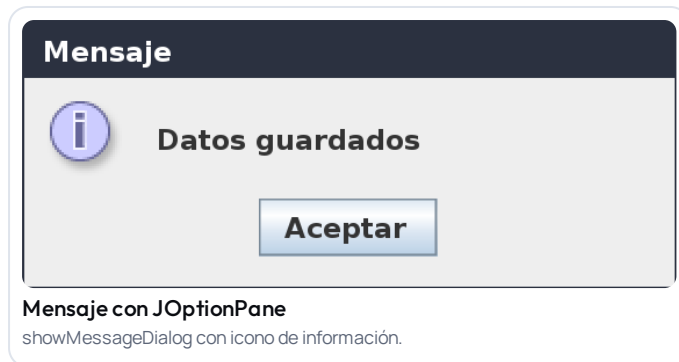
CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
import javax.swing.*;
```

CLASE O MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
showMessageDialog	Muestra un mensaje con un botón Aceptar.	<code>JOptionPane.showMessageDialog(null, "Datos guardados");</code>	Ventana modal con el mensaje y el botón Aceptar
showInputDialog	Pide un texto; devuelve null si se cancela.	<code>String n = JOptionPane.showInputDialog("Nombre:");</code>	Muestra un campo para escribir Devuelve el texto o null
showConfirmDialog	Pregunta Sí / No / Cancelar y devuelve un entero.	<code>int r = JOptionPane.showConfirmDialog(null, "¿Salir?");</code>	Devuelve el botón elegido (ver la fila siguiente)
Valores devueltos	Constantes con las que se compara la respuesta.	<code>int a = JOptionPane.YES_OPTION; int b = JOptionPane.NO_OPTION; int c = JOptionPane.CANCEL_OPTION; int d = JOptionPane.CLOSED_OPTION; System.out.println(a + " " + b); System.out.println(c + " " + d);</code>	0 1 2 -1
JMenuBar, JMenu, JMenuItem	Barra con menús y opciones; se construye y se lee.	<code>JMenuBar bar = new JMenuBar(); JMenu m = new JMenu("Archivo"); m.add(new JMenuItem("Abrir")); m.add(new JMenuItem("Guardar")); bar.add(m); System.out.println(bar.getMenuCount()); System.out.println(m.getItemCount()); JMenuItem i = m.getItem(1); System.out.println(i.getText());</code>	1 2 Guardar
Oyente de un menú	Cada JMenuItem usa ActionListener como un botón.	<code>JMenuItem s = new JMenuItem("Salir"); s.addActionListener(e -> System.out.println("Saliendo")); s.doClick();</code>	Saliendo
Atajo de teclado	setAccelerator asocia una combinación de teclas.	<code>JMenuItem s = new JMenuItem("Guardar"); KeyStroke k = KeyStroke.getKeyStroke("ctrl S"); s.setAccelerator(k); System.out.println(s.getAccelerator());</code>	ctrl pressed S
setJMenuBar	Coloca la barra de menús arriba de la ventana.	<code>f.setJMenuBar(bar);</code>	La barra aparece bajo el título de la ventana
JFileChooser	Diálogo para elegir un archivo del disco.	<code>JFileChooser fc = new JFileChooser(); int r = fc.showOpenDialog(null); if (r == JFileChooser.APPROVE_OPTION) { File f = fc.getSelectedFile(); }</code>	Abre el explorador de archivos APPROVE_OPTION si pulsa Abrir

JOptionPane y JFileChooser abren ventanas: se muestran como imágenes reales en la galería.

Diagramas y ventanas de ejemplo



TEMA 64

Tablas y listas con modelo

JTable y JList muestran datos, pero los datos viven en un modelo: DefaultTableModel y DefaultListModel. Cambias el modelo (addRow, removeRow, addElement) y la vista se actualiza sola.

RECUERDA El modelo es el cuaderno de notas y la tabla es la ventana por donde lo miras: si cambias el cuaderno, cambia lo que ves.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;

class Notas {
    static DefaultTableModel crear() {
        String[] c = {"ID", "Nombre", "Nota"};
        DefaultTableModel m =
            new DefaultTableModel(c, 0);
        m.addRow(new Object[]{"A1", "Ana", 9.5});
        m.addRow(new Object[]{"B2", "Luis", 7.0});
        return m;
    }
}
```

MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
DefaultTableModel 1	Modelo con columnas; se empieza con cero filas.	<pre>String[] c = {"ID", "Nombre", "Nota"}; DefaultTableModel m = new DefaultTableModel(c, 0); System.out.println(m.getColumnCount()); System.out.println(m.getRowCount());</pre>	3 0
addRow	Agrega una fila con un arreglo de objetos.	<pre>String[] c = {"Nombre", "Nota"}; DefaultTableModel m = new DefaultTableModel(c, 0); m.addRow(new Object[]{"Ana", 9.5}); m.addRow(new Object[]{"Luis", 7.0}); System.out.println(m.getRowCount());</pre>	2

MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
getValueAt	Lee una celda: fila y columna (desde 0).	<pre>DefaultTableModel m = Notas.crear(); System.out.println(m.getValueAt(0, 1)); System.out.println(m.getValueAt(1, 2)); System.out.println(m.getColumnName(2));</pre>	Ana 7.0 Nota
setValueAt	Cambia el valor de una celda.	<pre>DefaultTableModel m = Notas.crear(); m.setValueAt(8.0, 1, 2); System.out.println(m.getValueAt(1, 2));</pre>	8.0
removeRow	Elimina una fila por su índice.	<pre>DefaultTableModel m = Notas.crear(); m.removeRow(0); System.out.println(m.getRowCount()); System.out.println(m.getValueAt(0, 1));</pre>	1 Luis
Recorrer el modelo	Un for con <code>getRowCount</code> suma una columna.	<pre>DefaultTableModel m = Notas.crear(); double suma = 0; int n = m.getRowCount(); for (int i = 0; i < n; i++) { suma += (double) m.getValueAt(i, 2); } double prom = suma / n; System.out.println(prom);</pre>	8.25
JTable con modelo	La tabla muestra lo que tenga su modelo.	<pre>DefaultTableModel m = Notas.crear(); JTable t = new JTable(m); System.out.println(t.getModel() == m); System.out.println(t.getRowCount()); Object[] f = {"C3", "Eva", 9.0}; m.addRow(f); System.out.println(t.getRowCount());</pre>	true 2 3
getSelectedRow	Devuelve la fila elegida, o -1 si no hay.	<pre>JTable t = new JTable(Notas.crear()); System.out.println(t.getSelectedRow()); t.setSelectionInterval(1, 1); System.out.println(t.getSelectedRow());</pre>	-1 1
DefaultListModel	Modelo de lista: <code>addElement</code> y <code>removeElement</code> .	<pre>DefaultListModel<String> lm = new DefaultListModel<>(); lm.addElement("Ana"); lm.addElement("Luis"); JList<String> l = new JList<>(lm); lm.removeElement("Ana"); int n = l.getModel().getSize(); System.out.println(n); System.out.println(lm.get(0));</pre>	1 Luis

Notas.crear() es una ayuda con dos filas: Ana (9.5) y Luis (7.0).

Diagramas y ventanas de ejemplo

Notas del curso

ID	Nombre	Nota
A1	Ana	9.5
B2	Luis	7.0
C3	Eva	10.0
D4	Pedro	8.5

JTable con DefaultTableModel
 Tabla con encabezado dentro de un JScrollPane.

Materias

Programación
Física
Cálculo
Álgebra

Elegida: Cálculo

JList con DefaultListModel
 Lista con un elemento seleccionado.

Agregar estudiante

Eva

10

Agregar

Nombre	Nota
Ana	9.5
Luis	7.0
Eva	10

Formulario que agrega filas

Al pulsar el botón se agrega una fila al modelo.

TEMA 65

Dibujo con Graphics

Para dibujar, creas una clase que extiende JPanel y sobrescribes paintComponent(Graphics g). Con g eliges un color (setColor) y dibujas líneas, rectángulos, óvalos y texto. Nunca llamas a paintComponent tú: pides repaint().

RECUERDA

paintComponent es el lienzo y g es tu pincel: cambias de color con setColor y luego dibujas la forma.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```

import java.awt.*;
import java.awt.image.BufferedImage;
import javax.swing.*;

class Pizarra {
    BufferedImage img = new BufferedImage(
        60, 60, BufferedImage.TYPE_INT_RGB);
    Graphics2D g = img.createGraphics();
    Pizarra() {
        g.setColor(Color.WHITE);
        g.fillRect(0, 0, 60, 60);
    }
    String px(int x, int y) {
        int c = img.getRGB(x, y) & 0xFFFFFF;
        return String.format("%06x", c);
    }
}

```

MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
paintComponent	Método donde se dibuja; siempre llama a super primero.	<pre> class Lienzo extends JPanel { protected void paintComponent(Graphics g) { super.paintComponent(g); g.setColor(Color.BLACK); g.drawLine(0, 0, 50, 50); } } </pre>	true
setColor	Define el color de lo que se dibuje después.	<pre> Pizarra z = new Pizarra(); z.g.setColor(Color.RED); z.g.fillRect(10, 10, 30, 30); System.out.println(z.px(20, 20)); System.out.println(z.px(50, 50)); </pre>	ff0000 ffffff
drawLine	Línea entre dos puntos (x1, y1) y (x2, y2).	<pre> Pizarra z = new Pizarra(); z.g.setColor(Color.BLUE); z.g.drawLine(0, 0, 50, 50); System.out.println(z.px(20, 20)); System.out.println(z.px(20, 40)); </pre>	0000ff ffffff

MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
drawRect	Rectángulo solo con el borde (x, y, ancho, alto).	<pre>Pizarra z = new Pizarra(); z.g.setColor(Color.BLUE); z.g.drawRect(10, 10, 30, 30); System.out.println(z.px(10, 25)); System.out.println(z.px(25, 25));</pre>	0000ff ffffff
fillOval	Óvalo relleno dentro de un rectángulo imaginario.	<pre>Pizarra z = new Pizarra(); z.g.setColor(Color.GREEN); z.g.fillOval(10, 10, 40, 40); System.out.println(z.px(30, 30)); System.out.println(z.px(11, 11));</pre>	00ff00 ffffff
drawString	Escribe texto; y marca la línea base de las letras.	<pre>g.setColor(Color.BLACK); g.drawString("Hola", 10, 30);</pre>	Escribe Hola con su base en el punto (10, 30)
Color RGB	Un color se arma con rojo, verde y azul (0 a 255).	<pre>Color c = new Color(0, 128, 255); System.out.println(c.getRed()); System.out.println(c.getGreen()); System.out.println(c.getBlue());</pre>	0 128 255
Graphics2D	Versión avanzada: trazos gruesos y suavizado.	<pre>Graphics2D g2 = (Graphics2D) g; g2.setStroke(new BasicStroke(4f)); g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);</pre>	Las líneas salen gruesas (4 px) y sin dientes de sierra
repaint()	Pide volver a pintar; Swing llama paintComponent.	<pre>x = x + 10; panel.repaint();</pre>	Swing vuelve a ejecutar paintComponent con x nuevo

Pizarra dibuja sobre una imagen en memoria; px(x, y) da el color de un pixel (hex RGB).

Diagramas y ventanas de ejemplo

Formas básicas

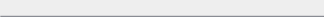
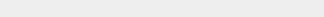
Formas básicas
drawLine, drawRect, fillRect, drawOval y fillOval.

Casa

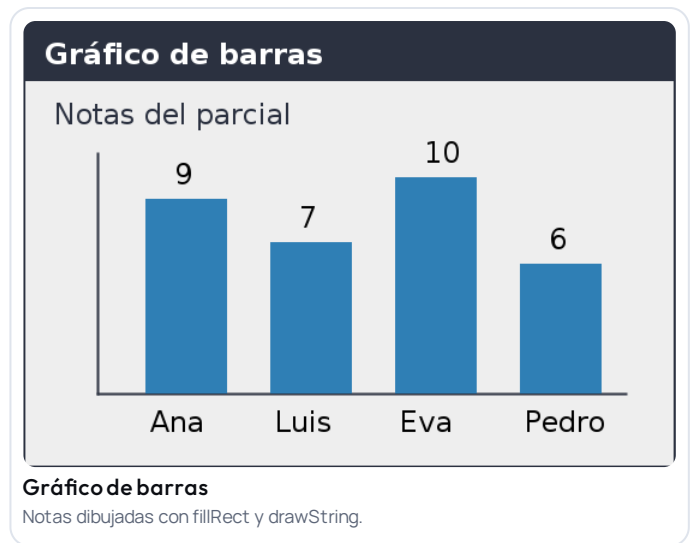
Una casa con formas
Rectángulo, triángulo, círculo y texto.

Graphics2D



 grosor 2
 grosor 4
 grosor 6
 grosor 8
 new Color(0, 128, 255)

Colores y trazos con Graphics2D
Colores RGB, BasicStroke y degradado.



TEMA 66

Arquitectura de una aplicación GUI: MVC

El patrón MVC separa la aplicación en tres partes: el modelo guarda los datos y las reglas, la vista son los componentes de la ventana y el controlador conecta ambos con oyentes. Así la lógica se prueba sin ventana. Y la GUI nunca se bloquea.

RECUERDA Es un restaurante: el modelo es la cocina, la vista es el comedor y el controlador es el mesero que lleva los pedidos.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

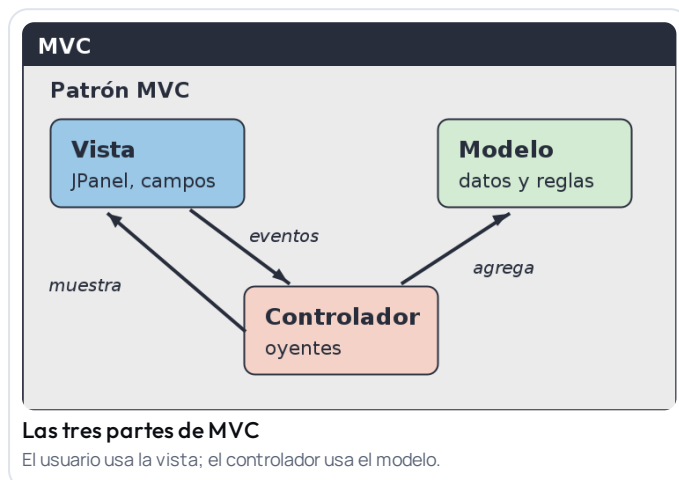
```
import javax.swing.*;
record Alumno(String nombre, int nota) {}
class Modelo {
    ArrayList<Alumno> lista =
        new ArrayList<>();
    boolean agregar(String n, int x) {
        return x >= 0 && x <= 10
            && lista.add(new Alumno(n, x));
    }
}
class Vista extends JPanel {
    JTextField nombre = new JTextField(6);
    JTextField nota = new JTextField(3);
    JButton btn = new JButton("Agregar");
    JTextArea lista = new JTextArea(4, 14);
    Vista() {
        add(new JLabel("Nombre:"));
        add(nombre);
        add(new JLabel("Nota:"));
        add(nota);
        add(btn);
        add(new JScrollPane(lista));
    }
}
```

PARTE O BUENA PRÁCTICA	DESCRIPCIÓN	EJEMPLO			RESULTADO
Las tres partes	Cada clase tiene una sola responsabilidad.	Modelo	Vista	Controlador	Modelo: datos y reglas Vista: ventana y campos Controlador: oyentes que los unen
Modelo	No usa Swing; valida y guarda. Se prueba sin ventana.	<pre>Modelo m = new Modelo(); boolean a = m.agregar("Ana", 9); boolean b = m.agregar("Eva", 15); System.out.println(a + " " + b); System.out.println(m.lista.size());</pre>			true false 1

PARTE O BUENA PRÁCTICA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Alumno (record)	Dato inmutable del modelo; imprime sus campos.	<pre>Alumno a = new Alumno("Ana", 9); System.out.println(a); System.out.println(a.nota());</pre>	Alumno[nombre=Ana, nota=9] 9
Vista	Solo crea y organiza los componentes.	<pre>Vista v = new Vista(); int n = v.getComponentCount(); System.out.println(n); v.nombre.setText("Ana"); System.out.println(v.nombre.getText());</pre>	6 Ana
Controlador	Un oyente lee la vista, llama al modelo y actualiza la vista.	<pre>static void enlazar(Modelo m, Vista v) { v.btn.addActionListener(e -> { String n = v.nombre.getText(); int x = Integer.parseInt(v.nota.getText()); v.lista.append(m.agregar(n, x) ? n + "\n" : "Error\n"); }); }</pre>	1 guardado Ana Error
Validar la entrada	El controlador atrapa texto que no es un número.	<pre>try { Integer.parseInt("abc"); } catch (NumberFormatException e) { System.out.println("Nota inválida"); }</pre>	Nota inválida
GUI desde otro hilo	Un hilo secundario cambia la GUI con invokeLater.	<pre>JLabel l = new JLabel("0"); Thread t = new Thread(() -> SwingUtilities.invokeLater(() -> l.setText("listo"))); t.start(); t.join(); SwingUtilities.invokeAndWait(() -> { }); System.out.println(l.getText());</pre>	listo
No bloquear la GUI	Una tarea larga en el oyente congela la ventana.	<pre>btn.addActionListener(e -> { Thread.sleep(5000); etiqueta.setText("Listo"); });</pre>	La ventana se congela 5 s: no responde a clics ni se repinta
SwingWorker	Trabaja en segundo plano y entrega el resultado.	<pre>SwingWorker<Integer, Void> w = new SwingWorker<>() { protected Integer doInBackground() { return 6 * 7; } }; w.execute(); System.out.println(w.get());</pre>	42

Ejemplo: formulario de estudiante. El modelo acepta notas de 0 a 10; la vista no sabe nada de reglas.

Diagramas y ventanas de ejemplo



Registro de estudiantes

Nombre: Nota:

Ana Mora - 9
Luis Paz - 7
Nota inválida

Formulario del estudiante

Vista con dos alumnos agregados por el controlador.

Introducción a la programación concurrente

Hacer varias cosas a la vez: hilos, sincronización y pools.

TEMA 67

Concurrencia: procesos e hilos

Un proceso es un programa en ejecución con su propia memoria. Dentro de él viven uno o más hilos (threads), que comparten esa memoria. Todo programa Java empieza con un hilo: el hilo principal main.

RECUERDA Un restaurante es un proceso; los cocineros son hilos: comparten la misma cocina y trabajan a la vez.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

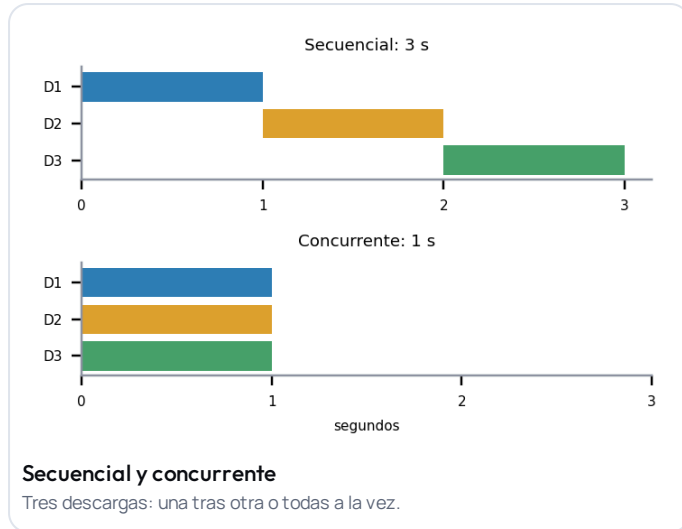
```
class Util {
    static void dormir(long ms) {
        try {
            Thread.sleep(ms);
        } catch (InterruptedException e) {
            return;
        }
    }
}
```

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Proceso	Programa en ejecución, con memoria propia.	Proceso = programa en ejecución	Ej: Chrome, IntelliJ, tu programa Java. Cada uno con su memoria.
Hilo (thread)	Flujo de trabajo dentro de un proceso.	Hilo = tarea dentro del proceso	Un proceso tiene 1 o más hilos que comparten su memoria.
Hilo principal	main es el primer hilo de todo programa.	<code>Thread yo = Thread.currentThread();</code> <code>System.out.println(yo.getName());</code>	main
Nombre de un hilo	Se puede dar un nombre al crearlo.	<code>Thread t = new Thread(() -> {}, "caja");</code> <code>System.out.println(t.getName());</code>	caja
Otro hilo trabaja	Cada hilo ejecuta su propio código.	<code>Thread t = new Thread(() -> {</code> <code>Thread yo = Thread.currentThread();</code> <code>System.out.println(yo.getName());</code> <code>}, "cajero-1");</code> <code>t.start();</code> <code>t.join();</code> <code>System.out.println("main sigue");</code>	cajero-1 main sigue
Tarea lenta aparte	Un hilo calcula mientras main sigue libre.	<code>String[] r = new String[1];</code> <code>Thread t = new Thread(() -> {</code> <code>Util.dormir(30);</code> <code>r[0] = "reporte listo";</code> <code>});</code> <code>t.start();</code> <code>t.join();</code> <code>System.out.println(r[0]);</code>	reporte listo
Para qué sirve	Ventanas que responden y tareas lentas aparte.	Hilo 1: la ventana (clics) Hilo 2: descarga del archivo	La ventana no se congela mientras otro hilo descarga o calcula.
Secuencial vs concurrente	Tres descargas de 1 s cada una.	3 descargas seguidas -> 3 s 3 descargas a la vez -> 1 s	Secuencial: 3 s Concurrente: 1 s aprox. (si el equipo tiene varios núcleos)

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Núcleos del equipo	Cuántos hilos pueden correr de verdad a la vez.	<pre>Runtime rt = Runtime.getRuntime(); int n = rt.availableProcessors(); System.out.println(n >= 1);</pre>	true

Concurrente: varias tareas en progreso. Paralelo: varias tareas ejecutándose en el mismo instante (varios núcleos).

Diagramas y ventanas de ejemplo



TEMA 68

Crear hilos

Hay tres formas de crear un hilo: extender Thread, implementar Runnable o pasar una lambda. El trabajo del hilo va en run(), pero se arranca con start(): start() crea el hilo nuevo y run() solo sería una llamada normal.

RECUERDA start() es tocar el timbre para que llegue un cocinero nuevo; run() es cocinar tú mismo en la cocina.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class Util {
    static String yo() {
        Thread t = Thread.currentThread();
        return t.getName();
    }
}
```

FORMA	DESCRIPCIÓN	EJEMPLO	RESULTADO
extends Thread	Heredar de Thread y sobrescribir run().	<pre>class Descarga extends Thread { public void run() { System.out.println(getName()); } }</pre>	bajador fin
implements Runnable	La tarea es un Runnable; el hilo la ejecuta.	<pre>class Cajero implements Runnable { public void run() { System.out.println(Util.yo()); } }</pre>	caja-1
Lambda	Runnable corto: la forma más usada hoy.	<pre>Thread t = new Thread(() -> System.out.println("cobrando")); t.start(); t.join();</pre>	cobrando
run() directo	Ejecuta en el hilo actual: NO crea hilo nuevo.	<pre>Runnable r = () -> System.out.println(Util.yo()); Thread t = new Thread(r, "extra"); t.run();</pre>	main

FORMA	DESCRIPCIÓN	EJEMPLO	RESULTADO
start()	Crea el hilo nuevo y ejecuta run() allí.	<pre>Runnable r = () -> System.out.println(Util.yo()); Thread t = new Thread(r, "extra"); t.start(); t.join();</pre>	extra
Un Runnable, dos hilos	La misma tarea la pueden correr varios hilos.	<pre>Runnable r = () -> System.out.println(Util.yo()); Thread a = new Thread(r, "A"); Thread b = new Thread(r, "B"); a.start(); a.join(); b.start(); b.join();</pre>	A B
start() dos veces	Un hilo solo se arranca una vez.	<pre>Thread t = new Thread(() -> {}); t.start(); t.join(); t.start();</pre>	IllegalThreadStateException : null
Varios hilos, un arreglo	Cada hilo escribe en su propia casilla.	<pre>int[] r = new int[3]; Thread[] hs = new Thread[3]; for (int i = 0; i < 3; i++) { int k = i; hs[i] = new Thread(() -> r[k] = k); hs[i].start(); } for (Thread h : hs) h.join(); System.out.println(Arrays.toString(r));</pre>	[0, 1, 2]
Hilo virtual (Java 21)	Hilos muy livianos para muchas tareas.	<pre>Runnable r = () -> System.out.println("virtual"); Thread t = Thread.ofVirtual().start(r); t.join();</pre>	virtual

Siempre usa start(). El resultado de estas filas no cambia porque cada hilo se espera con join().

TEMA 69

Ciclo de vida y control de hilos

Un hilo pasa por estados: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING y TERMINATED. Con sleep() pausa, con join() otro hilo lo espera, con interrupt() se le pide parar y con setDaemon(true) no impide cerrar el programa.

RECUERDA join() es esperar a tu amigo en la puerta: no sigues hasta que él llegue. interrupt() es tocarle el hombro.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

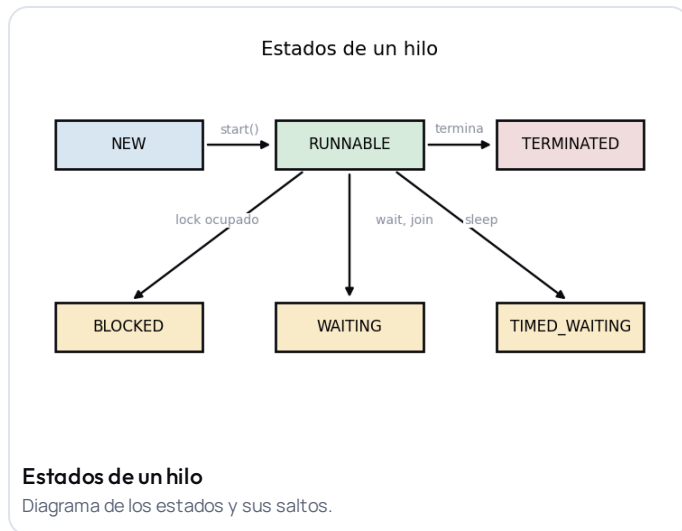
```
import java.lang.Thread.State;
class Util {
    static void dormir(long ms) {
        try {
            Thread.sleep(ms);
        } catch (InterruptedException e) {
            return;
        }
    }
}
```

MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Los 6 estados	Thread.State los enumera.	<pre>for (State s : State.values()) { System.out.println(s); }</pre>	NEW RUNNABLE BLOCKED WAITING TIMED_WAITING TERMINATED

MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
getState(): NEW y fin	Antes de start() es NEW; al terminar, TERMINATED.	<pre>Thread t = new Thread() -> {}; System.out.println(t.getState()); t.start(); t.join(); System.out.println(t.getState());</pre>	NEW TERMINATED
TIMED_WAITING	Dentro de sleep() el hilo está en espera con tiempo.	<pre>Runnable r = () -> Util.dormir(300); Thread t = new Thread(r); t.start(); State s = t.getState(); while (s == State.RUNNABLE) { s = t.getState(); } System.out.println(s); t.join();</pre>	TIMED_WAITING
sleep() + join()	main espera a que el hilo termine su trabajo.	<pre>int[] total = {0}; Runnable r = () -> { Util.dormir(30); total[0] = 100; }; Thread t = new Thread(r); t.start(); t.join(); System.out.println(total[0]);</pre>	100
Olvidar join()	Sin join(), main puede leer antes de tiempo.	<pre>int[] total = {0}; Runnable r = () -> total[0] = 100; Thread t = new Thread(r); t.start(); System.out.println(total[0]);</pre>	Puede imprimir 0 o 100: depende de quién llegue primero. Siempre usa join().
join(ms) e isAlive()	Espera como máximo ms; isAlive() dice si sigue vivo.	<pre>Runnable r = () -> Util.dormir(5000); Thread t = new Thread(r); t.start(); t.join(50); System.out.println(t.isAlive()); t.interrupt(); t.join(); System.out.println(t.isAlive());</pre>	true false
setDaemon(true)	Un hilo daemon no mantiene vivo el programa.	<pre>Runnable r = () -> Util.dormir(5000); Thread t = new Thread(r); t.setDaemon(true); t.start(); System.out.println(t.isDaemon()); System.out.println(Thread.currentThread().isDaemon()); t.interrupt(); t.join();</pre>	true false
interrupt() en sleep()	Despierta al hilo con InterruptedException.	<pre>Thread t = new Thread() -> { try { Thread.sleep(5000); } catch (InterruptedException e) { System.out.println("despierto"); } }; t.start(); t.interrupt(); t.join();</pre>	despierto
interrupt() en un ciclo	El hilo revisa la marca y se detiene solo.	<pre>Thread t = new Thread() -> { while (!Thread.interrupted()) { Thread.onSpinWait(); } System.out.println("detenido"); }; t.start(); t.interrupt(); t.join();</pre>	detenido

Regla de oro de estas filas: todo hilo que se arranca se espera con join() antes de terminar.

Diagramas y ventanas de ejemplo



TEMA 70

Condiciones de carrera

Hay una condición de carrera cuando varios hilos leen y modifican el mismo dato a la vez y el resultado depende de quién llegue primero. `valor++` parece un paso, pero son tres: leer, sumar y escribir; por eso se pierden actualizaciones.

RECUERDA Dos personas anotan a la vez en la misma hoja: ambas leen 5, ambas escriben 6. Se perdió una suma.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
class Contador {
    int valor = 0;
    void incrementar() {
        valor++;
    }
}

class Hilos {
    static void correr(int n, Runnable r)
        throws Exception {
        Thread[] hs = new Thread[n];
        for (int i = 0; i < n; i++) {
            hs[i] = new Thread(r);
            hs[i].start();
        }
        for (Thread h : hs) {
            h.join();
        }
    }
}
```

SITUACIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
Qué es una carrera	Resultado que depende del orden de los hilos.	Hay carrera si: 1. varios hilos 2. comparten un dato 3. alguno lo modifica	Las tres condiciones juntas generan resultados impredecibles.
valor++ son 3 pasos	No es atómico: leer, sumar, escribir.	<code>valor++;</code>	1. leer valor (5) 2. calcular 5 + 1 3. escribir 6 Otro hilo puede meterse en medio.
Contador sin proteger	Dos hilos suman 100000 veces cada uno.	<pre>Contador c = new Contador(); Hilos.correr(2, () -> { for (int i = 0; i < 100000; i++) { c.incrementar(); } }); System.out.println("esperado: 200000"); System.out.println("real: " + c.valor);</pre>	esperado: 200000 real: 143582 Otra ejecución dará otro valor (casi nunca 200000).

SITUACIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
Nunca sobran sumas	Se pierden sumas, pero jamás aparecen de más.	<pre>Contador c = new Contador(); Hilos.correr(2, () -> { for (int i = 0; i < 100000; i++) { c.incrementar(); } }); System.out.println(c.valor <= 200000);</pre>	true
Actualización perdida	Así se pierde una suma entre dos hilos.	valor = 5 al inicio	Hilo A lee 5 Hilo B lee 5 Hilo A escribe 6 Hilo B escribe 6 Resultado: 6 (debía ser 7)
Dos cajeros, una cuenta	Revisar y luego actuar también es una carrera.	<pre>if (saldo >= 80) { // el otro hilo retira aquí saldo = saldo - 80; }</pre>	Saldo 100; A y B retiran 80. Ambos ven saldo >= 80. Saldo final: -60
No compartir nada	Cada hilo escribe su casilla; main suma al final.	<pre>int[] p = new int[2]; Thread a = new Thread(() -> p[0] = 500); Thread b = new Thread(() -> p[1] = 700); a.start(); b.start(); a.join(); b.join(); System.out.println(p[0] + p[1]);</pre>	1200
Solución: synchronized	Solo un hilo a la vez entra al método.	<pre>class ContadorSeguro { int valor = 0; synchronized void incrementar() { valor++; } }</pre>	200000

Hilos.correr(n, r) crea n hilos con la tarea r, los arranca y espera a todos con join().

Sincronización: synchronized, volatile y locks

synchronized permite que un solo hilo a la vez entre a un método o bloque (cada objeto tiene un cerrojo). volatile hace que todos vean el último valor de una variable. ReentrantLock es un cerrojo explícito con lock() y unlock().

RECUERDA synchronized es el candado del baño: entra uno, los demás esperan. Con try/finally siempre se abre al salir.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
class Hilos {
    static void correr(int n, Runnable r)
        throws Exception {
        Thread[] hs = new Thread[n];
        for (int i = 0; i < n; i++) {
            hs[i] = new Thread(r);
            hs[i].start();
        }
        for (Thread h : hs) {
            h.join();
        }
    }
}
```

HERRAMIENTA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Método synchronized	Un hilo a la vez ejecuta el método.	<pre>class Contador { private int valor; synchronized void sumar() { valor++; } synchronized int get() { return valor; } }</pre>	40000
Bloque synchronized	Protege solo las líneas críticas con un candado.	<pre>class Cuenta { int saldo; final Object candado = new Object(); void depositar(int m) { synchronized (candado) { saldo += m; } } }</pre>	40000
volatile	Todos los hilos ven el cambio de la bandera.	<pre>class Vigia extends Thread { volatile boolean activo = true; void detener() { activo = false; } public void run() { while (activo) { } System.out.println("detenido"); } }</pre>	detenido
volatile no basta para ++	Asegura ver el valor, no hace atómicas las sumas.	<pre>volatile int total; total++;</pre>	volatile: todos VEN el valor. total++ sigue siendo 3 pasos: usa synchronized o Atomic.
ReentrantLock	lock() y unlock() dentro de try/finally.	<pre>class Caja { Lock lock = new ReentrantLock(); int total; void cobrar(int m) { lock.lock(); try { total += m; } finally { lock.unlock(); } } }</pre>	20000

HERRAMIENTA	DESCRIPCIÓN	EJEMPLO	RESULTADO
tryLock()	Intenta tomar el cerrojo sin quedarse esperando.	<pre>Lock lock = new ReentrantLock(); lock.lock(); boolean[] ok = new boolean[1]; Thread t = new Thread(() -> ok[0] = lock.tryLock()); t.start(); t.join(); System.out.println("otro: " + ok[0]); lock.unlock();</pre>	otro: false
Reentrancia	Un hilo que ya tiene el cerrojo puede volver a entrar.	<pre>class Demo { synchronized void a() { b(); } synchronized void b() { System.out.println("b entra"); } }</pre>	b entra
Retiro sin sobregiro	revisar y restar dentro del mismo synchronized.	<pre>class Cuenta { int saldo = 100; synchronized boolean sacar(int m) { if (saldo < m) { return false; } saldo -= m; return true; } }</pre>	retiros ok: 3 saldo: 10

Con 5 hilos que retiran 30 de una cuenta de 100, solo 3 retiros pasan: el resultado es exacto en cada ejecución.

TEMA 72

Atomic y colecciones concurrentes

AtomicInteger suma de forma segura sin synchronized. Las colecciones de java.util.concurrent (ConcurrentHashMap, CopyOnWriteArrayList, BlockingQueue) se pueden usar desde varios hilos. BlockingQueue permite el patrón productor-consumidor.

RECUERDA Una BlockingQueue es la banda de una caja: el productor deja productos, el consumidor los toma, y si está vacía espera.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
class Hilos {
    static void correr(int n, Runnable r)
        throws Exception {
        Thread[] hs = new Thread[n];
        for (int i = 0; i < n; i++) {
            hs[i] = new Thread(r);
            hs[i].start();
        }
        for (Thread h : hs) {
            h.join();
        }
    }
}
```

CLASE / MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
incrementAndGet()	Suma 1 de forma atómica, sin perder sumas.	<pre>AtomicInteger n = new AtomicInteger(); Hilos.correr(4, () -> { for (int i = 0; i < 10000; i++) { n.incrementAndGet(); } }); System.out.println(n.get());</pre>	40000

CLASE / MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Otros métodos atómicos	getAndIncrement, addAndGet, compareAndSet.	<pre>AtomicInteger a = new AtomicInteger(5); System.out.println(a.getAndIncrement()); System.out.println(a.addAndGet(10)); boolean ok = a.compareAndSet(16, 0); System.out.println(ok); System.out.println(a.get());</pre>	5 16 true 0
ConcurrentHashMap.p.merge	Cuenta ventas por producto desde varios hilos.	<pre>ConcurrentMap<String, Integer> m = new ConcurrentHashMap<>(); Hilos.correr(4, () -> { for (int i = 0; i < 1000; i++) { m.merge("pan", 1, Integer::sum); } }); System.out.println(m);</pre>	{pan=4000}
putIfAbsent	Guarda solo si la clave no existe (atómico).	<pre>ConcurrentMap<String, Integer> m = new ConcurrentHashMap<>(); Integer r = m.putIfAbsent("pan", 5); System.out.println(r); r = m.putIfAbsent("pan", 9); System.out.println(r); System.out.println(m);</pre>	null 5 {pan=5}
HashMap con hilos	HashMap no es seguro entre hilos.	<pre>var m = new HashMap<String, Integer>(); // varios hilos hacen m.put(...)</pre>	Puede perder datos o fallar. Usa ConcurrentHashMap.
ArrayList y su falla	Modificarla mientras se recorre lanza excepción.	<pre>var l = new ArrayList<>(List.of(1, 2)); for (int x : l) { l.add(x * 10); }</pre>	ConcurrentModificationException: null
CopyOnWriteArrayList	Recorre una copia: se puede modificar sin falla.	<pre>var b = List.of(1, 2); var l = new CopyOnWriteArrayList<>(b); for (int x : l) { l.add(x * 10); } System.out.println(l);</pre>	[1, 2, 10, 20]
BlockingQueue	put/take esperan si está llena o vacía.	<pre>BlockingQueue<String> q = new ArrayBlockingQueue<>(2); q.put("a"); q.put("b"); System.out.println(q.offer("c")); System.out.println(q.take()); System.out.println(q.poll()); System.out.println(q.poll());</pre>	false a b null
Productor-consumidor	El 0 avisa que ya no hay más productos.	<pre>BlockingQueue<Integer> q = new LinkedBlockingQueue<>(); Thread prod = new Thread(() -> q.addAll(List.of(10, 20, 30, 0))); prod.start(); int suma = 0, x; while ((x = q.take()) != 0) suma += x; prod.join(); System.out.println(suma);</pre>	60

Todas las filas usan `join()`: el resultado es el mismo en cada ejecución.

ExecutorService y pools de hilos

En vez de crear hilos a mano, se entregan tareas a un `ExecutorService`, que reutiliza un grupo de hilos (pool). `submit()` devuelve un `Future` con el resultado de un `Callable`. Al final se llama `shutdown()` y `awaitTermination()`.

RECUERDA Un pool es una empresa con 3 empleados: tú entregas encargos, ellos los reparten y te dan un recibo (`Future`).

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
class Util {
    static void dormir(long ms) {
        try {
            Thread.sleep(ms);
        } catch (InterruptedException e) {
            return;
        }
    }
}
```

MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Executors	Fábrica de pools; <code>shutdown()</code> cierra el pool.	<pre>ExecutorService pool = Executors.newFixedThreadPool(3); System.out.println(pool.isShutdown()); pool.shutdown(); System.out.println(pool.isShutdown());</pre>	false true
execute()	Corre tareas <code>Runnable</code> ; espera el final del pool.	<pre>var p = Executors.newFixedThreadPool(4); var caja = new AtomicInteger(100); for (int i = 0; i < 50; i++) { p.execute(() -> caja.addAndGet(10)); } p.shutdown(); p.awaitTermination(2, TimeUnit.SECONDS); System.out.println(caja);</pre>	600
submit() y Future	<code>get()</code> espera y entrega el resultado.	<pre>var p = Executors.newFixedThreadPool(2); var f = p.submit(() -> 6 * 7); System.out.println(f.get()); System.out.println(f.isDone()); p.shutdown();</pre>	42 true
Callable	Tarea que devuelve un valor (<code>Runnable</code> no).	<pre>Callable<String> tarea = () -> "listo"; var p = Executors.newFixedThreadPool(1); Future<String> f = p.submit(tarea); System.out.println(f.get()); p.shutdown();</pre>	listo
invokeAll()	Lanza varias tareas; los <code>Future</code> salen en orden.	<pre>var p = Executors.newFixedThreadPool(3); List<Callable<Integer>> ts = List.of(() -> 120, () -> 80, () -> 50); for (var f : p.invokeAll(ts)) { System.out.println(f.get()); } p.shutdown();</pre>	120 80 50
ExecutionException	<code>get()</code> entrega el error que lanzó la tarea.	<pre>var p = Executors.newFixedThreadPool(1); var f = p.submit(() -> 10 / (1 - 1)); try { f.get(); } catch (ExecutionException e) { Throwable c = e.getCause(); System.out.println(c.getMessage()); } p.shutdown();</pre>	/ by zero

MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Pool cerrado	Después de shutdown() ya no acepta tareas.	<pre>var p = Executors.newFixedThreadPool(1); p.shutdown(); try { p.submit(() -> 1); } catch (RejectedExecutionException e) { System.out.println("pool cerrado"); }</pre>	pool cerrado
shutdownNow()	Interrumpe las tareas en curso.	<pre>var p = Executors.newFixedThreadPool(1); p.execute(() -> Util.dormir(5000)); p.shutdownNow(); boolean fin = p.awaitTermination(1, TimeUnit.SECONDS); System.out.println(fin);</pre>	true
Un solo hilo	Las tareas salen una tras otra, en orden.	<pre>var p = Executors.newFixedThreadPool(1); List<String> log = new ArrayList<>(); p.execute(() -> log.add("abrir")); p.execute(() -> log.add("cobrar")); p.execute(() -> log.add("cerrar")); p.shutdown(); p.awaitTermination(1, TimeUnit.SECONDS); System.out.println(log);</pre>	[abrir, cobrar, cerrar]

En las filas, *p* es el pool (`Executors.newFixedThreadPool()`). Se cierra siempre con `shutdown()`.

TEMA 74

CompletableFuture e hilos con GUI

`CompletableFuture` encadena pasos asíncronos: `supplyAsync()` calcula en otro hilo, `thenApply()` transforma, `thenCombine()` une dos resultados y `join()` espera. En Swing, la ventana vive en el hilo de eventos (EDT): el trabajo lento va en un `SwingWorker`.

RECUERDA `CompletableFuture` es una receta en cadena: cuando termina un paso, pasa solo al siguiente. El EDT es el mesero: no lo mandes a cocinar.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

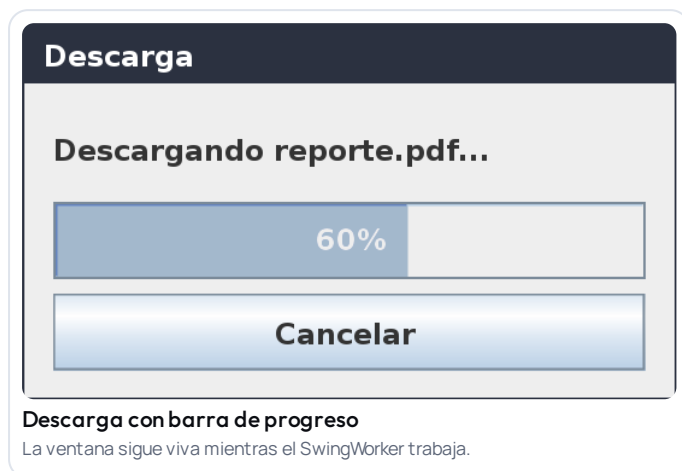
```
import java.util.concurrent.*;
import javax.swing.SwingUtilities;
import javax.swing.SwingWorker;
import javax.swing.JFrame;
```

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
supplyAsync + join	Calcula en otro hilo; <code>join()</code> espera el valor.	<pre>var f = CompletableFuture .supplyAsync(() -> 21); System.out.println(f.join());</pre>	21
thenApply	Transforma el resultado cuando esté listo.	<pre>var f = CompletableFuture .supplyAsync(() -> 100) .thenApply(p -> p * 112 / 100); System.out.println(f.join());</pre>	112
thenCombine	Une los resultados de dos tareas independientes.	<pre>var precio = CompletableFuture .supplyAsync(() -> 30); var envio = CompletableFuture .supplyAsync(() -> 4); var total = precio.thenCombine(envio, (p, e) -> p + e); System.out.println(total.join());</pre>	34
allOf	Espera a que terminen todas las tareas.	<pre>var a = CompletableFuture .supplyAsync(() -> 5); var b = CompletableFuture .supplyAsync(() -> 7); CompletableFuture.allOf(a, b).join(); System.out.println(a.join() + b.join());</pre>	12

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
exceptionally	Da un valor de respaldo si la tarea falla.	<pre>var f = CompletableFuture .supplyAsync() -> 10 / (1 - 1)) .exceptionally(e -> -1); System.out.println(f.join());</pre>	-1
El hilo de la GUI (EDT)	invokeAndWait corre código dentro del EDT.	<pre>SwingUtilities.invokeLaterAndWait(() -> { boolean edt = SwingUtilities .isEventDispatchThread(); System.out.println("EDT: " + edt); });</pre>	EDT: true
GUI congelada	Un ciclo largo en el listener bloquea la ventana.	<pre>boton.addActionListener(e -> { for (int i = 0; i < 1000000; i++) { descargar(); } });</pre>	El EDT queda ocupado: la ventana no repinta ni atiende clics.
invokeLater	Crea la ventana en el EDT, no en main.	<pre>SwingUtilities.invokeLater(() -> { JFrame f = new JFrame("Tienda"); f.setSize(300, 200); f.setVisible(true); });</pre>	Abre una ventana de 300 x 200 creada dentro del hilo de eventos.
SwingWorker	doInBackground trabaja aparte; done() actualiza la GUI.	<pre>new SwingWorker<String, Void>() { protected String doInBackground() { return "archivo.zip"; } protected void done() { etiqueta.setText("Listo"); } }.execute();</pre>	doInBackground: hilo aparte done: de vuelta en el EDT (la ventana nunca se congela)

join() espera el valor sin declarar excepciones comprobadas; en las filas de la GUI el código solo se muestra.

Diagramas y ventanas de ejemplo



Problemas clásicos: deadlock, starvation y wait/notify

Un deadlock ocurre cuando dos hilos se esperan mutuamente para siempre. Se evita bloqueando siempre en el mismo orden. Starvation es que un hilo nunca consigue turno. `wait()` y `notifyAll()` permiten que los hilos se avisen entre sí.

RECUERDA Deadlock: dos autos en un puente angosto, cada uno esperando que el otro retroceda. La regla es pasar siempre en el mismo orden.

CLASES DE APOYO (YA EXISTEN EN TODOS LOS EJEMPLOS)

```
import java.lang.management.ManagementFactory;
import java.util.concurrent.locks.*;
class Buzon {
    private String dato;
    private void esperar() {
        try { wait(); }
        catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
    }
    synchronized void poner(String d) {
        while (dato != null) { esperar(); }
        dato = d;
        notifyAll();
    }
    synchronized String tomar() {
        while (dato == null) { esperar(); }
        String r = dato;
        dato = null;
        notifyAll();
        return r;
    }
}
```

PROBLEMA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Deadlock	Dos hilos toman dos cerrojos en orden contrario.	<pre>// Hilo A synchronized (x) { synchronized (y) { ... } } // Hilo B synchronized (y) { synchronized (x) { ... } }</pre>	A tiene x y espera y B tiene y y espera x Nadie avanza: el programa se cuelga.
Detectar deadlock	ThreadMXBean avisa si hay hilos trabados.	<pre>var mx = ManagementFactory .getThreadMXBean(); long[] ids = mx.findDeadlockedThreads(); System.out.println(ids == null);</pre>	true
Orden de cerrojos	Se bloquea primero la cuenta de menor id.	<pre>static void mover(Cuenta x, Cuenta y) { Cuenta p = x.id < y.id ? x : y; Cuenta s = p == x ? y : x; synchronized (p) { synchronized (s) { x.saldo--; y.saldo++; } } }</pre>	800 1200
Starvation	Un hilo nunca consigue el cerrojo o la CPU.	<pre>while (true) { synchronized (recurso) { ... } }</pre>	Hilos muy "golosos" acaparan el recurso; otro hilo espera sin fin.
Lock justo (fair)	ReentrantLock(true) atiende en orden de llegada.	<pre>var lock = new ReentrantLock(true); System.out.println(lock.isFair());</pre>	true

PROBLEMA	DESCRIPCIÓN	EJEMPLO	RESULTADO
LiveLock	Los hilos se mueven, pero sin avanzar.	A cede el paso a B B cede el paso a A	Ambos cambian de estado sin parar, pero nadie termina su tarea.
wait() sin synchronized	wait() exige tener el cerrojo del objeto.	<code>Object lock = new Object(); lock.wait();</code>	<code>IllegalMonitorStateException: current thread is not owner</code>
Productor-consumidor	Un buzón de 1 casilla con wait/notifyAll.	<code>Buzon b = new Buzon(); Thread prod = new Thread(() -> List.of("pan", "sal", "fin") .forEach(b::poner)); prod.start(); for (int i = 0; i < 3; i++) { System.out.println(b.tomar()); } prod.join();</code>	pan sal fin
while, no if	Tras despertar, hay que volver a revisar la condición.	<code>while (dato == null) { wait(); }</code>	Con while se revisa otra vez: el despertar puede ser falso o lo ganó otro hilo.

Cuenta (usada en Orden de cerrojos) tiene los campos id y saldo. Buzon guarda un solo dato a la vez.

Diagramas y ventanas de ejemplo

